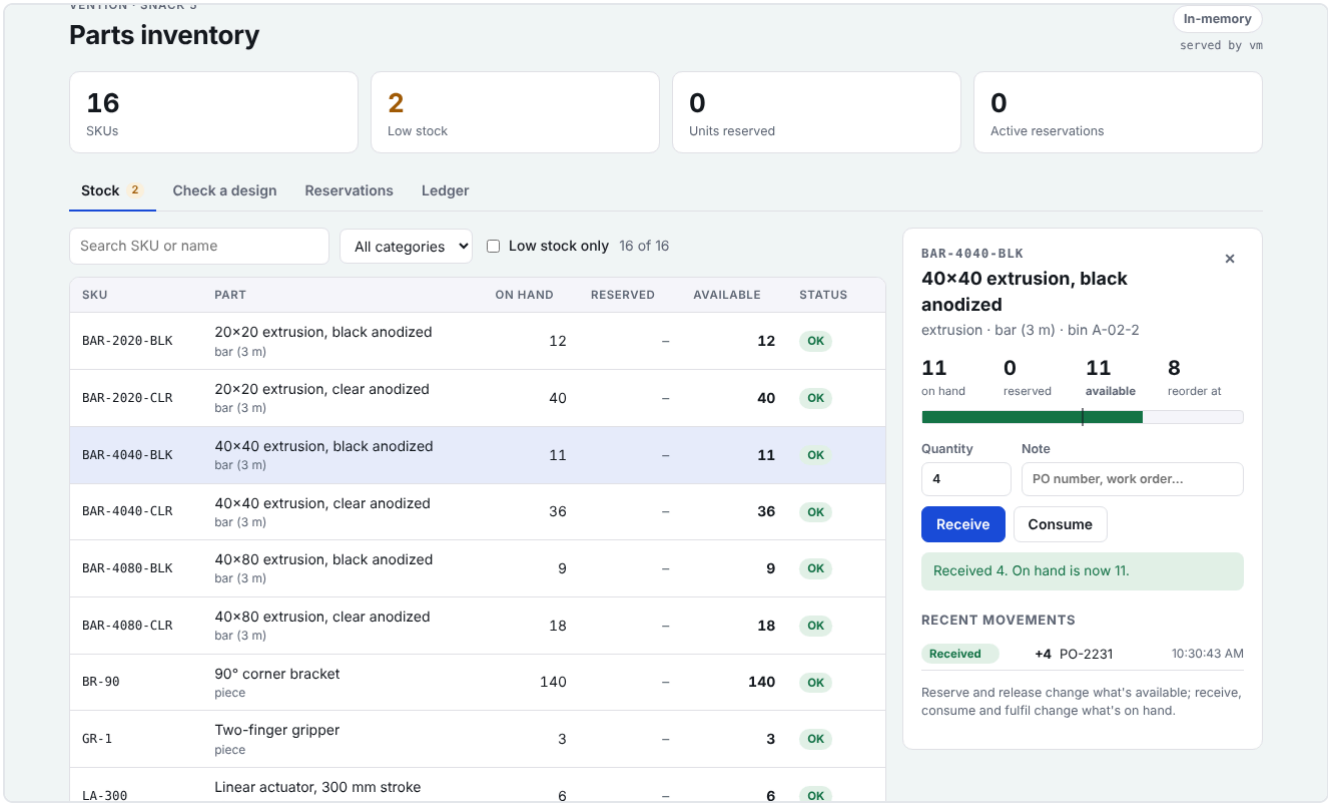
VENTION · PORTFOLIO WALKTHROUGH

Snack 5, Line by Line

A parts inventory on PostgreSQL or MongoDB: data models, transactions and locking, atomic updates, contract tests, React, Docker and Kubernetes, every file explained.



5 · Parts Inventory

Projects in `Portfolio/vention/` · Code quoted directly from the repository, with file and line numbers

Contents

How to read this book 3

Chapter 5 · Parts Inventory 4

5.1 The shape of the system 5

5.2 Relational and document data models 5

5.3 Transactions, isolation and race conditions 6

5.4 Atomicity in MongoDB 7

5.5 Repositories and contract tests 8

5.6 Cut lists and bin packing 9

5.7 Choosing the database at deploy time 10

5.8 Setting up the database 10

5.9 Two apps, one database 13

Line by line 14

The data model and the one interface 14

Errors and shared helpers 19

From a machine design to stock 21

The reference implementation 28

The PostgreSQL schema 32

The PostgreSQL repository 38

The MongoDB repository 44

Choosing a database, and the service layer 52

The HTTP API 54

Contract tests: one suite, every database 61

Adding a part, end to end 66

Shapes for the robot arm's scene 74

The React page 83

Containers: Compose profiles and Kubernetes overlays 89

Conveyor belts in the database 95

Splitters: sorting rules as data, and editable parts 101

Interview questions 115

Exercises 118

Appendix A · Running the projects 119

Appendix B · Glossary 120

BEFORE YOU START

How to read this book

This walkthrough covers Snack 5 on its own. It's also Chapter 5 of the combined book. It has five parts: Each chapter covers one snack and has the same five parts:

1. **Concepts.** The ideas the code relies on, with diagrams, before any code: transforms and glTF, inverse kinematics and React's rendering model, physics engines, characterization testing.
2. **Line by line.** Every source file that matters, quoted from the repository in order, with each chunk explained. The code is sliced from the real files, so the line numbers match what you'll see in your editor.
3. **Going deeper.** Boxes inside the line-by-line parts that step back to what a production system, a full engine or a large team would do differently.
4. **Interview questions.** Questions this project invites, with model answers grounded in the code.
5. **Exercises.** Extensions to try, each with a hint.

Two reading paths work well. To prepare for a conversation about the projects, read each chapter's concepts and questions. To understand or extend the code, read the line-by-line parts with the project open next to the book.

The project

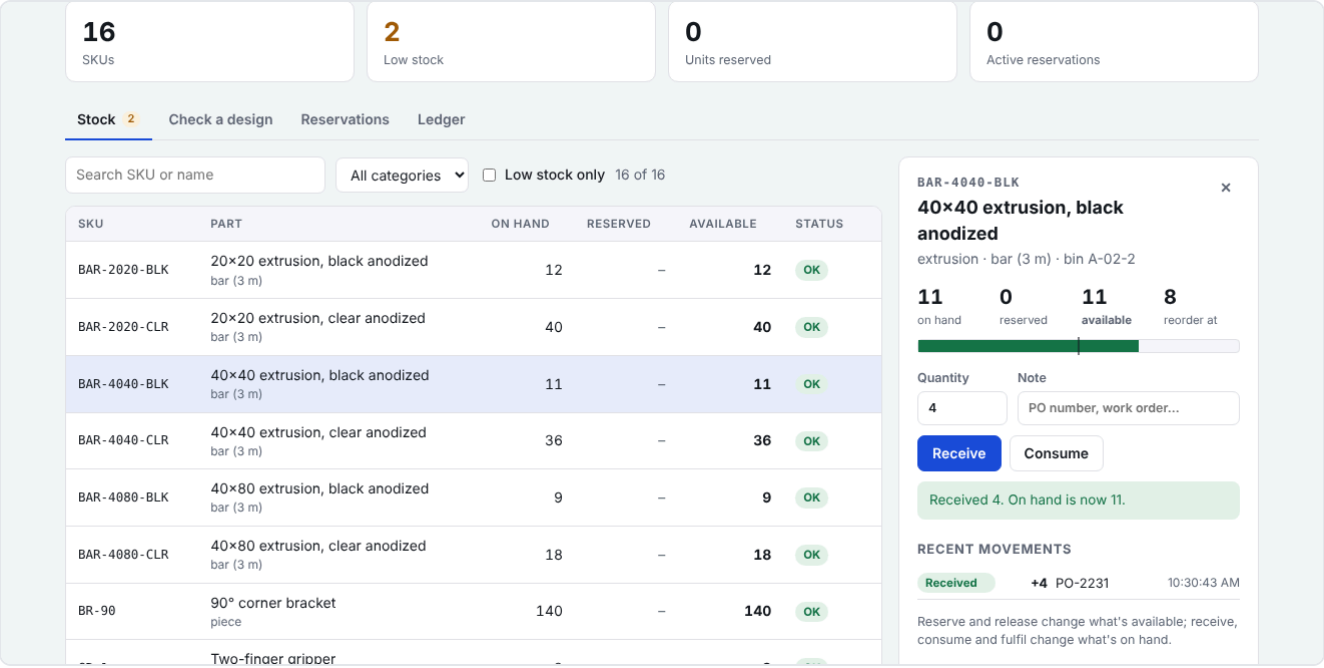
	Project	Stack	What it shows
5	**Parts Inventory**	Node.js · React · PostgreSQL or MongoDB · Docker · Kubernetes	A stock system for the parts the other snacks use: check a machine design against live stock and reserve it, all or nothing, on PostgreSQL or MongoDB behind one interface.

CHAPTER 5 · `vention/snack5-inventory`

Parts Inventory

A stock system for the parts the other snacks use: check a machine design against live stock and reserve it, all or nothing, on PostgreSQL or MongoDB behind one interface.

Node.js · React · PostgreSQL or MongoDB · Docker · Kubernetes



The stock page with a 40×40 black bar open: four bars just received, logged in its movement history.

The one idea to take away

The database is the only place two API replicas share, so it's where “is there enough?” is answered:

`design` → `plan(): stock lines` → `repo.reserve()` → `locks / conditional updates` → `reservation or shortages`

One interface, three implementations, one contract suite that keeps them identical:

`InventoryRepo` → `PostgresRepo · MongoRepo · MemoryRepo` → `contractTests()` → `same 19 tests on each`

CONCEPTS · 5.1

The shape of the system

Snack 5 is a classic three-tier web app: a static front end, a stateless API, and a database. The interesting part is the seam between the API and the database: one TypeScript interface, `InventoryRepo`, with three implementations.

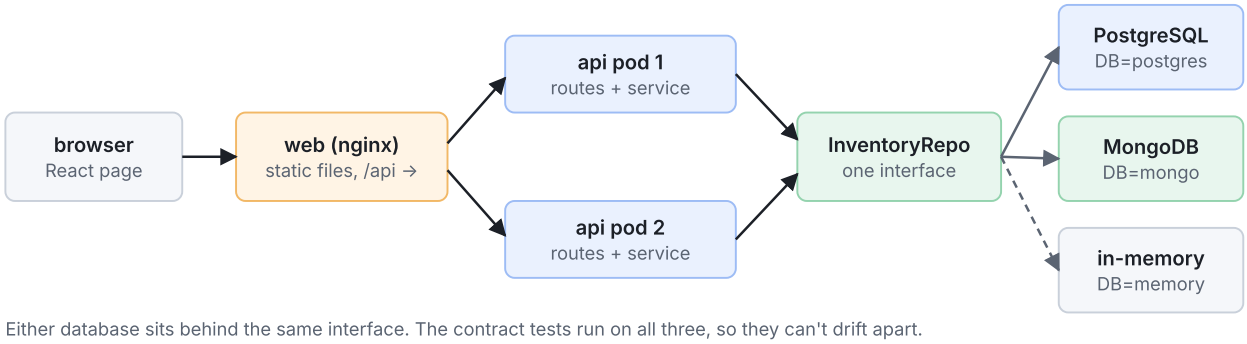


Figure 5.1. Snack 5's architecture. Two API replicas share one database; one environment variable chooses which kind.

Two API replicas run at once, in Kubernetes and potentially in Compose. That's the point: the replicas share nothing but the database, so **the database is the only place where "is there enough stock?" can be answered reliably**. Any check done in API memory would be wrong as soon as the other replica changed something.

CONCEPTS · 5.2

Relational and document data models

A **relational** database stores data in tables of rows with a fixed set of columns, and relates tables with keys. A **document** database stores self-contained JSON-like documents, which can nest arrays and objects.

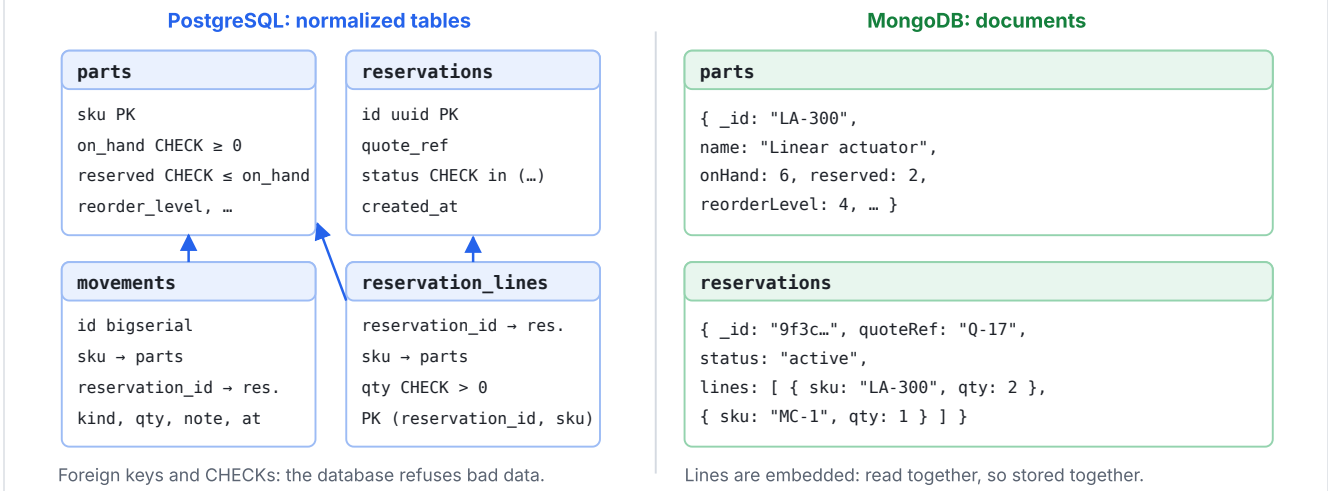


Figure 5.2. The same data in both models. Postgres splits it into related tables; MongoDB nests what's read together.

- **Normalization** (Postgres side): each fact is stored once. A reservation line is a row that points at a part and a reservation. Joins put them back together, and foreign keys guarantee every pointer is valid.
- **Embedding** (MongoDB side): data that's always read together is stored together. A reservation and its lines are one document, read and written in one operation, with no join.
- **Schema**: Postgres enforces column types and constraints on every write. MongoDB accepts any shape unless you add a JSON Schema validator; here the TypeScript types and the repository are what keep documents consistent.
- **Rule of thumb**: model relationally when data is shared and queried many ways (parts appear in many reservations and reports); embed when a piece of data belongs to one parent and is only read through it (a reservation's lines).

CONCEPTS · 5.3

Transactions, isolation and race conditions

A **transaction** groups statements so they all happen or none do (atomicity), and so concurrent transactions don't see each other's half-finished work (isolation). Together with consistency and durability that's **ACID**.



Figure 5.3. The race the database has to prevent. Without a lock, both requests act on a value that's already stale.

The left side is a **lost update**, also called check-then-act: both requests read "1 left", both decide to reserve, both write. Each request was correct on its own; together they promised a part that doesn't exist. Three tools prevent it:

- **Row locks.** `SELECT … FOR UPDATE` locks the rows until the transaction ends; a second transaction wanting them waits. Used in `inv_reserve`, where several rows must be checked together.
- **Conditional updates.** Put the rule in the update itself: `UPDATE … WHERE on_hand - reserved >= 2`. Under READ COMMITTED, Postgres re-checks that condition on the latest row version if it had to wait, so the update simply does nothing if the stock is gone. Used for single-part changes, on both databases.
- **Constraints.** `CHECK (reserved <= on_hand)` makes the database refuse any statement that would break the rule. It doesn't help you produce a nice error, but it catches every bug the other two miss.

Locks bring a new risk, **deadlock**: transaction 1 locks part A and wants B while transaction 2 holds B and wants A. Postgres detects this and kills one of them. The standard prevention is a **global lock order**: everyone locks in SKU order, so a cycle can't form. The SQL test fired 30 reservations at once, half listing the parts in reverse, with no deadlocks.

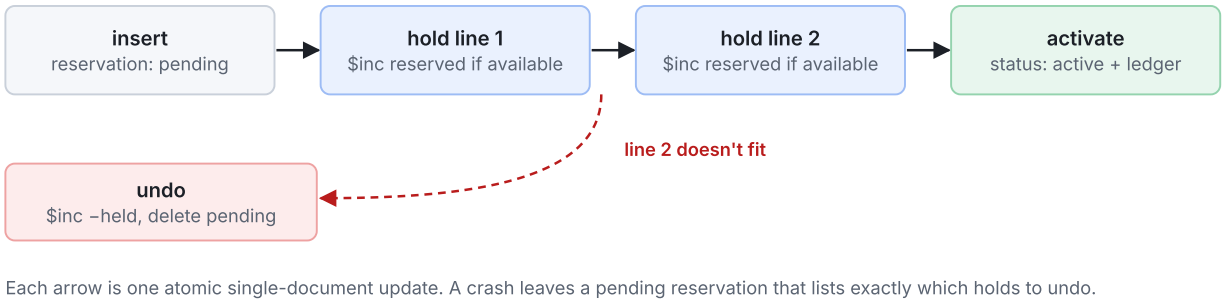
> Postgres's stricter isolation levels (REPEATABLE READ, SERIALIZABLE) can also prevent these anomalies, by aborting one of the conflicting transactions with a serialization error that the application must retry. Explicit locks avoid the retry loop, at the cost of making requests wait.

CONCEPTS · 5.4

Atomicity in MongoDB

MongoDB guarantees that a write to **one document** is atomic, including all the fields it changes and the filter it matched. Multi-document transactions exist too, but only on a replica set or sharded cluster, not a single standalone server.

So the MongoDB repository leans on single-document atomicity. Each part is one document, and each rule goes in the filter: `updateOne({ _id: sku, $expr: { $gte: [onHand − reserved, qty] } }, { $inc: { reserved: qty } })`. Either the document matched and was changed, or nothing happened. A reservation spans several documents, so it becomes a **saga**:



Each arrow is one atomic single-document update. A crash leaves a pending reservation that lists exactly which holds to undo.

Figure 5.4. The MongoDB reservation saga. Without a multi-document transaction, all-or-nothing is built from atomic steps plus compensation.

A saga gives atomicity eventually rather than instantly: between steps, other requests can see a partial hold. The invariant that matters (never promise more than is on the shelf) holds at every moment; the cost is that a racing request might be refused while a hold that's about to be undone is in place.

CONCEPTS · 5.5

Repositories and contract tests

The **repository pattern** hides storage behind an interface shaped like the domain (`reserve`, `consume`), not like the database (`query`, `find`). Routes, the service and the UI never learn which database is in use.

An interface alone promises method names, not behaviour. A **contract test suite** is what makes the promise real: one set of tests, written against the interface, run against every implementation. If MongoDB returned shortages in a different shape, or forgot to write the ledger on release, its run would fail while Postgres's passed.

Keeping an in-memory implementation is worth its 140 lines: it defines the intended behaviour with no concurrency or I/O in the way, makes the API tests fast and runnable anywhere, and lets the whole app run with no database for a demo.

CONCEPTS · 5.6

Cut lists and bin packing

How many 3 m bars does a cut list need? That's the **bin-packing problem**: pack items of given sizes into the fewest bins of fixed capacity. Finding the true minimum is NP-hard, so practical tools use heuristics.

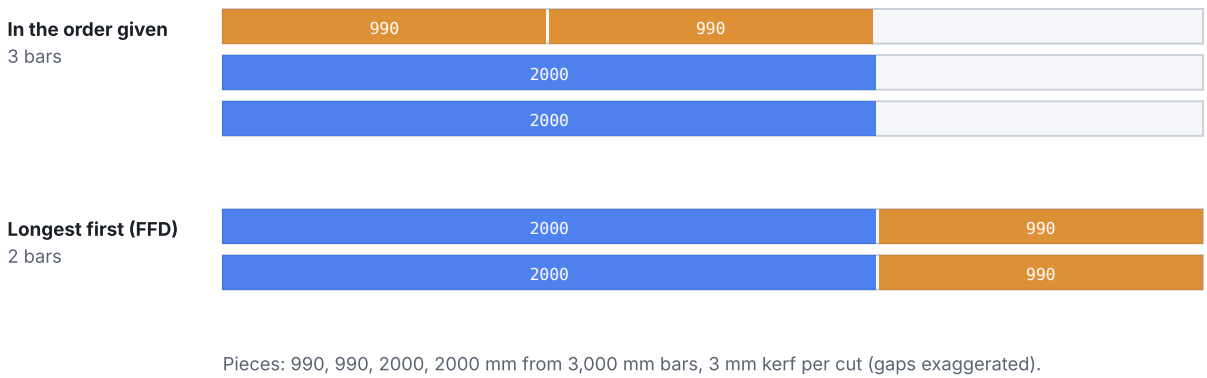


Figure 5.5. First-fit decreasing on a cut list. Placing big pieces first leaves gaps the small ones fill.

First-fit decreasing (FFD) sorts pieces longest first and puts each in the first bar with room. It's simple, fast (n log n for the sort plus a scan), and provably never needs more than about 11/9 of the optimal number of bars (plus a small constant). The kerf is what makes this real rather than textbook: every cut loses 3 mm, so two 1,500 mm pieces need two bars.

Industrial cutting software also handles multiple stock lengths, offcuts kept for later, and grouping cuts to reduce saw setups. Those are optimisation problems in their own right (the “cutting stock problem”), usually solved with linear programming.

CONCEPTS · 5.7

Choosing the database at deploy time

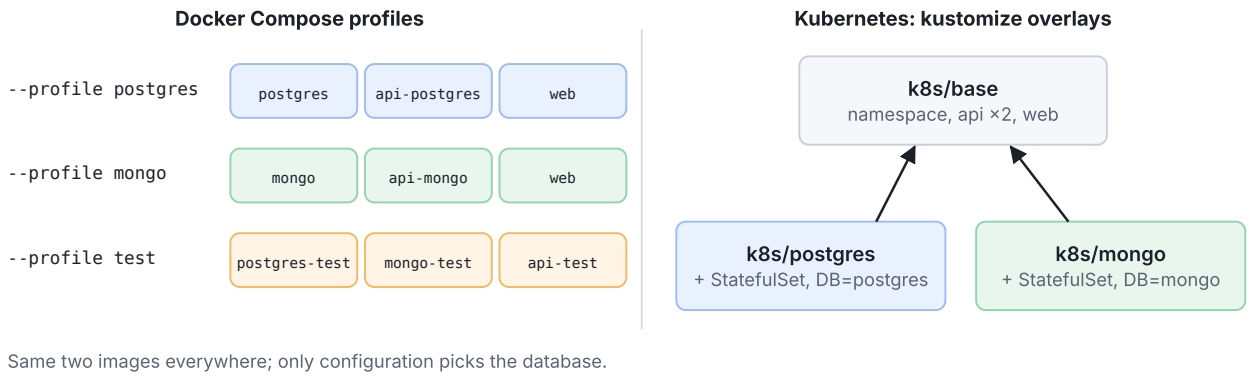


Figure 5.6. How the database is chosen at deploy time, in Compose and in Kubernetes.

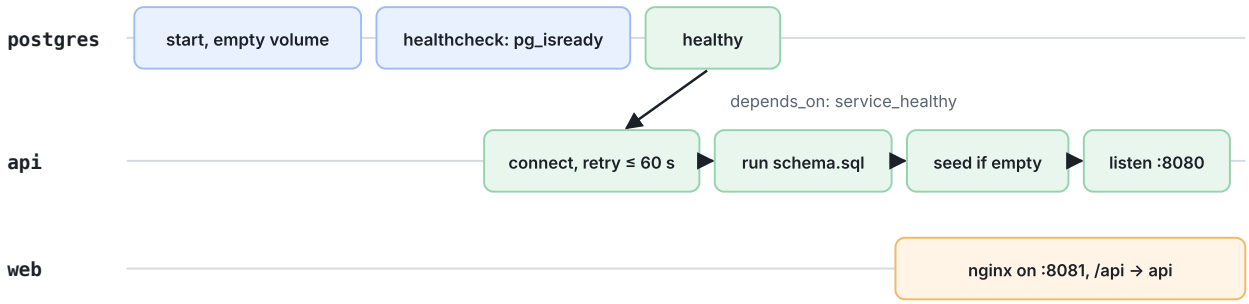
Configuration comes from the environment, the **twelve-factor** rule: the same image runs in Compose, Kubernetes and tests, and only environment variables differ. Compose selects services with **profiles**; Kubernetes selects with **kustomize overlays** that add a database StatefulSet and generate the API's settings on top of a shared base.

Databases run as **StatefulSets**, not Deployments: each pod gets a stable name (`postgres-0`) and its own persistent volume that follows it across restarts. The API's readiness probe checks the database, so Kubernetes only routes traffic to pods that can actually serve it.

CONCEPTS · 5.8

Setting up the database

There's no separate install or migration step. Docker Compose starts the database, and the API sets it up the first time it connects. The whole sequence, on an empty machine:



Every step can run again safely: tables are CREATE ... IF NOT EXISTS, and seeding only fills an empty database.

Figure 5.7. What happens on the first docker compose --profile postgres up. MongoDB follows the same sequence with mongosh as its healthcheck.

1. **Compose starts PostgreSQL** with a named volume, `pg-data`, so the data survives restarts and rebuilds. The official image creates the `inventory` user and database from its environment variables on first boot.
2. **Compose waits for the healthcheck.** `pg_isready` runs every 5 s. The API container only starts once it passes (`depends_on: condition: service_healthy`).
3. **The API connects**, retrying every 2 s for up to a minute. In Kubernetes there's no `depends_on`, so this retry loop, plus the startup probe that allows 150 s, is what makes ordering not matter.
4. **The API runs `schema.sql`**: four tables, their constraints and indexes, and the two functions. Every statement is `CREATE … IF NOT EXISTS` or `CREATE OR REPLACE`, so it's safe on every start, and two API replicas starting together don't conflict.
5. **The API seeds the stock**, only if `parts` is empty: 16 parts in one transaction. Restarting never duplicates stock, and your changes survive.
6. **The API starts listening**, the readiness probe passes, and nginx starts receiving answers on port 8081.

I ran exactly this schema and seed against a real PostgreSQL 16 and then used it the way the app does. This is the session, trimmed, with the user and database names shown as they are under Docker (`inventory`):

psql: the database after first start

```
inventory=> \dt
 Schema |       Name        | Type  | Owner
--------+-------------------+-------+-----------
 public | movements         | table | inventory
 public | parts             | table | inventory
 public | reservation_lines | table | inventory
 public | reservations      | table | inventory

inventory=> SELECT sku, on_hand, reserved, on_hand - reserved AS available, reorder_level
            FROM parts WHERE on_hand - reserved <= reorder_level;
     sku      | on_hand | reserved | available | reorder_level
--------------+---------+----------+-----------+---------------
 BAR-4040-BLK |       7 |        0 |         7 |             8
 M5-BOLT      |      22 |        0 |        22 |            25
 PL-300       |       8 |        0 |         8 |            10
```

The three low-stock parts are deliberate, so the low-stock filter has something to show on day one. Then a reservation that fits, one that doesn't, and an attempt to break the rules directly:

psql: reserving, and the safety net

```
inventory=> SELECT inv_reserve('Q-1001',
              '[{"sku":"LA-300","qty":2},{"sku":"MC-1","qty":1},{"sku":"PL-200","qty":4}]');
 {"id": "a8ca2dfa-b176-403f-9d62-71a9b00ed112", "ok": true}

inventory=> SELECT inv_reserve('Q-1002', '[{"sku":"LA-300","qty":8}]');
 {"ok": false, "shortages": [{"sku": "LA-300", "need": 8, "known": true, "available": 4}]}

inventory=> SELECT id, sku, kind, qty, note FROM movements;
 id |  sku   |  kind   | qty |        note
----+--------+---------+-----+---------------------
  1 | LA-300 | reserve |   2 | Reserved for Q-1001
  2 | MC-1   | reserve |   1 | Reserved for Q-1001
  3 | PL-200 | reserve |   4 | Reserved for Q-1001

inventory=> UPDATE parts SET reserved = 99 WHERE sku = 'MC-1';
ERROR:  new row for relation "parts" violates check constraint "reserved_within_stock"
```

The refused reservation changed nothing and wrote no ledger lines. The last statement bypasses the application entirely, and the database still refuses: that's what “rules in the database” means.

To look inside your own database while the app runs:

PowerShell, from vention/snack5-inventory

```
# PostgreSQL
docker compose --profile postgres exec postgres psql -U inventory -d inventory
#   then e.g.  SELECT * FROM movements ORDER BY id DESC LIMIT 10;

# MongoDB
docker compose --profile mongo exec mongo mongosh -u inventory -p inventory-dev-password --
authenticationDatabase admin inventory
#   then e.g.  db.parts.find({ $expr: { $lte: [{ $subtract: ["$onHand", "$reserved"] },
"$reorderLevel"] } })

# Start again from the seed stock (deletes that database's volume)
docker compose --profile postgres down -v
```

CONCEPTS · 5.9

Two apps, one database

The robot arm app (Chapter 1) now stores its scene in this database: shapes on the floor, some of them standing for inventory parts. Two front ends sharing one API is common, and it raises two questions this chapter's design already answers.

- **Where do rules live?** In the API and the database, not in either front end. The arm app checks sizes and SKUs as you type for convenience; the API checks again, and Postgres's constraints and foreign key are the final word.
- **How do browsers on other origins call it?** With CORS headers from the API, allowed for pages on localhost by default.
- **What links the data?** The SKU. A shape for a part carries it, so the arm app can show “2 in scene, 1 on hand”. Placing a part in the scene doesn't change stock: a scene is a layout, and holding stock is a reservation's job.

CHAPTER 5

Line by line

16 parts, in the order data flows through the code.

PART 5.1 · `api/src/domain/types.ts`

The data model and the one interface

Everything starts with the types: what a part, a movement and a reservation are, and the single interface every database has to implement.

Parts

api/src/domain/types.ts · lines 6–30

```
/**
 * The inventory's data model and the one interface every database implements.
 *
 * Quantities are whole stock units: bars, plates, packs or pieces, as given by `unit`.
 * `available` is what can still be promised: on hand minus what's reserved for quotes.
 */

export type Category = "extrusion" | "plate" | "fastener" | "motor" | "actuator" | "controller"
| "gripper";

export interface Part {
  sku: string;
  name: string;
  category: Category;
  /** What one unit of stock is, e.g. "bar (3 m)" or "pack of 50". */
  unit: string;
  onHand: number;
  reserved: number;
  available: number;
  /** Reorder when available falls to this level or below. */
  reorderLevel: number;
  low: boolean;
  /** Bin location in the warehouse, e.g. "A-02-1". */
  location: string;
```

```
  updatedAt: string;
}
```

A part's stock is three numbers: **on hand** (physically on the shelf), **reserved** (promised to quotes) and **available** = on hand − reserved. Available is what sales can still promise, so it's what every check uses. `Category` is a string literal union, like Snack 1's `Axis`: a typo is a compile error.

`available` and `low` are **derived**: no database stores them. They're computed in one place (`withDerived`, below), so the three implementations can't disagree about what “low stock” means.

Seed parts and movements

`api/src/domain/types.ts · lines 32–54`

```
/** A part as it's first stocked, before anything is reserved. */
export interface SeedPart {
  sku: string;
  name: string;
  category: Category;
  unit: string;
  onHand: number;
  reorderLevel: number;
  location: string;
}

export type MovementKind = "receive" | "consume" | "reserve" | "release" | "fulfil";

/** One line of the stock ledger. Every change to a part writes one. */
export interface Movement {
  id: string;
  sku: string;
  kind: MovementKind;
  qty: number;
  note: string;
  reservationId: string | null;
  at: string;
}
```

`SeedPart` is a part before anything has happened to it: no reservations, no timestamp. `Movement` is one line of the **stock ledger**. Every change writes one, and they're never edited, so the current numbers can always be explained by replaying the ledger. Accountants call this an append-only journal; event-sourced systems take the same idea further and store only the events.

Lines, reservations and shortages

api/src/domain/types.ts · lines 56–80

```
export interface Line {
  sku: string;
  qty: number;
}

export type ReservationStatus = "active" | "released" | "fulfilled";

export interface Reservation {
  id: string;
  quoteRef: string;
  status: ReservationStatus;
  lines: Line[];
  createdAt: string;
  closedAt: string | null;
}

/** Why a reservation couldn't be made: `available` is 0 and `known` false for SKUs not stocked
at all. */
export interface Shortage {
  sku: string;
  need: number;
  available: number;
  known: boolean;
}

export type ReserveResult = { ok: true; reservation: Reservation } | { ok: false; shortages:
Shortage[] };
```

A `Line` is a SKU and a quantity, the common currency of the whole system: a design becomes lines, a reservation holds lines, a shortage reports a line that didn't fit. `known: false` distinguishes “we stock it but ran out” from “we don't stock that SKU at all”, which a sales person handles very differently.

`ReserveResult` is a **discriminated union**: check `ok` and TypeScript narrows to either `reservation` or `shortages`. Running out of stock is an expected outcome, not an exception, so it's a return value.

The repository interface

api/src/domain/types.ts · lines 82–163

```
export interface PartFilter {
  /** Case-insensitive match on SKU or name. */
  search?: string;
  /** Only parts at or below their reorder level. */
  lowOnly?: boolean;
}

export type DbKind = "postgres" | "mongo" | "memory";

/**
 * Everything the API needs from a database. Postgres, MongoDB and the in-memory
 * reference implementation all pass the same contract tests (tests/contract.ts).
 *
 * Rules every implementation enforces:
 * - stock never goes negative, and reserved never exceeds on hand, even under concurrent
requests;
 * - consuming can only use available stock, never stock reserved for someone else;
 * - a reservation is all or nothing: if any line is short, nothing is reserved;
 * - every change writes a movement.
 */
export interface InventoryRepo {
  readonly kind: DbKind;
  /** Creates tables/indexes if missing. Safe to call on every start. */
  init(): Promise<void>;
  /** Inserts the seed parts if there are none yet. Returns whether it did. */
  seedIfEmpty(parts: readonly SeedPart[]): Promise<boolean>;
  /** Throws if the database can't be reached. */
  ping(): Promise<void>;
  close(): Promise<void>;

  listParts(filter?: PartFilter): Promise<Part[]>;
  getParts(skus: readonly string[]): Promise<Part[]>;
  /**
   * Adds a new part. Initial stock is written to the ledger as a receipt, so the
   * ledger still explains every unit. Throws ConflictError if the SKU already exists.
   */
  createPart(part: SeedPart): Promise<Part>;
  /** Changes name, category, location or reorder level; never stock. Throws NotFoundError for
```

```
an unknown SKU. */
  updatePart(sku: string, patch: PartPatch): Promise<Part>;
  /** Adds stock. Throws NotFoundError for an unknown SKU. */
  receive(sku: string, qty: number, note: string): Promise<Part>;
  /** Removes available stock. Throws NotFoundError, or ConflictError if not enough is
available. */
  consume(sku: string, qty: number, note: string): Promise<Part>;
  /** Newest first. */
  movements(filter: { sku?: string; limit: number }): Promise<Movement[]>;

  /** All or nothing. Lines with the same SKU are added together. */
  reserve(quoteRef: string, lines: readonly Line[]): Promise<ReserveResult>;
  /** Newest first. */
  reservations(limit: number): Promise<Reservation[]>;
  /** Gives reserved stock back. Throws NotFoundError, or ConflictError unless active. */
  release(id: string): Promise<Reservation>;
  /** Ships reserved stock: on hand and reserved both go down. Same errors as release. */
  fulfil(id: string): Promise<Reservation>;

  /** Shapes in the robot arm's scene, oldest first. */
  listShapes(): Promise<Shape[]>;
  createShape(shape: NewShape): Promise<Shape>;
  /** Throws NotFoundError for an unknown id. */
  updateShape(id: string, patch: ShapePatch): Promise<Shape>;
  /** Throws NotFoundError for an unknown id. */
  deleteShape(id: string): Promise<void>;

  /** Conveyor belts in the robot arm's scene, oldest first. */
  listConveyors(): Promise<Conveyor[]>;
  createConveyor(conveyor: NewConveyor): Promise<Conveyor>;
  /** Throws NotFoundError for an unknown id. */
  updateConveyor(id: string, patch: ConveyorPatch): Promise<Conveyor>;
  /** Throws NotFoundError for an unknown id. */
  deleteConveyor(id: string): Promise<void>;

  /** Splitters (diverter gates on belts), oldest first. Deleting a belt deletes its splitters.
*/
  listSplitters(): Promise<Splitter[]>;
  /** The caller checks the belt exists and `along` fits on it. */
  createSplitter(splitter: NewSplitter): Promise<Splitter>;
  /** Throws NotFoundError for an unknown id. */
```

```
  updateSplitter(id: string, patch: SplitterPatch): Promise<Splitter>;
  /** Throws NotFoundError for an unknown id. */
  deleteSplitter(id: string): Promise<void>;

  /** Deletes everything. Tests only. */
  reset(): Promise<void>;
}
```

This is the **repository pattern**: the rest of the code talks to `InventoryRepo`, never to a database driver. The doc comment is the real specification. Four rules hold for every implementation, and the contract tests check each of them on each database.

Everything returns a `Promise`, even in the in-memory version, which could answer synchronously. An interface that's async everywhere lets any implementation do I/O without changing a single caller.

Errors are part of the contract too: “unknown SKU” is a `NotFoundError` and “not enough” is a `ConflictError` in every implementation, so the HTTP layer maps them to 404 and 409 without knowing where they came from.

GOING DEEPER

Ledgers, event sourcing and audit

The movements table is an append-only ledger beside the current counts. **Event sourcing** goes one step further: store only the events, and compute current stock by replaying them (with snapshots for speed). It gives a perfect audit trail and time travel (“what did we have on March 3?”), at the cost of more complex reads. A ledger next to counters, as here, gets most of the audit benefit with simple queries.

PART 5.2 · `api/src/domain/errors.ts`

Errors and shared helpers

Three error classes carry their HTTP status, and two small helpers keep the implementations consistent.

Errors that know their status

api/src/domain/errors.ts · lines 1–19

```
/** Errors the HTTP layer turns into status codes. */

export class NotFoundError extends Error {
  readonly status = 404;
}

export class ConflictError extends Error {
  readonly status = 409;
}

export class ValidationError extends Error {
  readonly status = 400;
  readonly problems: string[];

  constructor(problems: string[]) {
    super(problems.join("; "));
    this.problems = problems;
  }
}
```

`readonly status = 404` is a class field, so every `NotFoundError` has it. The HTTP handler reads `err.status` and doesn't need a table of error types. `ValidationError` collects every problem with a request, so a user fixing a design sees all its mistakes at once instead of one per attempt.

Merging lines, and derived fields

api/src/domain/lines.ts · lines 1–14

```
import type { Line, Part } from "./types.ts";

/** Adds up lines with the same SKU and sorts by SKU (the order rows are locked in, to avoid
deadlocks). */
export function mergeLines(lines: readonly Line[]): Line[] {
  const total = new Map<string, number>();
  for (const { sku, qty } of lines) total.set(sku, (total.get(sku) ?? 0) + qty);
  return [...total].map(([sku, qty]) => ({ sku, qty })).sort((a, b) => (a.sku < b.sku ? -1 :
a.sku > b.sku ? 1 : 0));
}

/** Derived fields, computed the same way for every database. */
export function withDerived(p: Omit<Part, "available" | "low">): Part {
  const available = p.onHand - p.reserved;
  return { ...p, available, low: available <= p.reorderLevel };
}
```

`mergeLines` adds up repeated SKUs and **sorts by SKU**. The sort isn't cosmetic: it's the order every database locks rows in, which is what prevents deadlocks (chapter concept 5.2). Merging first also means a design listing PL-200 twice is checked against its total, not each half separately.

Postgres sorts SKUs with its own collation, which can order punctuation differently from JavaScript's `<`. That's harmless: in Postgres, every lock is taken by Postgres's own `ORDER BY sku`, so all transactions agree; in MongoDB, this JavaScript order is the only one used. What matters is one consistent order per database, not the same order everywhere.

PART 5.3 · `api/src/domain/design.ts`

From a machine design to stock

Snack 4 prices a design; this file works out what stock it consumes. Most of it is bookkeeping, but one function is a classic algorithm: packing cut pieces into stock bars.

The input format

api/src/domain/design.ts · lines 1–28

```
/**
 * Turns a machine design, in the same shape Snack 4's quote calculator takes, into the
```

```
 * stock it needs.
 *
 * Most items map one to one (4 × PL-200 plates need 4 plates). Two don't:
 * - fasteners are stocked in packs of 50, so 120 bolts need 3 packs;
 * - extrusions are cut from 3 m stock bars, so the question is how many bars the
 *   pieces fit into. That's the bin-packing problem; first-fit decreasing never uses more
 *   than about 11/9 of the optimal number of bars, and on short cut lists is usually optimal.
 */
import type { Line } from "./types.ts";
import { mergeLines } from "./lines.ts";
import { ValidationError } from "./errors.ts";

/** Same item shapes as snack4-refactor-kata/src/types.ts. */
export interface DesignItem {
  type: string;
  sku: string;
  qty: number;
  profile?: string;
  /** mm, extrusions only */
  length?: number;
  finish?: "clear" | "black";
}

export interface Design {
  items: DesignItem[];
}
```

The item shape is copied from Snack 4's `types.ts`, deliberately. The two projects talk about the same machine, so a quote and a stock check use the same vocabulary. A shared npm package would be the next step if they lived in one codebase.

Constants and results

api/src/domain/design.ts · lines 30–50

```
export const BAR_LENGTH_MM = 3000;
/** Material lost to the saw blade per cut. */
export const KERF_MM = 3;
export const MIN_PIECE_MM = 50;
export const FASTENER_PACK = 50;
export const PROFILES = ["20x20", "40x40", "40x80"];

export interface Requirement extends Line {
  /** Where the number came from, for people: "120 bolts in packs of 50". */
  detail: string;
}

export interface Plan {
  requirements: Requirement[];
  /** Items that can't be turned into stock (unknown profile, piece too short). */
  warnings: string[];
}

export function barSku(profile: string, finish: string | undefined): string {
  return `BAR-${profile.replace("x", "")}-${finish === "black" ? "BLK" : "CLR"}`;
}
```

Stock bars are 3 m. The **kerf** is the width of material the saw blade turns into chips: 3 mm per cut. Ignore it and two 1,500 mm pieces seem to fit in one bar, but the cut between them needs 3 mm the bar doesn't have.

`barSku` derives the stock SKU from the design item: a black 40×80 piece is cut from `BAR-4080-BLK`. Each `Requirement` keeps a `detail` string, “4 × 1200 mm cut from 2 × 3 m bars”, because a number nobody can explain is a number nobody trusts.

Splicing long pieces

api/src/domain/design.ts · lines 52–68

```
/**
 * First-fit decreasing: sort pieces longest first, put each into the first bar with room.
 * Each cut also loses one saw kerf. Pieces longer than a bar are spliced: whole bars
 * plus a remainder piece that's packed like any other.
 */
export function barsNeeded(piecesMm: readonly number[], barMm = BAR_LENGTH_MM, kerfMm =
KERF_MM): number {
  let wholeBars = 0;
  const pieces: number[] = [];
  for (const len of piecesMm) {
    if (len > barMm) {
      wholeBars += Math.floor(len / barMm);
      const rest = len % barMm;
      if (rest > 0) pieces.push(rest);
    } else {
      pieces.push(len);
    }
  }
```

A piece longer than a bar is joined from several. Snack 4 adds a splice fee; here, a 4,000 mm piece needs one whole bar plus a 1,000 mm remainder, and the remainder goes into the packing like any other piece, so it can share a bar.

First-fit decreasing

api/src/domain/design.ts · lines 69–79

```
  pieces.sort((a, b) => b - a);
  const free: number[] = []; // remaining length in each opened bar
  for (const len of pieces) {
    // A piece fits if the bar has its length left. Cutting it off costs a kerf, unless
    // it takes the rest of the bar, so the leftover is never negative.
    const i = free.findIndex((room) => room >= len);
    if (i >= 0) free[i] = Math.max(0, free[i] - len - kerfMm);
    else free.push(Math.max(0, barMm - len - kerfMm));
  }
  return wholeBars + free.length;
}
```

`free` holds the length left in each bar opened so far. For each piece, longest first, `findIndex` finds the first bar with room; if none has room, a new bar is opened. The answer is how many bars were opened.

The kerf rule is in one expression: a piece fits if the bar has the piece's length left, and cutting it off costs the piece plus a kerf, floored at zero. So a piece that uses exactly the rest of the bar needs no cut, which is right: there's nothing left to cut it from.

Sorting longest first is what makes the heuristic good. Big pieces are placed while there's freedom, and small pieces fill the gaps they leave. Without the sort, two 990 mm pieces can take a bar together, leaving no bar to share with a 2,000 mm piece; one test pins exactly that case.

Turning items into requirements

api/src/domain/design.ts · lines 81–111

```
export function plan(design: Design): Plan {
  const warnings: string[] = [];
  const cutLists = new Map<string, number[]>(); // bar SKU -> piece lengths
  const cutText = new Map<string, string[]>();
  const direct: Requirement[] = [];

  for (const item of design.items) {
    if (item.type === "extrusion") {
      const profile = item.profile ?? "";
      const length = item.length ?? 0;
      if (!PROFILES.includes(profile)) {
        warnings.push(`${item.sku}: unknown profile ${profile || "(none)"}, not checked`);
        continue;
      }
      if (length < MIN_PIECE_MM) {
        warnings.push(`${item.sku}: ${length} mm is below the ${MIN_PIECE_MM} mm minimum, not
checked`);
        continue;
      }
      const sku = barSku(profile, item.finish);
      const list = cutLists.get(sku) ?? [];
      for (let i = 0; i < item.qty; i++) list.push(length);
      cutLists.set(sku, list);
      cutText.set(sku, [...(cutText.get(sku) ?? []), `${item.qty} × ${length} mm`]);
    } else if (item.type === "fastener") {
      const packs = Math.ceil(item.qty / FASTENER_PACK);
      direct.push({ sku: item.sku, qty: packs, detail: `${item.qty} pcs in packs of
```

```
${FASTENER_PACK}` });
    } else {
      // plates, motors, actuators, controllers, and anything else stocked by SKU
      direct.push({ sku: item.sku, qty: item.qty, detail: `${item.qty} × ${item.type}` });
    }
  }
```

Extrusions are collected into one **cut list per stock SKU**, across items: two different design items both cut from 40×40 clear bars share bars. Fasteners round up to packs. Everything else maps one to one by SKU, including item types Snack 4 doesn't price, like a gripper, so the inventory still checks them.

Items that can't be turned into stock (an unknown profile, a piece under 50 mm) become **warnings**, not errors: the rest of the design can still be checked, and the user sees what was skipped.

api/src/domain/design.ts · lines 113–123

```
  const bars: Requirement[] = [...cutLists].map(([sku, pieces]) => {
    const qty = barsNeeded(pieces);
    return { sku, qty, detail: `${cutText.get(sku)!.join(" + ")} cut from ${qty} × 3 m bar${qty
=== 1 ? "" : "s"}` };
  });

  // Merge requirements for the same SKU, keeping every detail.
  const details = new Map<string, string[]>();
  for (const r of [...bars, ...direct]) details.set(r.sku, [...(details.get(r.sku) ?? []),
r.detail]);
  const requirements = mergeLines([...bars, ...direct]).map((l) => ({ ...l, detail:
details.get(l.sku)!.join("; ") }));
  return { requirements, warnings };
}
```

Each cut list is packed, then all requirements are merged by SKU, joining their details. The result is sorted, deduplicated, and explains itself.

Validating untrusted input

api/src/domain/design.ts · lines 125–154

```
/** Validates untrusted JSON into a Design, collecting every problem rather than stopping at
the first. */
export function parseDesign(input: unknown): Design {
  const problems: string[] = [];
  const items = (input as { items?: unknown } | null)?.items;
```

```
  if (!Array.isArray(items) || items.length === 0) throw new ValidationError(["design.items
must be a non-empty array"]);
  if (items.length > 500) throw new ValidationError(["design.items has more than 500 items"]);

  const out: DesignItem[] = [];
  items.forEach((raw: unknown, i: number) => {
    const it = raw as Record<string, unknown> | null;
    const where = `items[${i}]`;
    if (!it || typeof it !== "object") return problems.push(`${where} must be an object`);
    const { type, sku, qty } = it;
    if (typeof type !== "string" || !type) problems.push(`${where}.type must be a string`);
    if (typeof sku !== "string" || !isSku(sku)) problems.push(`${where}.sku must be a SKU like
"PL-200"`);
    if (!isQty(qty)) problems.push(`${where}.qty must be a whole number from 1 to 100000`);
    if (type === "extrusion") {
      if (typeof it.profile !== "string") problems.push(`${where}.profile is required for
extrusions`);
      if (typeof it.length !== "number" || !Number.isFinite(it.length) || it.length <= 0 ||
it.length > 20000)
        problems.push(`${where}.length must be a length in mm up to 20000`);
      if (it.finish !== undefined && it.finish !== "clear" && it.finish !== "black")
problems.push(`${where}.finish must be "clear" or "black"`);
    }
    out.push({
      type: String(type), sku: String(sku), qty: Number(qty),
      ...(type === "extrusion" ? { profile: String(it.profile), length: Number(it.length),
finish: it.finish as DesignItem["finish"] } : {}),
    });
  });
  if (problems.length) throw new ValidationError(problems);
  return { items: out };
}
```

A design arrives as JSON from a browser, so nothing about it can be assumed. `parseDesign` checks each field and **collects** problems with their path (`items[3].qty`) instead of stopping at the first. Limits matter too: at most 500 items and 100,000 per line, so one request can't make the server pack a billion pieces.

The output is built fresh from checked fields rather than passing the input through, so unexpected extra properties never reach the database.

api/src/domain/design.ts · lines 156–162

```
export function isSku(s: unknown): s is string {
  return typeof s === "string" && /^[A-Z0-9][A-Z0-9-]{1,31}$/.test(s);
}

export function isQty(n: unknown): n is number {
  return typeof n === "number" && Number.isInteger(n) && n >= 1 && n <= 100_000;
}
```

Type predicates (`s is string`) let TypeScript narrow the type after the check, so callers don't need casts.

PART 5.4 · `api/src/db/memory.ts`

The reference implementation

The in-memory repository is about 140 lines of maps and arrays. It defines what “correct” means; the database versions have to behave exactly like it.

Why it's atomic for free

api/src/db/memory.ts · lines 1–32

```
/**
 * Reference implementation: plain maps in one process. It defines the behaviour the
 * contract tests expect, runs the API with no database (DB=memory), and is what the
 * API tests use. JavaScript runs one callback at a time and nothing here awaits midway,
 * so every method is atomic for free.
 */
import { randomUUID } from "node:crypto";
import { ConflictError, NotFoundError } from "../domain/errors.ts";
import { mergeLines, withDerived } from "../domain/lines.ts";
import type {
  InventoryRepo, Line, Movement, MovementKind, Part, PartFilter, Reservation, ReserveResult,
SeedPart, Shortage,
} from "../domain/types.ts";
import type { NewShape, Shape, ShapePatch } from "../domain/shapes.ts";
import type { Conveyor, ConveyorPatch, NewConveyor } from "../domain/conveyors.ts";
import type { NewSplitter, Splitter, SplitterPatch } from "../domain/splitters.ts";
import type { PartPatch } from "../domain/newPart.ts";
```

```
interface PartRow { sku: string; name: string; category: Part["category"]; unit: string;
onHand: number; reserved: number; reorderLevel: number; location: string; updatedAt: string }

export class MemoryRepo implements InventoryRepo {
  readonly kind = "memory";
  private parts = new Map<string, PartRow>();
  private moves: Movement[] = [];
  private res = new Map<string, Reservation>();
  private seq = 0;
  private shapes = new Map<string, Shape>();
  private conveyors = new Map<string, Conveyor>();
  private splitters = new Map<string, Splitter>();

  async init(): Promise<void> {}
  async ping(): Promise<void> {}
  async close(): Promise<void> {}
```

JavaScript runs one piece of code at a time on its event loop. A method that never `await`s in the middle can't be interleaved with another request, so every method here is atomic without any locking. That's precisely what a real database has to work to provide, and why this is a good reference: it shows the intended behaviour with no concurrency noise.

Reading

api/src/db/memory.ts · lines 34–52

```
async seedIfEmpty(seed: readonly SeedPart[]): Promise<boolean> {
  if (this.parts.size > 0) return false;
  const now = new Date().toISOString();
  for (const s of seed) this.parts.set(s.sku, { ...s, reserved: 0, updatedAt: now });
  return true;
}

async listParts(filter: PartFilter = {}): Promise<Part[]> {
  const q = filter.search?.trim().toLowerCase();
  return [...this.parts.values()]
    .map(out)
    .filter((p) => !q || p.sku.toLowerCase().includes(q) || p.name.toLowerCase().includes(q))
    .filter((p) => !filter.lowOnly || p.low)
    .sort((a, b) => (a.sku < b.sku ? -1 : 1));
}

async getParts(skus: readonly string[]): Promise<Part[]> {
  return skus.flatMap((s) => (this.parts.has(s) ? [out(this.parts.get(s)!)] : []));
}
```

Seeding only happens on an empty store, so restarting the API never duplicates stock. Listing applies the same filters as the SQL and Mongo queries, and every result goes through `out()`, which adds the derived fields.

Reserving, all or nothing

api/src/db/memory.ts · lines 92–113

```
async reserve(quoteRef: string, lines: readonly Line[]): Promise<ReserveResult> {
  const want = mergeLines(lines);
  const shortages: Shortage[] = [];
  for (const { sku, qty } of want) {
    const p = this.parts.get(sku);
    const available = p ? p.onHand - p.reserved : 0;
    if (available < qty) shortages.push({ sku, need: qty, available, known: p !== undefined });
  }
  if (shortages.length) return { ok: false, shortages };

  const reservation: Reservation = {
    id: randomUUID(), quoteRef, status: "active", lines: want, createdAt: new
Date().toISOString(), closedAt: null,
  };
  this.res.set(reservation.id, reservation);
  for (const { sku, qty } of want) {
    const p = this.parts.get(sku)!;
    p.reserved += qty;
    this.touch(p);
    this.log(sku, "reserve", qty, `Reserved for ${quoteRef}`, reservation.id);
  }
  return { ok: true, reservation: structuredClone(reservation) };
}
```

Two passes: first check every line, and only if all fit, apply them all. With no `await` between the passes, no other request can change stock in between. The returned reservation is a `structuredClone`, so a caller mutating it can't reach into the store, the same isolation a database gives by returning copies of rows.

Closing a reservation

api/src/db/memory.ts · lines 204–218

```
private close_(id: string, action: "release" | "fulfil"): Reservation {
  const r = this.res.get(id);
  if (!r) throw new NotFoundError(`No reservation ${id}`);
  if (r.status !== "active") throw new ConflictError(`Reservation is already ${r.status}`);
  for (const { sku, qty } of r.lines) {
    const p = this.parts.get(sku)!;
    p.reserved -= qty;
    if (action === "fulfil") p.onHand -= qty;
    this.touch(p);
    this.log(sku, action, qty, `${action === "fulfil" ? "Shipped" : "Released"} for
${r.quoteRef}`, id);
  }
  r.status = action === "fulfil" ? "fulfilled" : "released";
  r.closedAt = new Date().toISOString();
  return structuredClone(r);
}
```

Release and fulfil share one function: both remove the reservation's hold, and fulfil also takes the stock off the shelf. Only an active reservation can be closed, so a double-click can't release the same stock twice.

PART 5.5 · `api/src/db/schema.sql`

The PostgreSQL schema

Tables, constraints and two functions. The database enforces the stock rules itself, so they hold even if the application has a bug.

Parts, with the rules as constraints

api/src/db/schema.sql · lines 1–17

```
-- Inventory schema for PostgreSQL. Run on every API start: everything is idempotent.
--
-- The database itself enforces the stock rules, so no bug or race in the API can break them:
--   on_hand >= 0, reserved >= 0, reserved <= on_hand.

CREATE TABLE IF NOT EXISTS parts (
  sku            text PRIMARY KEY,
  name           text NOT NULL,
  category       text NOT NULL,
  unit           text NOT NULL,
  on_hand        integer NOT NULL DEFAULT 0 CHECK (on_hand >= 0),
  reserved       integer NOT NULL DEFAULT 0 CHECK (reserved >= 0),
  reorder_level  integer NOT NULL DEFAULT 0 CHECK (reorder_level >= 0),
  location       text NOT NULL DEFAULT '',
  updated_at     timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT reserved_within_stock CHECK (reserved <= on_hand)
);
```

`CREATE TABLE IF NOT EXISTS` makes the script safe to run on every start. The `CHECK` constraints are the last line of defence: whatever the code does, Postgres rejects any statement that would leave stock negative or reserve more than is on hand, and the whole transaction rolls back.

Reservations, their lines, and the ledger

api/src/db/schema.sql · lines 19–46

```
CREATE TABLE IF NOT EXISTS reservations (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  quote_ref   text NOT NULL,
  status      text NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'released',
'fulfilled')),
  created_at  timestamptz NOT NULL DEFAULT clock_timestamp(),
  closed_at   timestamptz
);

CREATE TABLE IF NOT EXISTS reservation_lines (
  reservation_id  uuid NOT NULL REFERENCES reservations (id) ON DELETE CASCADE,
  sku             text NOT NULL REFERENCES parts (sku),
  qty             integer NOT NULL CHECK (qty > 0),
```

```
  PRIMARY KEY (reservation_id, sku)
);

-- The stock ledger: one row per change, never updated.
CREATE TABLE IF NOT EXISTS movements (
  id              bigserial PRIMARY KEY,
  sku             text NOT NULL REFERENCES parts (sku),
  kind            text NOT NULL CHECK (kind IN ('receive', 'consume', 'reserve', 'release',
'fulfil')),
  qty             integer NOT NULL CHECK (qty > 0),
  note            text NOT NULL DEFAULT '',
  reservation_id  uuid REFERENCES reservations (id) ON DELETE CASCADE,
  at              timestamptz NOT NULL DEFAULT clock_timestamp()
);

CREATE INDEX IF NOT EXISTS movements_by_sku ON movements (sku, id DESC);
CREATE INDEX IF NOT EXISTS reservations_by_date ON reservations (created_at DESC);
```

A reservation's lines are a separate table, the **normalized** relational design: one row per SKU per reservation, with a composite primary key so a SKU can't appear twice. Foreign keys mean a line can't reference a part that doesn't exist, and `ON DELETE CASCADE` removes lines with their reservation.

`gen_random_uuid()` is built into Postgres 13+. `clock_timestamp()` rather than `now()` because `now()` is frozen at the start of a transaction: several reservations created in one transaction would get identical timestamps and an unstable order.

The indexes match the queries: “movements for this SKU, newest first” and “reservations, newest first”. `movements_by_sku` is on `(sku, id DESC)`, so Postgres can read the newest rows for a SKU straight off the index.

Reserving: merge, validate, lock

`api/src/db/schema.sql · lines 98–124`

```
-- Reserve every line of a quote, or nothing.
--   p_lines: [{"sku": "PL-200", "qty": 4}, ...]
--   returns {"ok": true, "id": "<uuid>"} or {"ok": false, "shortages": [...]}
--
-- The rows are locked (FOR UPDATE) in SKU order before anything is checked. Two quotes
-- reserving the same parts at the same moment therefore run one after the other, and
-- always lock in the same order, so they can't deadlock.
CREATE OR REPLACE FUNCTION inv_reserve(p_quote text, p_lines jsonb) RETURNS jsonb
LANGUAGE plpgsql AS $$
```

```
DECLARE
  v_id uuid;
  v_wanted jsonb;
  v_shortages jsonb;
BEGIN
  -- Add up lines with the same SKU: [{"sku", "qty"}], sorted by SKU.
  SELECT coalesce(jsonb_agg(jsonb_build_object('sku', sku, 'qty', qty) ORDER BY sku), '[]')
    INTO v_wanted
    FROM (SELECT l ->> 'sku' AS sku, sum((l ->> 'qty')::integer) AS qty
            FROM jsonb_array_elements(p_lines) AS l GROUP BY 1) AS t;

  IF EXISTS (SELECT 1 FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer) WHERE qty
<= 0) THEN
    RAISE EXCEPTION 'quantities must be positive' USING ERRCODE = '22023';
  END IF;

  PERFORM 1 FROM parts
    WHERE sku IN (SELECT w.sku FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer))
    ORDER BY sku FOR UPDATE;
```

The function receives the lines as JSONB and first merges them by SKU into `v_wanted`. `jsonb_to_recordset` turns that JSON back into rows whenever it's needed, which avoids a temporary table.

Then the key statement: `SELECT … ORDER BY sku FOR UPDATE` locks every part row the reservation touches, in SKU order. Until this transaction ends, any other transaction that wants to change those rows waits. And because everyone locks in the same order, two reservations can never each hold a lock the other needs: no deadlock.

Reserving: check, then write

api/src/db/schema.sql · lines 126–150

```
  SELECT jsonb_agg(jsonb_build_object(
           'sku', w.sku, 'need', w.qty,
           'available', coalesce(p.on_hand - p.reserved, 0),
           'known', p.sku IS NOT NULL) ORDER BY w.sku)
    INTO v_shortages
    FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer)
    LEFT JOIN parts p ON p.sku = w.sku
    WHERE p.sku IS NULL OR p.on_hand - p.reserved < w.qty;

  IF v_shortages IS NOT NULL THEN
```

```
    RETURN jsonb_build_object('ok', false, 'shortages', v_shortages);
  END IF;

  INSERT INTO reservations (quote_ref) VALUES (p_quote) RETURNING id INTO v_id;
  INSERT INTO reservation_lines (reservation_id, sku, qty)
    SELECT v_id, w.sku, w.qty FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer);
  UPDATE parts p SET reserved = p.reserved + w.qty, updated_at = now()
    FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer) WHERE p.sku = w.sku;
  INSERT INTO movements (sku, kind, qty, note, reservation_id)
    SELECT w.sku, 'reserve', w.qty, 'Reserved for ' || p_quote, v_id
      FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer) ORDER BY w.sku;

  RETURN jsonb_build_object('ok', true, 'id', v_id);
END;
$$;
```

With the rows locked, the check can't go stale. A `LEFT JOIN` finds both kinds of shortage in one query: SKUs not stocked at all (`p.sku IS NULL`) and SKUs with too little available. If there are any, the function returns them and writes nothing.

Otherwise four statements create the reservation, its lines, raise `reserved`, and write the ledger. They all run in the function's transaction, so another session sees either none of it or all of it.

Closing a reservation

api/src/db/schema.sql · lines 152–187

```
-- Release (give the stock back) or fulfil (ship it) an active reservation.
-- Returns 'ok', 'not_found', or the reservation's current status if it isn't active.
CREATE OR REPLACE FUNCTION inv_close_reservation(p_id uuid, p_action text) RETURNS text
LANGUAGE plpgsql AS $$
DECLARE
  v_status text;
  v_quote text;
BEGIN
  IF p_action NOT IN ('release', 'fulfil') THEN
    RAISE EXCEPTION 'unknown action %', p_action USING ERRCODE = '22023';
  END IF;

  SELECT status, quote_ref INTO v_status, v_quote FROM reservations WHERE id = p_id FOR UPDATE;
  IF NOT FOUND THEN RETURN 'not_found'; END IF;
  IF v_status <> 'active' THEN RETURN v_status; END IF;
```

```
  PERFORM 1 FROM parts WHERE sku IN (SELECT sku FROM reservation_lines WHERE reservation_id =
p_id) ORDER BY sku FOR UPDATE;

  UPDATE parts p
     SET reserved = p.reserved - l.qty,
         on_hand = CASE WHEN p_action = 'fulfil' THEN p.on_hand - l.qty ELSE p.on_hand END,
         updated_at = now()
    FROM reservation_lines l
   WHERE l.reservation_id = p_id AND p.sku = l.sku;

  INSERT INTO movements (sku, kind, qty, note, reservation_id)
    SELECT sku, p_action, qty,
           CASE WHEN p_action = 'fulfil' THEN 'Shipped for ' ELSE 'Released for ' END ||
v_quote, p_id
      FROM reservation_lines WHERE reservation_id = p_id ORDER BY sku;

  UPDATE reservations
     SET status = CASE WHEN p_action = 'fulfil' THEN 'fulfilled' ELSE 'released' END, closed_at
= now()
   WHERE id = p_id;
  RETURN 'ok';
END;
$$;
```

Lock the reservation row first. Two simultaneous “release” requests then queue: the first changes the status; the second, once it gets the lock, sees `released` and returns that instead of releasing again. The parts are locked in SKU order, as in `inv_reserve`, before both counters change in one `UPDATE … FROM` join.

Returning a status string instead of raising an error keeps the expected outcomes (`not_found`, already closed) out of Postgres's error machinery; TypeScript turns them into the right exception.

GOING DEEPER

Why functions in the database?

Putting `inv_reserve` in PL/pgSQL makes a reservation one round trip and one transaction, with the locking right next to the data. The cost is logic in two languages and a deploy step for the database. The alternative is the same steps from TypeScript: `BEGIN`, `SELECT … FOR UPDATE`, checks, writes, `COMMIT`, on one pooled connection. Both are common; what matters is that the lock and the check happen inside one transaction.

PART 5.6 · `api/src/db/postgres-sql.ts`

The PostgreSQL repository

Every SQL statement lives in one module, and the repository is a thin layer that runs them and maps rows to the domain types.

Reading, with parameters

api/src/db/postgres-sql.ts · lines 1–14

```
/**
 * Every SQL statement PostgresRepo runs, in one dependency-free module so they can also be
 * exercised directly with psql, without the Node driver.
 */
export const Q = {
  listParts: `
    SELECT sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at
      FROM parts
     WHERE ($1::text IS NULL OR sku ILIKE '%' || $1 || '%' OR name ILIKE '%' || $1 || '%')
       AND (NOT $2::boolean OR on_hand - reserved <= reorder_level)
     ORDER BY sku`,
  getParts: `
    SELECT sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at
      FROM parts WHERE sku = ANY($1::text[]) ORDER BY sku`,
```

Every value is a **parameter** (`$1`, `$2`), never pasted into the SQL text, which is what makes SQL injection impossible: the database receives the statement and the values separately.

`($1::text IS NULL OR …)` makes a filter optional in one statement. The search uses `ILIKE`, case-insensitive `LIKE`; the repository escapes `%` and `_` first, so a user searching for “200_mm” gets that text, not a wildcard.

Changing one part: rule and ledger in one statement

api/src/db/postgres-sql.ts · lines 31–47

```
receive: `
  WITH p AS (
    UPDATE parts SET on_hand = on_hand + $2, updated_at = now() WHERE sku = $1
    RETURNING sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at
  ), m AS (
    INSERT INTO movements (sku, kind, qty, note) SELECT sku, 'receive', $2, $3 FROM p
  )
  SELECT * FROM p`,
consume: `
  WITH p AS (
    UPDATE parts SET on_hand = on_hand - $2, updated_at = now()
     WHERE sku = $1 AND on_hand - reserved >= $2
    RETURNING sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at
  ), m AS (
    INSERT INTO movements (sku, kind, qty, note) SELECT sku, 'consume', $2, $3 FROM p
  )
  SELECT * FROM p`,
```

A **CTE** (`WITH … AS`) chains statements: the `UPDATE` returns the changed row, and the `INSERT` writes a ledger line for each row it returned, zero or one. Both are one statement, so they succeed or fail together.

For consume, the rule is in the `WHERE`: `on_hand - reserved >= $2`. Under Postgres's default READ COMMITTED isolation, if two consumes race for the same row, the second waits for the first, then **re-evaluates the WHERE against the new row**. If there's no longer enough, it updates nothing. That's why no explicit lock is needed here.

Calling the functions, and reading reservations

api/src/db/postgres-sql.ts · lines 48–62

```
movements: `
  SELECT id, sku, kind, qty, note, reservation_id, at FROM movements
   WHERE ($1::text IS NULL OR sku = $1)
   ORDER BY id DESC LIMIT $2`,
reserve: `SELECT inv_reserve($1, $2::jsonb) AS result`,
close: `SELECT inv_close_reservation($1::uuid, $2) AS result`,
reservations: `
  SELECT r.id, r.quote_ref, r.status, r.created_at, r.closed_at,
         coalesce(jsonb_agg(jsonb_build_object('sku', l.sku, 'qty', l.qty) ORDER BY l.sku)
                  FILTER (WHERE l.sku IS NOT NULL), '[]') AS lines
    FROM reservations r LEFT JOIN reservation_lines l ON l.reservation_id = r.id
   WHERE ($1::uuid IS NULL OR r.id = $1)
   GROUP BY r.id
   ORDER BY r.created_at DESC, r.id
   LIMIT $2`,
```

Reserve and close are one-line calls to the PL/pgSQL functions. Reading a reservation aggregates its lines into a JSON array with `jsonb_agg … FILTER (WHERE l.sku IS NOT NULL)`, so a reservation comes back as one row with its lines, the shape the API returns.

The repository class

api/src/db/postgres.ts · lines 48–79

```
export class PostgresRepo implements InventoryRepo {
  readonly kind = "postgres";
  private readonly pool: pg.Pool;

  /** Connection settings come from the standard PG* environment variables, or `config`. */
  constructor(config: pg.PoolConfig = {}) {
    this.pool = new pg.Pool({ max: 10, ...config });
  }

  async init(): Promise<void> {
    await this.pool.query(readFileSync(new URL("./schema.sql", import.meta.url), "utf8"));
  }

  async seedIfEmpty(seed: readonly SeedPart[]): Promise<boolean> {
    const { rows } = await this.pool.query<{ n: number }>(Q.count);
```

```
    if (rows[0].n > 0) return false;
    const client = await this.pool.connect();
    try {
      await client.query("BEGIN");
      for (const s of seed) {
        await client.query(Q.insertPart, [s.sku, s.name, s.category, s.unit, s.onHand,
s.reorderLevel, s.location]);
      }
      await client.query("COMMIT");
      return true;
    } catch (err) {
      await client.query("ROLLBACK");
      throw err;
    } finally {
      client.release();
    }
  }
```

`pg.Pool` keeps up to ten connections open and lends one per query. With no arguments it reads the standard `PGHOST`, `PGUSER`, `PGPASSWORD` and `PGDATABASE` variables, the same ones `psql` uses, so Compose and Kubernetes configure it with environment variables alone.

Seeding is the one place with an explicit transaction: `BEGIN`, the inserts, `COMMIT`, or `ROLLBACK` on any error. It needs one dedicated connection from the pool (`connect()`), because a transaction lives on a connection, and `finally` returns it whatever happens.

Telling “not found” from “not enough”

`api/src/db/postgres.ts · lines 112–142`

```
async receive(sku: string, qty: number, note: string): Promise<Part> {
  const { rows } = await this.pool.query<PartRow>(Q.receive, [sku, qty, note]);
  if (!rows[0]) throw new NotFoundError(`No part ${sku}`);
  return toPart(rows[0]);
}

async consume(sku: string, qty: number, note: string): Promise<Part> {
  const { rows } = await this.pool.query<PartRow>(Q.consume, [sku, qty, note]);
  if (rows[0]) return toPart(rows[0]);
  // No row updated: either the SKU doesn't exist or there wasn't enough available.
  const [part] = await this.getParts([sku]);
  if (!part) throw new NotFoundError(`No part ${sku}`);
  throw new ConflictError(`Only ${part.available} of ${sku} available`);
```

```
}

async movements({ sku, limit }: { sku?: string; limit: number }): Promise<Movement[]> {
  const { rows } = await this.pool.query<MovementRow>(Q.movements, [sku ?? null, limit]);
  return rows.map((m) => ({
    id: String(m.id), sku: m.sku, kind: m.kind, qty: m.qty, note: m.note, reservationId:
m.reservation_id, at: m.at.toISOString(),
  }));
}

async reserve(quoteRef: string, lines: readonly Line[]): Promise<ReserveResult> {
  const { rows } = await this.pool.query<{ result: { ok: true; id: string } | { ok: false;
shortages: Shortage[] } }>(
    Q.reserve, [quoteRef, JSON.stringify(mergeLines(lines))],
  );
  const result = rows[0].result;
  if (!result.ok) return { ok: false, shortages: result.shortages };
  return { ok: true, reservation: await this.reservation(result.id) };
}
```

When the consume statement updates nothing, it doesn't say why. A follow-up read decides between 404 and 409. That read isn't atomic with the update, but it doesn't need to be: it only chooses the error message.

For reserve, the lines are merged and sent as one JSON parameter. The function's answer is already parsed by the driver (Postgres JSONB comes back as a JavaScript object), and on success the full reservation is read back.

Closing, and mapping rows

api/src/db/postgres.ts · lines 241–274

```
  private async closeReservation(id: string, action: "release" | "fulfil"):
Promise<Reservation> {
    if (!UUID.test(id)) throw new NotFoundError(`No reservation ${id}`);
    const { rows } = await this.pool.query<{ result: string }>(Q.close, [id, action]);
    const result = rows[0].result;
    if (result === "not_found") throw new NotFoundError(`No reservation ${id}`);
    if (result !== "ok") throw new ConflictError(`Reservation is already ${result}`);
    return this.reservation(id);
  }

  private async reservation(id: string): Promise<Reservation> {
```

```
    const { rows } = await this.pool.query<ReservationRow>(Q.reservations, [id, 1]);
    if (!rows[0]) throw new NotFoundError(`No reservation ${id}`);
    return toReservation(rows[0]);
  }
}

/** `%` and `_` in a search box are literal characters, not wildcards. */
function escapeLike(s: string): string {
  return s.replace(/[\\%_]/g, (c) => `\\${c}`);
}

function toPart(r: PartRow): Part {
  return withDerived({
    sku: r.sku, name: r.name, category: r.category, unit: r.unit, onHand: r.on_hand, reserved:
r.reserved,
    reorderLevel: r.reorder_level, location: r.location, updatedAt: r.updated_at.toISOString(),
  });
}

function toReservation(r: ReservationRow): Reservation {
  return {
    id: r.id, quoteRef: r.quote_ref, status: r.status, lines: r.lines,
    createdAt: r.created_at.toISOString(), closedAt: r.closed_at ? r.closed_at.toISOString() :
null,
  };
}
```

An ID that isn't a UUID can't exist, so it's “not found” before reaching the database, where it would otherwise be a type error. The mapping functions are where `snake_case` columns become `camelCase` fields and `Date` objects become ISO strings, so nothing outside this file knows Postgres's naming.

GOING DEEPER

Optimistic concurrency, the other approach

Row locks are **pessimistic**: assume conflict, make others wait. **Optimistic** concurrency assumes conflicts are rare: each row has a `version`, an update says `WHERE version = $seen`, and if no row matched, someone else got there first and the caller retries. It never blocks, which suits long user edits (two people editing the same part's details), but under heavy contention retries pile up. Stock reservations, where contention is the whole point, suit locks better.

PART 5.7 · `api/src/db/mongo.ts`

The MongoDB repository

The same interface on a document database. MongoDB guarantees atomicity per document, so every rule is written into an update's filter, and the multi-document reservation becomes a small saga.

The approach

api/src/db/mongo.ts · lines 1–19

```
/**
 * MongoDB implementation.
 *
 * Each part is one document, and MongoDB updates a single document atomically. So every
 * stock rule is written into the update's filter: "add 4 to reserved, but only if
 * onHand - reserved is at least 4". If another request got there first, the filter no
 * longer matches and nothing changes. That gives the same guarantees as the Postgres
 * row-level checks, with no multi-document transaction, so a standalone mongod works.
 *
 * A reservation touches several documents, so it's done as a small saga:
 *   1. insert the reservation as "pending";
 *   2. hold each line with a conditional $inc, in SKU order;
 *   3. if any line fails, undo the holds already made and delete the pending reservation;
 *   4. otherwise mark it "active" and write the ledger.
 * Concurrent readers can briefly see a hold that's about to be undone. Stock is never
 * over-reserved; at worst, a racing reservation is refused when it could just have fit.
 * A crash between steps leaves a "pending" reservation that says exactly which holds to
 * undo. With a replica set, the same steps could run inside a transaction instead.
 */
```

The comment is the design in brief, and it's honest about the trade-off: no transaction, so a racing request can briefly see a hold that's about to be undone. Stock is never over-promised; the worst case is a refusal that a transaction would have accepted.

Documents

api/src/db/mongo.ts · lines 32–51

```
interface PartDoc {
  _id: string; // the SKU
  name: string; category: Part["category"]; unit: string;
  onHand: number; reserved: number; reorderLevel: number; location: string; updatedAt: Date;
}
interface MovementDoc { _id: string; seq: number; sku: string; kind: MovementKind; qty: number;
note: string; reservationId: string | null; at: Date }
interface ReservationDoc {
  _id: string; quoteRef: string; status: Reservation["status"] | "pending"; lines: Line[];
createdAt: Date; closedAt: Date | null;
}
interface CounterDoc { _id: string; value: number }
/** A shape is stored almost exactly as the API returns it: documents fit nested data like size
and position. */
interface ShapeDoc extends Omit<Shape, "id" | "createdAt" | "updatedAt"> { _id: string;
createdAt: Date; updatedAt: Date }
/** Conveyors too: the nested position stays nested. */
interface ConveyorDoc extends Omit<Conveyor, "id" | "createdAt" | "updatedAt"> { _id: string;
createdAt: Date; updatedAt: Date }
/** A splitter's rules are nested lists of objects: a document holds them as they are. */
interface SplitterDoc extends Omit<Splitter, "id" | "createdAt" | "updatedAt"> { _id: string;
createdAt: Date; updatedAt: Date }

/** `onHand - reserved >= qty`, as a query filter. */
const atLeastAvailable = (qty: number) => ({ $expr: { $gte: [{ $subtract: ["$onHand",
"$reserved"] }, qty] } });
```

The SKU *is* the part's `_id`, so lookups by SKU use the primary index and two parts can't share a SKU. A reservation **embeds** its lines, the document model's answer to Postgres's `reservation_lines` table: they're always read together, so they're stored together.

`atLeastAvailable` builds the rule as a filter. `$expr` lets a query compare two fields of the same document, “onHand minus reserved is at least qty”, which plain query operators can't.

Connecting, indexes and seeding

api/src/db/mongo.ts · lines 53–100

```
readonly kind = "mongo";
private readonly client: MongoClient;
```

```
  private readonly db: Db;
  private readonly parts: Collection<PartDoc>;
  private readonly moves: Collection<MovementDoc>;
  private readonly res: Collection<ReservationDoc>;
  private readonly counters: Collection<CounterDoc>;
  private readonly shapes: Collection<ShapeDoc>;
  private readonly conveyors: Collection<ConveyorDoc>;
  private readonly splitters: Collection<SplitterDoc>;

  constructor(url: string, dbName = "inventory") {
    this.client = new MongoClient(url, { serverSelectionTimeoutMS: 5000 });
    this.db = this.client.db(dbName);
    this.parts = this.db.collection<PartDoc>("parts");
    this.moves = this.db.collection<MovementDoc>("movements");
    this.res = this.db.collection<ReservationDoc>("reservations");
    this.counters = this.db.collection<CounterDoc>("counters");
    this.shapes = this.db.collection<ShapeDoc>("shapes");
    this.conveyors = this.db.collection<ConveyorDoc>("conveyors");
    this.splitters = this.db.collection<SplitterDoc>("splitters");
  }

  async init(): Promise<void> {
    await this.client.connect();
    await this.moves.createIndex({ sku: 1, seq: -1 });
    await this.moves.createIndex({ seq: -1 });
    await this.res.createIndex({ createdAt: -1 });
  }

  async seedIfEmpty(seed: readonly SeedPart[]): Promise<boolean> {
    if ((await this.parts.estimatedDocumentCount()) > 0) return false;
    const now = new Date();
    try {
      await this.parts.insertMany(
        seed.map((s) => ({
          _id: s.sku, name: s.name, category: s.category, unit: s.unit,
          onHand: s.onHand, reserved: 0, reorderLevel: s.reorderLevel, location: s.location,
updatedAt: now,
        })),
        { ordered: false },
      );
    } catch (err) {
```

```
    // Another API replica seeded at the same moment: duplicate keys are fine.
    if ((err as { code?: number }).code !== 11000) throw err;
  }
  return true;
}
```

Indexes are created on start; `createIndex` does nothing if one already exists. Seeding inserts with `ordered: false` and ignores duplicate-key errors (code 11000): with two API replicas starting together, both may try to seed, and the second's inserts simply collide with the first's.

Conditional updates

api/src/db/mongo.ts · lines 147–171

```
  const doc = await this.parts.findOneAndUpdate(
    { _id: sku },
    { $inc: { onHand: qty }, $set: { updatedAt: new Date() } },
    { returnDocument: "after" },
  );
  if (!doc) throw new NotFoundError(`No part ${sku}`);
  await this.log([{ sku, kind: "receive", qty, note, reservationId: null }]);
  return toPart(doc);
}

async consume(sku: string, qty: number, note: string): Promise<Part> {
  const doc = await this.parts.findOneAndUpdate(
    { _id: sku, ...atLeastAvailable(qty) },
    { $inc: { onHand: -qty }, $set: { updatedAt: new Date() } },
    { returnDocument: "after" },
  );
  if (!doc) {
    const [part] = await this.getParts([sku]);
    if (!part) throw new NotFoundError(`No part ${sku}`);
    throw new ConflictError(`Only ${part.available} of ${sku} available`);
  }
  await this.log([{ sku, kind: "consume", qty, note, reservationId: null }]);
  return toPart(doc);
}
```

`findOneAndUpdate` with the rule in the filter is the MongoDB equivalent of Postgres's `UPDATE … WHERE … RETURNING`. MongoDB applies a single-document update atomically: the filter is evaluated and the `$inc` applied as one step, so two racing consumes can't both pass the check. `returnDocument:`

`"after"` returns the updated document.

The reservation saga

api/src/db/mongo.ts · lines 180–204

```
  const want = mergeLines(lines);
  const id = randomUUID();
  await this.res.insertOne({ _id: id, quoteRef, status: "pending", lines: want, createdAt: new
Date(), closedAt: null });

  const held: Line[] = [];
  for (const line of want) {
    const r = await this.parts.updateOne(
      { _id: line.sku, ...atLeastAvailable(line.qty) },
      { $inc: { reserved: line.qty }, $set: { updatedAt: new Date() } },
    );
    if (r.modifiedCount !== 1) break;
    held.push(line);
  }

  if (held.length < want.length) {
    for (const line of held) await this.parts.updateOne({ _id: line.sku }, { $inc: { reserved:
-line.qty } });
    await this.res.deleteOne({ _id: id });
    return { ok: false, shortages: await this.shortages(want, want[held.length]) };
  }

  await this.res.updateOne({ _id: id }, { $set: { status: "active" } });
  await this.log(want.map((l) => ({ sku: l.sku, kind: "reserve" as const, qty: l.qty, note:
`Reserved for ${quoteRef}`, reservationId: id })));
  return { ok: true, reservation: toReservation((await this.res.findOne({ _id: id }))!) };
}
```

Step by step: insert the reservation as `pending`; hold each line with a conditional `$inc`, in SKU order; at the first line that doesn't fit, stop. If anything failed, undo the holds already made, delete the pending reservation, and report the shortages. Otherwise mark it `active` and write the ledger.

Why `pending` first? If the process crashed halfway, the holds would otherwise be orphaned: reserved stock with no reservation explaining it. A pending reservation records exactly which holds might exist, so a cleanup job can undo them. `reservations()` hides pending ones, so users never see a half-made reservation.

Closing exactly once, and the ledger

api/src/db/mongo.ts · lines 297–346

```
   * Claim the reservation first by flipping its status with a conditional update, so two
   * simultaneous "release" clicks can't both give the stock back.
   */
  private async closeReservation(id: string, action: "release" | "fulfil"):
Promise<Reservation> {
    const doc = await this.res.findOneAndUpdate(
      { _id: id, status: "active" },
      { $set: { status: action === "fulfil" ? "fulfilled" : "released", closedAt: new Date() }
},
      { returnDocument: "after" },
    );
    if (!doc) {
      const existing = await this.res.findOne({ _id: id });
      if (!existing || existing.status === "pending") throw new NotFoundError(`No reservation
${id}`);
      throw new ConflictError(`Reservation is already ${existing.status}`);
    }
    for (const { sku, qty } of doc.lines) {
      const inc = action === "fulfil" ? { reserved: -qty, onHand: -qty } : { reserved: -qty };
      await this.parts.updateOne({ _id: sku }, { $inc: inc, $set: { updatedAt: new Date() } });
    }
    await this.log(doc.lines.map((l) => ({
      sku: l.sku, kind: action, qty: l.qty, note: `${action === "fulfil" ? "Shipped" :
"Released"} for ${doc.quoteRef}`, reservationId: id,
    })));
    return toReservation(doc);
  }

  /** Current shortages for `want`. If a race already freed the stock, still report the line
that failed. */
  private async shortages(want: Line[], failed: Line): Promise<Shortage[]> {
    const current = await this.getParts(want.map((l) => l.sku));
    const bySku = new Map(current.map((p) => [p.sku, p]));
    const out: Shortage[] = [];
    for (const { sku, qty } of want) {
      const p = bySku.get(sku);
      const available = p?.available ?? 0;
      if (available < qty || sku === failed.sku) out.push({ sku, need: qty, available, known: p
```

```
!== undefined });
    }
    return out;
  }

  /** Ledger entries get an increasing sequence number from a counter document, so "newest
first" is exact. */
  private async log(entries: Omit<MovementDoc, "_id" | "seq" | "at">[]): Promise<void> {
    if (entries.length === 0) return;
    const counter = await this.counters.findOneAndUpdate(
      { _id: "movements" },
      { $inc: { value: entries.length } },
      { upsert: true, returnDocument: "after" },
    );
    const last = counter!.value;
    const at = new Date();
    await this.moves.insertMany(entries.map((e, i) => ({ ...e, _id: randomUUID(), seq: last -
entries.length + i + 1, at })));
  }
}
```

Closing starts by **claiming** the reservation: a conditional update from `active` to the new status. Exactly one of several simultaneous requests can match `status: "active"`; the others find nothing and get a conflict. Only the winner then adjusts the parts.

The ledger needs “newest first” to be exact, but timestamps can tie and random IDs don't sort. A counter document, incremented atomically with `$inc` and `upsert`, hands out a block of sequence numbers in one round trip: the classic MongoDB auto-increment pattern.

GOING DEEPER

Doing it with a MongoDB transaction

On a replica set (even a one-node one, `--replSet rs0`), the saga can be replaced by `session.withTransaction(async () => { … })` around the same conditional updates. The driver retries the whole callback on transient conflicts. The reservation then becomes invisible until committed, like Postgres's. The saga version was chosen so the demo runs on a plain standalone `mongo` container; switching is a contained change inside one method.

PART 5.8 · `api/src/db/index.ts`

Choosing a database, and the service layer

One environment variable picks the implementation. A thin service layer sits between the routes and the repository.

The factory

api/src/db/index.ts · lines 1–31

```
import type { DbKind, InventoryRepo } from "../domain/types.ts";
import { MemoryRepo } from "./memory.ts";

/**
 * Picks the database from the DB environment variable: "postgres", "mongo" or "memory".
 * The drivers are imported only when chosen, so the API runs with neither installed in memory
mode.
 */
export async function createRepo(env: NodeJS.ProcessEnv = process.env): Promise<InventoryRepo>
{
  const kind = (env.DB ?? "memory") as DbKind;
  switch (kind) {
    case "postgres": {
      const { PostgresRepo } = await import("./postgres.ts");
      return new PostgresRepo(); // reads PGHOST, PGUSER, PGPASSWORD, PGDATABASE, PGPORT
    }
    case "mongo": {
      const { MongoRepo } = await import("./mongo.ts");
      return new MongoRepo(mongoUrl(env), env.MONGO_DB ?? "inventory");
    }
    case "memory":
      return new MemoryRepo();
    default:
      throw new Error(`DB must be postgres, mongo or memory, not "${kind}"`);
  }
}

export function mongoUrl(env: NodeJS.ProcessEnv): string {
  if (env.MONGO_URL) return env.MONGO_URL;
  const host = env.MONGO_HOST ?? "localhost";
  const auth = env.MONGO_USER ?
```

```
  `${encodeURIComponent(env.MONGO_USER)}:${encodeURIComponent(env.MONGO_PASSWORD ?? "")}@` : "";
  return `mongodb://${auth}${host}:27017/?authSource=admin`;
}
```

`await import()` loads a driver only when it's chosen. The API in memory mode runs with neither `pg` nor `mongodb` installed, which is also how the unit tests run anywhere.

`mongoUrl` builds a connection string from parts, URL-encoding the user and password: a password containing `@` or `:` would otherwise break the URL. `authSource=admin` is where the official image creates its root user.

Checking and reserving designs

api/src/service.ts · lines 1–42

```
/**
 * What the HTTP routes call. Turns designs into stock lines and answers "can we build
 * this?" without touching the database's own rules, which live in the repositories.
 */
import { parseDesign, plan, type Requirement } from "./domain/design.ts";
import { ValidationError } from "./domain/errors.ts";
import type { InventoryRepo, Reservation, Shortage } from "./domain/types.ts";

export interface CheckLine extends Requirement {
  name: string | null;
  unit: string | null;
  available: number;
  short: number;
}

export interface CheckResult {
  ok: boolean;
  lines: CheckLine[];
  warnings: string[];
}

export async function checkDesign(repo: InventoryRepo, input: unknown): Promise<CheckResult> {
  const { requirements, warnings } = plan(parseDesign(input));
  const parts = new Map((await repo.getParts(requirements.map((r) => r.sku))).map((p) =>
[p.sku, p]));
  const lines = requirements.map((r) => {
    const p = parts.get(r.sku);
    const available = p?.available ?? 0;
```

```
    return { ...r, name: p?.name ?? null, unit: p?.unit ?? null, available, short: Math.max(0,
r.qty - available) };
  });
  return { ok: lines.every((l) => l.short === 0), lines, warnings };
}

export type ReserveOutcome =
  | { ok: true; reservation: Reservation; warnings: string[] }
  | { ok: false; shortages: Shortage[]; warnings: string[] };

export async function reserveDesign(repo: InventoryRepo, quoteRef: string, input: unknown):
Promise<ReserveOutcome> {
  const { requirements, warnings } = plan(parseDesign(input));
  if (requirements.length === 0) throw new ValidationError(["nothing in this design maps to
stock", ...warnings]);
  const result = await repo.reserve(quoteRef, requirements.map(({ sku, qty }) => ({ sku, qty
})));
  return { ...result, warnings };
}
```

The service turns a design into requirements, then asks the repository. `checkDesign` is read-only and answers “can we build it?”; `reserveDesign` makes the promise. They're separate on purpose: a check is a snapshot that can be stale a second later, so reserving always re-checks inside the database.

A design with nothing reservable (every item warned about) is rejected up front, rather than creating an empty reservation.

PART 5.9 · `api/src/app.ts`

The HTTP API

About a dozen routes on Node's built-in `http` module, with a twenty-line router.

A tiny router

`api/src/app.ts · lines 1–28`

```
/**
 * The HTTP API on Node's built-in http module: about a dozen routes don't need a framework.
 * createApp() returns a plain request handler, so tests can mount it on a random port.
 */
```

```
import type { IncomingMessage, ServerResponse } from "node:http";
import { hostname } from "node:os";
import { isQty, isSku } from "./domain/design.ts";
import { NotFoundError, ValidationError } from "./domain/errors.ts";
import { parseNewPart, parsePartPatch } from "./domain/newPart.ts";
import { parseNewShape, parseShapePatch } from "./domain/shapes.ts";
import { parseConveyorPatch, parseNewConveyor } from "./domain/conveyors.ts";
import { parseNewSplitter, parseSplitterPatch } from "./domain/splitters.ts";
import type { InventoryRepo } from "./domain/types.ts";
import { checkDesign, reserveDesign } from "./service.ts";

type Params = Record<string, string>;
type Handler = (ctx: { params: Params; query: URLSearchParams; body: unknown }) =>
Promise<[number, unknown]>;

const MAX_BODY = 100_000; // bytes

export function createApp(repo: InventoryRepo, version = "1.0.0") {
  const routes: { method: string; pattern: RegExp; keys: string[]; handler: Handler }[] = [];
  const route = (method: string, path: string, handler: Handler) => {
    const keys: string[] = [];
    const pattern = new RegExp(`^${path.replace(/:(\w+)/g, (_, k: string) => (keys.push(k), "
([^/]+)"))}$`);
    routes.push({ method, pattern, keys, handler });
  };
```

Each route's path, like `/api/parts/:sku/receive`, is turned into a regular expression where `:sku` becomes a capture group, `([^/]+)`, and its name is remembered in `keys`. The comma expression `(keys.push(k), "([^/]+)")` records the name and returns the replacement in one go.

A handler receives path parameters, the query string and the parsed body, and returns a `[status, payload]` pair. Handlers never touch the response object, which makes them easy to read and test.

Routes

api/src/app.ts · lines 29–73

```
// ---- Health and info ----
route("GET", "/api/health/live", async () => [200, { ok: true }]);
route("GET", "/api/health/ready", async () => {
  await repo.ping();
  return [200, { ok: true, db: repo.kind }];
});
```

```
route("GET", "/api/info", async () => [200, { db: repo.kind, host: hostname(), version }]);

// ---- Parts ----
route("GET", "/api/parts", async ({ query }) => [200, await repo.listParts({
  search: query.get("search") ?? undefined,
  lowOnly: query.get("low") === "true",
})]);
route("POST", "/api/parts", async ({ body }) => [201, await
repo.createPart(parseNewPart(body))]);
route("GET", "/api/parts/:sku", async ({ params }) => {
  const [part] = await repo.getParts([sku(params.sku)]);
  return part ? [200, part] : [404, { error: `No part ${params.sku}` }];
});
route("PATCH", "/api/parts/:sku", async ({ params, body }) => [200, await
repo.updatePart(sku(params.sku), parsePartPatch(body))]);
route("POST", "/api/parts/:sku/receive", async ({ params, body }) => {
  const { qty, note } = adjustment(body);
  return [200, await repo.receive(sku(params.sku), qty, note)];
});
route("POST", "/api/parts/:sku/consume", async ({ params, body }) => {
  const { qty, note } = adjustment(body);
  return [200, await repo.consume(sku(params.sku), qty, note)];
});
route("GET", "/api/movements", async ({ query }) => {
  const s = query.get("sku");
  return [200, await repo.movements({ sku: s ? sku(s) : undefined, limit: limit(query, 50, 500)
})];
});

// ---- Designs and reservations ----
route("POST", "/api/check", async ({ body }) => [200, await checkDesign(repo, field(body,
"design"))]);
route("POST", "/api/reservations", async ({ body }) => {
  const quoteRef = field(body, "quoteRef");
  if (typeof quoteRef !== "string" || !/^[\w.-]{1,64}$/.test(quoteRef)) {
    throw new ValidationError(["quoteRef must be 1-64 letters, digits, dots, dashes or
underscores"]);
  }
  const out = await reserveDesign(repo, quoteRef, field(body, "design"));
  return out.ok ? [201, out] : [409, { error: "Not enough stock", ...out }];
});
```

```
route("GET", "/api/reservations", async ({ query }) => [200, await
repo.reservations(limit(query, 50, 200))]);
route("POST", "/api/reservations/:id/release", async ({ params }) => [200, await
repo.release(params.id)]);
route("POST", "/api/reservations/:id/fulfil", async ({ params }) => [200, await
repo.fulfil(params.id)]);
```

Health checks come in two kinds, matching what Kubernetes asks: **live** means the process is up; **ready** means it can reach the database. A database outage then takes pods out of the load balancer without restarting them.

A refused reservation is a **409 Conflict** carrying the shortages, not a 500: it's a normal outcome the UI shows the user.

Handling a request

`api/src/app.ts · lines 125–164`

```
  return async function handle(req: IncomingMessage, res: ServerResponse): Promise<void> {
    const started = performance.now();
    const url = new URL(req.url ?? "/", "http://localhost");

    // The robot arm app runs on its own dev server (another origin), so the browser asks
    // first (a CORS "preflight") and only sends the request if these headers allow it.
    const cors = corsHeaders(req.headers.origin);
    if (req.method === "OPTIONS") {
      res.writeHead(cors ? 204 : 403, cors ?? {});
      res.end();
      return;
    }

    let status = 500;
    let payload: unknown;
    try {
      const matching = routes.filter((r) => r.pattern.test(url.pathname));
      const r = matching.find((m) => m.method === req.method);
      if (!r) {
        [status, payload] = matching.length ? [405, { error: "Method not allowed" }] : [404, {
error: "Not found" }];
      } else {
        const m = r.pattern.exec(url.pathname)!;
        const params = Object.fromEntries(r.keys.map((k, i) => [k, decode(m[i + 1])]));
        const body = req.method === "POST" || req.method === "PATCH" ? await readJson(req) :
```

```
undefined;
        [status, payload] = await r.handler({ params, query: url.searchParams, body });
      }
    } catch (err) {
      const e = err as Error & { status?: number; problems?: string[] };
      status = e.status ?? 500;
      payload = status === 500 ? { error: "Internal error" } : { error: e.message, ...
(e.problems ? { problems: e.problems } : {}) };
      if (status === 500) console.error(err);
    }
    const text = JSON.stringify(payload);
    res.writeHead(status, { "content-type": "application/json; charset=utf-8", "cache-control":
"no-store", ...cors });
    res.end(text);
    if (process.env.LOG_REQUESTS !== "0" && !url.pathname.startsWith("/api/health")) {
      console.log(`${req.method} ${url.pathname} ${status} ${(performance.now() -
started).toFixed(1)}ms`);
    }
  };
}
```

Find the routes whose path matches; if none matches the method, it's 405 (path exists) or 404 (doesn't). Parse the body only for POST. Any thrown error with a `status` becomes that status with its message; anything else is a 500 whose details go to the log, not to the client.

Reading bodies safely

api/src/app.ts · lines 166–213

```
async function readJson(req: IncomingMessage): Promise<unknown> {
  const chunks: Buffer[] = [];
  let size = 0;
  for await (const chunk of req) {
    size += (chunk as Buffer).length;
    if (size > MAX_BODY) throw new ValidationError([`body is larger than ${MAX_BODY} bytes`]);
    chunks.push(chunk as Buffer);
  }
  if (size === 0) return {};
  try {
    return JSON.parse(Buffer.concat(chunks).toString("utf8"));
  } catch {
    throw new ValidationError(["body is not valid JSON"]);
```

```
  }
}

function field(body: unknown, key: string): unknown {
  return body && typeof body === "object" ? (body as Record<string, unknown>)[key] : undefined;
}

function sku(s: string): string {
  if (!isSku(s)) throw new ValidationError([`"${s}" is not a valid SKU`]);
  return s;
}

function adjustment(body: unknown): { qty: number; note: string } {
  const qty = field(body, "qty");
  const note = field(body, "note") ?? "";
  const problems: string[] = [];
  if (!isQty(qty)) problems.push("qty must be a whole number from 1 to 100000");
  if (typeof note !== "string" || note.length > 200) problems.push("note must be text up to 200
characters");
  if (problems.length) throw new ValidationError(problems);
  return { qty: qty as number, note: note as string };
}

function limit(query: URLSearchParams, fallback: number, max: number): number {
  const n = Number(query.get("limit") ?? fallback);
  return Number.isInteger(n) && n > 0 ? Math.min(n, max) : fallback;
}

/** A malformed escape like "%E0" in a path is the client's mistake, not a server error. */
function decode(s: string): string {
  try {
    return decodeURIComponent(s);
  } catch {
    throw new ValidationError([`"${s}" is not a valid URL segment`]);
  }
}
```

The body is read in chunks with a running size check, so a client can't make the server buffer a gigabyte. Invalid JSON, a bad quantity or a malformed URL escape are all 400s with a reason. Quantities must be whole numbers from 1 to 100,000; `limit` clamps page sizes so no one can ask for a million rows.

Starting up

api/src/server.ts · lines 1–38

```
/**
 * Entry point: connect to the database (retrying while it boots), create the schema,
 * seed the starting stock if the database is empty, then listen on PORT (8080).
 */
import { createServer } from "node:http";
import { createApp } from "./app.ts";
import { createRepo } from "./db/index.ts";
import { SEED_PARTS } from "./domain/seed.ts";

const port = Number(process.env.PORT ?? 8080);
const repo = await createRepo();

for (let attempt = 1; ; attempt++) {
  try {
    await repo.init();
    await repo.ping();
    break;
  } catch (err) {
    if (attempt >= 30) throw err;
    console.log(`Waiting for ${repo.kind} (attempt ${attempt}): ${(err as Error).message}`);
    await new Promise((r) => setTimeout(r, 2000));
  }
}
if (await repo.seedIfEmpty(SEED_PARTS)) console.log(`Seeded ${SEED_PARTS.length} parts`);

const server = createServer(createApp(repo));
server.listen(port, () => console.log(`Inventory API on :${port}, database: ${repo.kind}`));

// Kubernetes sends SIGTERM before stopping a pod: finish in-flight requests, then close the
pool.
for (const signal of ["SIGTERM", "SIGINT"] as const) {
  process.once(signal, () => {
    server.close(async () => {
      await repo.close();
      process.exit(0);
    });
    setTimeout(() => process.exit(1), 10_000).unref();
```

```
  });
}
```

Top-level `await` (allowed in ES modules) keeps startup linear. The retry loop matters in containers: Compose and Kubernetes may start the API before the database accepts connections, so it tries for up to a minute instead of crashing.

On `SIGTERM`, which Kubernetes sends before stopping a pod, the server stops accepting connections, finishes in-flight requests, closes the database pool, and exits. A 10 s backstop timer forces the exit; `unref()` keeps that timer from holding the process open by itself.

GOING DEEPER

Idempotency keys

If the network drops after the server reserved stock but before the browser got the answer, the user clicks again and reserves twice. Payment APIs solve this with an **idempotency key**: the client sends a unique key per intended action, the server stores the response under it, and a retry with the same key gets the stored response instead of a second reservation. Here, a unique constraint on `quote_ref` would be the simplest version.

PART 5.10 · `api/tests/contract.ts`

Contract tests: one suite, every database

The same nineteen tests run against each implementation. If MongoDB and Postgres disagree about anything the API relies on, one of them fails.

A test suite as a function

api/tests/contract.ts · lines 1–33

```
/**
 * The behaviour every InventoryRepo must have. Run against each database by
 * memory.test.ts, postgres.test.ts and mongo.test.ts, so the three can't drift apart.
 */
import { after, before, beforeEach, describe, test } from "node:test";
import assert from "node:assert/strict";
import { ConflictError, NotFoundError } from "../src/domain/errors.ts";
import type { InventoryRepo, SeedPart } from "../src/domain/types.ts";

export const FIXTURE: SeedPart[] = [
```

```
  { sku: "BAR-4040-CLR", name: "40x40 extrusion", category: "extrusion", unit: "bar (3 m)",
onHand: 10, reorderLevel: 4, location: "A-1" },
  { sku: "LA-300", name: "Linear actuator", category: "actuator", unit: "piece", onHand: 3,
reorderLevel: 2, location: "D-1" },
  { sku: "MC-1", name: "Motion controller", category: "controller", unit: "piece", onHand: 2,
reorderLevel: 2, location: "D-2" },
  { sku: "PL-200", name: "Mounting plate 200_mm", category: "plate", unit: "plate", onHand: 20,
reorderLevel: 5, location: "B-1" },
];

export function contractTests(name: string, makeRepo: () => Promise<InventoryRepo>) {
  describe(`${name} repository`, () => {
    let repo: InventoryRepo;

    before(async () => {
      repo = await makeRepo();
      await repo.init();
    });
    beforeEach(async () => {
      await repo.reset();
      assert.equal(await repo.seedIfEmpty(FIXTURE), true);
    });
    after(async () => {
      await repo.reset();
      await repo.close();
    });
```

`contractTests(name, makeRepo)` defines the suite for whatever repository the factory makes. `beforeEach` resets and reseeds, so tests can't leak state into each other, and the seed fixture is small enough to reason about: three actuators, two controllers.

Behaviour, including the edge cases

api/tests/contract.ts · lines 102–145

```
test("consume can't take stock that's reserved for a quote", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "LA-300", qty: 2 }]);
  assert.equal(r.ok, true);
  await assert.rejects(repo.consume("LA-300", 2, "line-side"), ConflictError);
  const p = await repo.consume("LA-300", 1, "line-side");
  assert.equal(p.onHand, 2);
  assert.equal(p.available, 0);
```

```
});

test("reserve holds every line and records who for", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "LA-300", qty: 2 }, { sku: "BAR-4040-CLR", qty: 3
}]);
  assert.ok(r.ok);
  assert.equal(r.reservation.status, "active");
  assert.equal(r.reservation.quoteRef, "Q-1");
  assert.deepEqual(r.reservation.lines, [{ sku: "BAR-4040-CLR", qty: 3 }, { sku: "LA-300", qty:
2 }]);
  const [bar, la] = await repo.getParts(["BAR-4040-CLR", "LA-300"]);
  assert.equal(bar.reserved, 3);
  assert.equal(la.available, 1);
  const moves = await repo.movements({ limit: 10 });
  assert.deepEqual(moves.map((m) => m.kind), ["reserve", "reserve"]);
  assert.ok(moves.every((m) => m.reservationId === r.reservation.id));
});

test("reserve adds up lines for the same SKU", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "LA-300", qty: 1 }, { sku: "LA-300", qty: 2 }]);
  assert.ok(r.ok);
  assert.deepEqual(r.reservation.lines, [{ sku: "LA-300", qty: 3 }]);
  assert.ok(!(await repo.reserve("Q-2", [{ sku: "LA-300", qty: 1 }])).ok);
});

test("reserve is all or nothing, and reports every shortage", async () => {
  const r = await repo.reserve("Q-1", [
    { sku: "BAR-4040-CLR", qty: 2 }, { sku: "LA-300", qty: 5 }, { sku: "GHOST-1", qty: 1 }, {
sku: "PL-200", qty: 1 },
  ]);
  assert.equal(r.ok, false);
  assert.ok(!r.ok);
  assert.deepEqual(
    r.shortages.map((s) => ({ ...s })).sort((a, b) => a.sku.localeCompare(b.sku)),
    [{ sku: "GHOST-1", need: 1, available: 0, known: false }, { sku: "LA-300", need: 5,
available: 3, known: true }],
  );
  for (const p of await repo.listParts()) assert.equal(p.reserved, 0, `${p.sku} was left
reserved`);
  assert.deepEqual(await repo.reservations(10), []);
```

```
  assert.deepEqual(await repo.movements({ limit: 10 }), []);
});
```

These tests describe rules, not implementations. “Consume can't take reserved stock” and “reserve is all or nothing” check the state after the call too, not just the return value: after a refused reservation, every part must have zero reserved, and there must be no reservation and no ledger line.

Concurrency, tested

api/tests/contract.ts · lines 147–185

```
test("concurrent reservations never over-reserve", async () => {
  const results = await Promise.all(Array.from({ length: 8 }, (_, i) => repo.reserve(`Q-${i}`,
[{ sku: "LA-300", qty: 1 }])));
  assert.equal(results.filter((r) => r.ok).length, 3);
  const [la] = await repo.getParts(["LA-300"]);
  assert.equal(la.reserved, 3);
  assert.equal(la.available, 0);
});

test("release gives the stock back", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "MC-1", qty: 2 }]);
  assert.ok(r.ok);
  const released = await repo.release(r.reservation.id);
  assert.equal(released.status, "released");
  assert.ok(released.closedAt);
  const [mc] = await repo.getParts(["MC-1"]);
  assert.equal(mc.reserved, 0);
  assert.equal(mc.onHand, 2);
});

test("fulfil ships the reserved stock", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "MC-1", qty: 2 }, { sku: "PL-200", qty: 4 }]);
  assert.ok(r.ok);
  const done = await repo.fulfil(r.reservation.id);
  assert.equal(done.status, "fulfilled");
  const [mc, pl] = await repo.getParts(["MC-1", "PL-200"]);
  assert.deepEqual([mc.onHand, mc.reserved, pl.onHand, pl.reserved], [0, 0, 16, 0]);
  const kinds = (await repo.movements({ limit: 10 })).map((m) => m.kind);
  assert.deepEqual(kinds.slice(0, 2), ["fulfil", "fulfil"]);
});
```

```
test("a reservation can only be closed once, even by simultaneous requests", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "LA-300", qty: 2 }]);
  assert.ok(r.ok);
  const outcomes = await Promise.allSettled([repo.release(r.reservation.id),
repo.release(r.reservation.id), repo.fulfil(r.reservation.id)]);
  assert.equal(outcomes.filter((o) => o.status === "fulfilled").length, 1);
  assert.ok(outcomes.filter((o) => o.status === "rejected").every((o) => (o as
PromiseRejectedResult).reason instanceof ConflictError));
  const [la] = await repo.getParts(["LA-300"]);
  assert.equal(la.reserved, 0);
});
```

`Promise.all` fires eight reservations for one actuator each, at once, when three exist. Exactly three must succeed. Against Postgres or MongoDB these are real concurrent requests on separate connections; against the in-memory store they interleave on the event loop. Either way, a broken implementation over-reserves and fails.

The close test fires two releases and a fulfil at the same reservation. Exactly one may win; the losers must be conflicts, and the stock must be released once.

Running the suite per database

api/tests/postgres.test.ts · lines 1–14

```
/**
 * Runs the contract against a real PostgreSQL. Skipped unless PGHOST is set, e.g.:
 *   docker compose --profile test run --rm api-test
 * Uses its own database name (PGDATABASE), and wipes it between tests.
 */
import { test } from "node:test";
import { contractTests } from "./contract.ts";

if (process.env.PGHOST) {
  const { PostgresRepo } = await import("../src/db/postgres.ts");
  contractTests("postgres", async () => new PostgresRepo());
} else {
  test("postgres repository", { skip: "set PGHOST (and PGUSER, PGPASSWORD, PGDATABASE) to run"
}, () => {});
}
```

Database runs skip themselves unless connection settings are present, so `npm test` works on any machine, and `docker compose --profile test run api-test` runs everything against throwaway Postgres and MongoDB containers.

GOING DEEPER

What concurrency tests can and can't prove

A test that fires eight requests at once and sees three succeed shows the race is handled **in that run**. It can't prove there's no interleaving that fails, because the scheduler decides the timing. That's why the design doesn't rely on the tests alone: constraints in the database make the bad state impossible to store, and the tests check the error handling and the happy path around them.

PART 5.11 · `api/src/domain/newPart.ts`

Adding a part, end to end

The newest feature, followed through every layer: a form in the browser, validation, one route, and the same operation on three storage back ends, each safe against two people adding the same SKU at once.

Validating a new part

api/src/domain/newPart.ts · lines 1–43

```
/** Validates a new part sent from the "Add part" form. */
import { isSku } from "./design.ts";
import { ValidationError } from "./errors.ts";
import type { Category, SeedPart } from "./types.ts";

export const CATEGORIES: readonly Category[] = ["extrusion", "plate", "fastener", "motor",
"actuator", "controller", "gripper"];

const MAX_QTY = 100_000;

export function parseNewPart(input: unknown): SeedPart {
  const b = (input && typeof input === "object" ? input : {}) as Record<string, unknown>;
  const problems: string[] = [];
  const text = (key: string, max: number, required = true): string => {
    const v = b[key];
```

```
    if (v === undefined && !required) return "";
    if (typeof v !== "string" || (required && v.trim() === "") || v.trim().length > max) {
      problems.push(`${key} must be text${required ? "" : " (optional)"} up to ${max}
characters`);
      return "";
    }
    return v.trim();
  };
  const whole = (key: string): number => {
    const v = b[key] ?? 0;
    if (typeof v !== "number" || !Number.isInteger(v) || v < 0 || v > MAX_QTY) {
      problems.push(`${key} must be a whole number from 0 to ${MAX_QTY}`);
      return 0;
    }
    return v;
  };

  const sku = typeof b.sku === "string" ? b.sku.trim().toUpperCase() : "";
  if (!isSku(sku)) problems.push('sku must be 2-32 capital letters, digits or dashes, like "PL-
400"');
  const name = text("name", 80);
  const category = b.category as Category;
  if (!CATEGORIES.includes(category)) problems.push(`category must be one of
${CATEGORIES.join(", ")}`);
  const unit = text("unit", 30);
  const location = text("location", 20, false);
  const onHand = whole("onHand");
  const reorderLevel = whole("reorderLevel");

  if (problems.length) throw new ValidationError(problems);
  return { sku, name, category, unit, onHand, reorderLevel, location };
}
```

The same approach as `parseDesign`: check every field, collect every problem, and build a clean object from the checked values. Two small kindnesses happen here rather than in the form: the SKU is trimmed and upper-cased, so `pl-400` becomes `PL-400`, and text fields are trimmed so a stray space can't create a second “Mounting plate”.

`text` and `whole` are local helpers, closures over `b` and `problems`, so each field is one line. Quantities are whole numbers from 0 (a part can be added before any stock arrives) up to 100,000.

One route

api/src/app.ts · line 42

```
route("POST", "/api/parts", async ({ body }) => [201, await
repo.createPart(parseNewPart(body))]);
```

A `POST` to the collection creates a member: the REST convention. It returns **201 Created** with the new part. A taken SKU is a `ConflictError` from the repository, so it becomes **409**; invalid fields are a `ValidationError`, **400**, listing every problem. The route itself is one line because every decision lives in a layer that can be tested on its own.

The contract: create, and record the starting stock

api/src/domain/types.ts · lines 113–117

```
/**
 * Adds a new part. Initial stock is written to the ledger as a receipt, so the
 * ledger still explains every unit. Throws ConflictError if the SKU already exists.
 */
createPart(part: SeedPart): Promise<Part>;
```

The interface says what all three implementations must do, including the part that's easy to forget: starting stock is written to the ledger as a receipt. Otherwise the ledger would explain every change except the first, and “why do we have 12 of these?” would have no answer.

In memory

api/src/db/memory.ts · lines 54–60

```
async createPart(s: SeedPart): Promise<Part> {
  if (this.parts.has(s.sku)) throw new ConflictError(`Part ${s.sku} already exists`);
  const p: PartRow = { ...s, reserved: 0, updatedAt: new Date().toISOString() };
  this.parts.set(s.sku, p);
  if (s.onHand > 0) this.log(s.sku, "receive", s.onHand, "Initial stock", null);
  return out(p);
}
```

Check, insert, log. Atomic because nothing awaits in between, as with every method in this class.

In PostgreSQL: one statement

api/src/db/postgres-sql.ts · lines 15–24

```
createPart: `
  WITH p AS (
    INSERT INTO parts (sku, name, category, unit, on_hand, reorder_level, location)
    VALUES ($1, $2, $3, $4, $5, $6, $7)
    ON CONFLICT (sku) DO NOTHING
    RETURNING sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at
  ), m AS (
    INSERT INTO movements (sku, kind, qty, note) SELECT sku, 'receive', on_hand, 'Initial
stock' FROM p WHERE on_hand > 0
  )
  SELECT * FROM p`,
```

api/src/db/postgres.ts · lines 99–104

```
async createPart(s: SeedPart): Promise<Part> {
  const { rows } = await this.pool.query<PartRow>(Q.createPart, [s.sku, s.name, s.category,
s.unit, s.onHand, s.reorderLevel, s.location]);
  // ON CONFLICT DO NOTHING returns no row when the SKU is taken, even if two requests race.
  if (!rows[0]) throw new ConflictError(`Part ${s.sku} already exists`);
  return toPart(rows[0]);
}
```

`ON CONFLICT (sku) DO NOTHING` relies on the primary key. If the SKU exists, or another request inserts it a microsecond earlier, Postgres inserts nothing and `RETURNING` returns no row, which the repository turns into a conflict. There's no check-then-insert gap to race through: the unique index is the check.

The ledger insert is chained in a CTE and only runs for rows the insert returned with stock above zero (the movements table requires `qty > 0`). The foreign key from the movement to the new part is satisfied within the same statement, because Postgres checks foreign keys at the end of the statement, not in the middle.

In MongoDB: let the unique index decide

api/src/db/mongo.ts · lines 124–138

```
async createPart(s: SeedPart): Promise<Part> {
  const doc: PartDoc = {
    _id: s.sku, name: s.name, category: s.category, unit: s.unit,
    onHand: s.onHand, reserved: 0, reorderLevel: s.reorderLevel, location: s.location,
updatedAt: new Date(),
  };
  try {
    await this.parts.insertOne(doc);
  } catch (err) {
    // The SKU is the _id, so the unique index rejects a duplicate, even if two requests race.
    if ((err as { code?: number }).code === 11000) throw new ConflictError(`Part ${s.sku}
already exists`);
    throw err;
  }
  if (s.onHand > 0) await this.log([{ sku: s.sku, kind: "receive", qty: s.onHand, note:
"Initial stock", reservationId: null }]);
  return toPart(doc);
}
```

The SKU is the document's `_id`, and `_id` always has a unique index. So `insertOne` either inserts or fails with error code 11000 (duplicate key), which becomes a conflict. As in Postgres, the database's uniqueness guarantee replaces any “does it exist?” query, which would leave a gap for a racing request.

Tested on every database

api/tests/contract.ts · lines 66–85

```
test("createPart adds a part and logs its initial stock as a receipt", async () => {
  const p = await repo.createPart({ sku: "PL-400", name: "Mounting plate, 400 mm", category:
"plate", unit: "plate", onHand: 12, reorderLevel: 5, location: "B-01-4" });
  assert.deepEqual([p.sku, p.onHand, p.reserved, p.available, p.low], ["PL-400", 12, 0, 12,
false]);
  assert.equal((await repo.getParts(["PL-400"]))[0].name, "Mounting plate, 400 mm");
  const [m] = await repo.movements({ sku: "PL-400", limit: 5 });
  assert.deepEqual([m.kind, m.qty, m.note], ["receive", 12, "Initial stock"]);
  // A part can start empty: no ledger line, since nothing was received.
  await repo.createPart({ sku: "SRV-750", name: "Servo motor, 750 W", category: "motor", unit:
"piece", onHand: 0, reorderLevel: 2, location: "" });
  assert.equal((await repo.movements({ sku: "SRV-750", limit: 5 })).length, 0);
  assert.equal((await repo.getParts(["SRV-750"]))[0].low, true);
});

test("createPart refuses a SKU that exists, even from simultaneous requests", async () => {
  await assert.rejects(repo.createPart({ ...FIXTURE[1], onHand: 99 }), ConflictError);
  assert.equal((await repo.getParts(["LA-300"]))[0].onHand, 3);
  const part = { sku: "GR-2", name: "Vacuum gripper", category: "gripper" as const, unit:
"piece", onHand: 1, reorderLevel: 0, location: "" };
  const results = await Promise.allSettled([repo.createPart(part), repo.createPart(part),
repo.createPart(part)]);
  assert.equal(results.filter((r) => r.status === "fulfilled").length, 1);
  assert.equal((await repo.movements({ sku: "GR-2", limit: 5 })).length, 1);
});
```

The first test checks the part, its derived fields, and the “Initial stock” receipt, then that a part added with no stock writes no ledger line and starts as low stock. The second fires three simultaneous creates of the same SKU: exactly one may succeed, and there must be exactly one ledger line. Against real PostgreSQL, the same statement was also fired from ten connections at once: one part, one receipt.

The form

web/src/AddPartPanel.tsx · lines 4–44

```
const CATEGORIES: { id: Category; unit: string; prefix: string; example: string }[] = [
  { id: "extrusion", unit: "bar (3 m)", prefix: "BAR-", example: "BAR-2040-CLR" },
  { id: "plate", unit: "plate", prefix: "PL-", example: "PL-400" },
```

```
  { id: "fastener", unit: "pack of 50", prefix: "", example: "M8-BOLT" },
  { id: "motor", unit: "piece", prefix: "SRV-", example: "SRV-750" },
  { id: "actuator", unit: "piece", prefix: "LA-", example: "LA-500" },
  { id: "controller", unit: "piece", prefix: "MC-", example: "MC-2" },
  { id: "gripper", unit: "piece", prefix: "GR-", example: "GR-2" },
];
const UNITS = ["piece", "plate", "bar (3 m)", "pack of 50", "pack of 100", "metre"];
const SKU = /^[A-Z0-9][A-Z0-9-]{1,31}$/;

interface Props {
  existing: Part[];
  onCreated: (part: Part) => void;
  onCancel: () => void;
}

/** Form for a new part. Checks what it can before sending; the API checks everything again. */
export function AddPartPanel({ existing, onCreated, onCancel }: Props) {
  const [category, setCategory] = useState<Category>("plate");
  const [sku, setSku] = useState("");
  const [name, setName] = useState("");
  const [unit, setUnit] = useState("plate");
  const [unitTouched, setUnitTouched] = useState(false);
  const [onHand, setOnHand] = useState("0");
  const [reorderLevel, setReorderLevel] = useState("0");
  const [location, setLocation] = useState("");
  const [busy, setBusy] = useState(false);
  const [problems, setProblems] = useState<string[]>([]);

  const meta = CATEGORIES.find((c) => c.id === category)!;
  const skuTaken = existing.some((p) => p.sku === sku);
  const skuBad = sku !== "" && !SKU.test(sku);

  function pickCategory(c: Category) {
    setCategory(c);
    // Suggest the usual unit for the category, unless the user already chose one.
    if (!unitTouched) setUnit(CATEGORIES.find((x) => x.id === c)!.unit);
  }
```

Each category carries its usual unit, SKU prefix and an example. Picking a category fills in the unit, **unless the user has already typed one** (`unitTouched`): helpful defaults shouldn't overwrite deliberate choices.

`skuTaken` and `skuBad` are computed on every render from state, not stored, so they can never be out of date. They give instant feedback while typing; the server still checks everything, because another user may add the same SKU between the check and the click.

web/src/AddPartPanel.tsx · lines 45–61

```
async function submit(e: FormEvent) {
  e.preventDefault();
  setBusy(true);
  setProblems([]);
  try {
    const part = await api.createPart({
      sku, name, category, unit, location,
      onHand: Number(onHand), reorderLevel: Number(reorderLevel),
    });
    onCreated(part);
  } catch (err) {
    const body = (err as { body?: { problems?: string[] } }).body;
    setProblems(body?.problems ?? [(err as Error).message]);
  } finally {
    setBusy(false);
  }
}
```

On failure, the API's list of problems is shown as it is. On success, the parent decides what happens next.

web/src/AddPartPanel.tsx · lines 84–92

```
<label className="full">
  SKU
  <input value={sku} required maxLength={32} placeholder={`e.g. ${meta.example}`} autoFocus
    onChange={(e) => setSku(e.target.value.toUpperCase().replace(/\s+/g, "-"))}
    aria-invalid={skuTaken || skuBad} className="mono" />
  {skuTaken && <span className="field-error">{sku} already exists</span>}
  {skuBad && <span className="field-error">Capital letters, digits and dashes only, 2–32
characters</span>}
  {!sku && meta.prefix && <span className="hint">Parts like this usually start with
{meta.prefix}</span>}
</label>
```

The SKU input upper-cases as you type and turns spaces into dashes. `aria-invalid` marks the field for screen readers and, through CSS, outlines it in red: one attribute for both, as with the panel toggles in Snack 1.

Showing the new part

web/src/StockView.tsx · lines 73–87

```
{adding && (
  <AddPartPanel
    existing={parts}
    onCancel={() => setAdding(false)}
    onCreated={(p) => {
      setAdding(false);
      setSearch("");
      setCategory("all");
      setLowOnly(false);
      setSelected(p.sku);
      setJustAdded(p.sku);
      onChanged();
    }}
  />
)}
```

After creating, the stock view clears every filter (a new part hidden by an old search would look like a failure), selects the part so its panel opens with the “Initial stock” receipt, marks the row so CSS can flash it green, and calls `onChanged`, which refetches the parts and bumps the version so every other view updates too.

PART 5.12 · `api/src/domain/shapes.ts`

Shapes for the robot arm's scene

Chapter 1's arm app places shapes on its floor, and some of them stand for inventory parts. They're stored here, beside the stock, so every replica and every browser sees the same scene. This part follows them through the API.

The model

api/src/domain/shapes.ts · lines 1–52

```
/**
 * Simple 3D shapes placed in the robot arm's scene (Snack 1). They're stored with the
 * inventory, so any API replica, and any browser, sees the same scene.
 *
 * Units are meters and degrees. The floor is y = 0; a shape always rests on it, so only
 * x and z are stored and its height decides where its center is.
 */
import { isSku } from "./design.ts";
import { ValidationError } from "./errors.ts";

export type ShapeKind = "box" | "cylinder" | "sphere" | "cone";
export const SHAPE_KINDS: readonly ShapeKind[] = ["box", "cylinder", "sphere", "cone"];

/**
 * size.x, size.y, size.z are the bounding box: width, height, depth.
 * A cylinder or cone is round, so its depth equals its width (the diameter);
 * a sphere's three sizes are all its diameter. parseNewShape enforces both.
 */
export interface Size {
  x: number;
  y: number;
  z: number;
}

export interface Shape {
  id: string;
  name: string;
  kind: ShapeKind;
  size: Size;
  /** "#rrggbb" */
  color: string;
  /** Floor position of the shape's center. */
  position: { x: number; z: number };
  /** Turn about the vertical axis, in degrees. */
  rotationY: number;
  /** The inventory part this shape stands for, or null for a plain shape. */
  sku: string | null;
  createdAt: string;
```

```
  updatedAt: string;
}

export type NewShape = Omit<Shape, "id" | "createdAt" | "updatedAt">;
/** What can change after a shape is placed: move it, turn it, rename or recolor it. */
export type ShapePatch = Partial<Pick<Shape, "name" | "color" | "position" | "rotationY">>;

export const LIMITS = {
  minSize: 0.01, // 1 cm
  maxSize: 2, // 2 m
  floor: 3, // positions within ±3 m of the arm's base
};

const COLOR = /^#[0-9a-f]{6}$/;
```

Meters and degrees, as in the arm app. `size` is always the bounding box; for round shapes the depth follows the diameter. `sku` links a shape to an inventory part, or is null for a plain shape. `ShapePatch` lists what may change after placing: position, turn, name and color. Kind and size can't, because the arm app treats a different size as a different object: delete and re-create instead.

Validation, including what can't change

`api/src/domain/shapes.ts · lines 102–157`

```
export function parseNewShape(input: unknown): NewShape {
  const b = asBody(input);
  const problems: string[] = [];

  const kind = b.kind as ShapeKind;
  if (!SHAPE_KINDS.includes(kind)) problems.push(`kind must be one of ${SHAPE_KINDS.join(",
")}`);

  const s = asBody(b.size);
  const dim = (v: unknown) => num(v, LIMITS.minSize, LIMITS.maxSize);
  let size: Size | undefined;
  const range = `from ${LIMITS.minSize} to ${LIMITS.maxSize} m`;
  if (kind === "sphere") {
    const d = dim(s.x);
    if (d === null) problems.push(`size.x (diameter) must be ${range}`);
    else size = { x: d, y: d, z: d };
  } else if (kind === "cylinder" || kind === "cone") {
    const d = dim(s.x);
```

```
    const h = dim(s.y);
    if (d === null) problems.push(`size.x (diameter) must be ${range}`);
    if (h === null) problems.push(`size.y (height) must be ${range}`);
    if (d !== null && h !== null) size = { x: d, y: h, z: d };
  } else if (kind === "box") {
    const [x, y, z] = [dim(s.x), dim(s.y), dim(s.z)];
    if (x === null || y === null || z === null) problems.push(`size.x, size.y and size.z
(width, height, depth) must each be ${range}`);
    else size = { x, y, z };
  }

  const name = readName(b.name, problems);
  const color = readColor(b.color, problems);
  const position = readPosition(b.position, problems);
  const rotationY = b.rotationY === undefined ? 0 : readRotation(b.rotationY, problems);
  let sku: string | null = null;
  if (b.sku !== undefined && b.sku !== null) {
    if (isSku(b.sku)) sku = b.sku;
    else problems.push('sku must be an inventory SKU like "PL-200", or left out for a plain
shape');
  }

  if (problems.length) throw new ValidationError(problems);
  return { name: name!, kind, size: size!, color: color!, position: position!, rotationY:
rotationY!, sku };
}

/** Validates a change to a placed shape. At least one field must be given. */
export function parseShapePatch(input: unknown): ShapePatch {
  const b = asBody(input);
  const problems: string[] = [];
  const patch: ShapePatch = {};
  if (b.name !== undefined) patch.name = readName(b.name, problems);
  if (b.color !== undefined) patch.color = readColor(b.color, problems);
  if (b.position !== undefined) patch.position = readPosition(b.position, problems);
  if (b.rotationY !== undefined) patch.rotationY = readRotation(b.rotationY, problems);
  const unknown = Object.keys(b).filter((k) => !["name", "color", "position",
"rotationY"].includes(k));
  if (unknown.length) problems.push(`can't change ${unknown.join(", ")}; delete and re-create
the shape instead`);
  if (!problems.length && Object.keys(patch).length === 0) problems.push("nothing to change:
```

```
send name, color, position or rotationY");
  if (problems.length) throw new ValidationError(problems);
  return patch;
}
```

Round shapes are normalized here: send a cylinder with any depth and it's stored with depth = diameter. So the rule lives in one place, and the database never holds a cylinder that's wider one way than the other.

`parseShapePatch` rejects unknown fields by name (“can't change kind”) instead of silently ignoring them, and an empty patch is an error. Silent ignoring is how clients end up believing a change was saved.

Postgres: a table with its rules

`api/src/db/schema.sql · lines 48–65`

```
-- Shapes placed in the robot arm's scene (Snack 1). Sizes and positions in meters.
CREATE TABLE IF NOT EXISTS shapes (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL CHECK (length(name) BETWEEN 1 AND 40),
  kind        text NOT NULL CHECK (kind IN ('box', 'cylinder', 'sphere', 'cone')),
  size_x      double precision NOT NULL CHECK (size_x BETWEEN 0.01 AND 2),
  size_y      double precision NOT NULL CHECK (size_y BETWEEN 0.01 AND 2),
  size_z      double precision NOT NULL CHECK (size_z BETWEEN 0.01 AND 2),
  color       text NOT NULL CHECK (color ~ '^#[0-9a-f]{6}$'),
  x           double precision NOT NULL CHECK (x BETWEEN -3 AND 3),
  z           double precision NOT NULL CHECK (z BETWEEN -3 AND 3),
  rotation_y  double precision NOT NULL DEFAULT 0,
  created_at  timestamptz NOT NULL DEFAULT clock_timestamp(),
  updated_at  timestamptz NOT NULL DEFAULT clock_timestamp()
);
-- A shape can stand for an inventory part. Added separately so databases created before it
gain the column.
ALTER TABLE shapes ADD COLUMN IF NOT EXISTS sku text REFERENCES parts (sku);
CREATE INDEX IF NOT EXISTS shapes_by_sku ON shapes (sku);
```

The table repeats the limits as `CHECK` constraints, including a regular expression for the color, so even a direct SQL insert can't store a 5 m box or the color “red”. The `sku` column is added with `ALTER TABLE … ADD COLUMN IF NOT EXISTS`, after the table: a database created before parts could be placed gains the column on the next start. That's the smallest possible migration, and why real projects move to a migrations tool once the schema starts changing.

The foreign key means a shape can only stand for a part that exists. The API checks first to give a friendly 400; the key is the guarantee.

api/src/db/postgres-sql.ts · lines 63–77

```
listShapes: `
  SELECT id, name, kind, size_x, size_y, size_z, color, x, z, rotation_y, sku, created_at,
updated_at
    FROM shapes ORDER BY created_at, id`,
createShape: `
  INSERT INTO shapes (name, kind, size_x, size_y, size_z, color, x, z, rotation_y, sku)
  VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10)
  RETURNING id, name, kind, size_x, size_y, size_z, color, x, z, rotation_y, sku, created_at,
updated_at`,
updateShape: `
  UPDATE shapes
     SET name = coalesce($2, name), color = coalesce($3, color),
         x = coalesce($4, x), z = coalesce($5, z), rotation_y = coalesce($6, rotation_y),
         updated_at = clock_timestamp()
   WHERE id = $1
  RETURNING id, name, kind, size_x, size_y, size_z, color, x, z, rotation_y, sku, created_at,
updated_at`,
deleteShape: `DELETE FROM shapes WHERE id = $1 RETURNING id`,
```

The update uses `coalesce($n, column)`: a NULL parameter means “keep what's there”. One statement handles every combination of fields a patch might carry, instead of building SQL text for each, which is where injection bugs come from.

api/src/db/postgres.ts · lines 156–182

```
async listShapes(): Promise<Shape[]> {
  const { rows } = await this.pool.query<ShapeRow>(Q.listShapes);
  return rows.map(toShape);
}

async createShape(s: NewShape): Promise<Shape> {
  const { rows } = await this.pool.query<ShapeRow>(Q.createShape, [
    s.name, s.kind, s.size.x, s.size.y, s.size.z, s.color, s.position.x, s.position.z,
s.rotationY, s.sku,
  ]);
  return toShape(rows[0]);
}
```

```
async updateShape(id: string, p: ShapePatch): Promise<Shape> {
  if (!UUID.test(id)) throw new NotFoundError(`No shape ${id}`);
  // NULL means "keep the current value" (coalesce in the SQL).
  const { rows } = await this.pool.query<ShapeRow>(Q.updateShape, [
    id, p.name ?? null, p.color ?? null, p.position?.x ?? null, p.position?.z ?? null,
p.rotationY ?? null,
  ]);
  if (!rows[0]) throw new NotFoundError(`No shape ${id}`);
  return toShape(rows[0]);
}

async deleteShape(id: string): Promise<void> {
  if (!UUID.test(id)) throw new NotFoundError(`No shape ${id}`);
  const { rowCount } = await this.pool.query(Q.deleteShape, [id]);
  if (!rowCount) throw new NotFoundError(`No shape ${id}`);
}
```

MongoDB: nested data as it is

api/src/db/mongo.ts · lines 42–43

```
/** A shape is stored almost exactly as the API returns it: documents fit nested data like size
and position. */
interface ShapeDoc extends Omit<Shape, "id" | "createdAt" | "updatedAt"> { _id: string;
createdAt: Date; updatedAt: Date }
```

api/src/db/mongo.ts · lines 219–243

```
  return (await this.shapes.find({}).sort({ createdAt: 1, _id: 1 }).toArray()).map(toShape);
}

async createShape(s: NewShape): Promise<Shape> {
  const now = new Date();
  const doc: ShapeDoc = { ...s, _id: randomUUID(), createdAt: now, updatedAt: now };
  await this.shapes.insertOne(doc);
  return toShape(doc);
}

async updateShape(id: string, patch: ShapePatch): Promise<Shape> {
  const doc = await this.shapes.findOneAndUpdate(
    { _id: id },
    { $set: { ...patch, updatedAt: new Date() } },
```

```
    { returnDocument: "after" },
  );
  if (!doc) throw new NotFoundError(`No shape ${id}`);
  return toShape(doc);
}

async deleteShape(id: string): Promise<void> {
  const r = await this.shapes.deleteOne({ _id: id });
  if (r.deletedCount !== 1) throw new NotFoundError(`No shape ${id}`);
}
```

Here the document model shines. A shape has nested `size` and `position` objects, and MongoDB stores them as they are: the document is the API response with `_id` for `id`. Postgres needs five columns and a mapping function for the same data. `$set` with the patch updates exactly the fields it carries.

Shapes saved before the `sku` field existed simply don't have it; `toShape` reads a missing `sku` as null. That's MongoDB's version of the `ALTER TABLE` above: old documents keep their old shape, and the reading code copes.

Routes, and letting another page call the API

`api/src/app.ts · lines 75–88`

```
// ---- Shapes in the robot arm's scene (Snack 1) ----
route("GET", "/api/shapes", async () => [200, await repo.listShapes()]);
route("POST", "/api/shapes", async ({ body }) => {
  const shape = parseNewShape(body);
  // A shape that stands for a part must name one that exists.
  if (shape.sku && (await repo.getParts([shape.sku])).length === 0) throw new
ValidationError([`No part ${shape.sku} in the inventory`]);
  return [201, await repo.createShape(shape)];
});
route("PATCH", "/api/shapes/:id", async ({ params, body }) => [200, await
repo.updateShape(params.id, parseShapePatch(body))]);
route("DELETE", "/api/shapes/:id", async ({ params }) => {
  await repo.deleteShape(params.id);
  return [200, { deleted: params.id }];
});
```

Four routes: list, create, patch, delete. Creating a shape for a part checks that the part exists first.

The arm app runs on its own dev server, `http://localhost:5173`, a different **origin** from the API. Browsers block a page from reading responses from another origin unless the server says it's allowed: that's **CORS**. For anything but a simple GET, the browser first sends an `OPTIONS` **preflight** asking “may I send a PATCH with a JSON body?”. The handler answers it before routing: 204 with the allowed methods and headers, or 403.

api/src/app.ts · lines 215–233

```
/**
 * Which other web pages may call this API from a browser. By default, any page served from
 * this machine (http://localhost:<any port>), which covers the robot arm's Vite dev server.
 * CORS_ORIGINS="https://a.example,https://b.example" replaces that list.
 */
function corsHeaders(origin: string | undefined): Record<string, string> | null {
  if (!origin) return null;
  const allowed = process.env.CORS_ORIGINS
    ? process.env.CORS_ORIGINS.split(",").map((o) => o.trim()).includes(origin)
    : /^http:\/\/(localhost|127\.0\.0\.1)(:\d+)?$/.test(origin);
  if (!allowed) return null;
  return {
    "access-control-allow-origin": origin,
    "access-control-allow-methods": "GET, POST, PATCH, DELETE",
    "access-control-allow-headers": "content-type",
    "access-control-max-age": "600",
    vary: "origin",
  };
}
```

By default only pages served from this machine are allowed, matched with an anchored regular expression so `http://localhost.evil.example` doesn't sneak through. `Vary: Origin` tells caches the answer depends on who's asking. In production, `CORS_ORIGINS` lists the exact sites instead. CORS protects users' browsers, not the API: a script outside a browser ignores it, which is why it's no substitute for authentication.

Tested on every database

api/tests/contract.ts · lines 192–212

```
test("shapes: create, list oldest first, move, rename, delete", async () => {
  const box = await repo.createShape({ name: "Crate", kind: "box", size: { x: 0.3, y: 0.2, z:
0.25 }, color: "#d97706", position: { x: 0.5, z: -0.2 }, rotationY: 15, sku: null });
  const can = await repo.createShape({ name: "Actuator", kind: "box", size: { x: 0.4, y: 0.06,
z: 0.06 }, color: "#2563eb", position: { x: -0.4, z: 0.3 }, rotationY: 0, sku: "LA-300" });
  assert.equal(box.sku, null);
  assert.equal(can.sku, "LA-300");
  assert.equal((await repo.listShapes())[1].sku, "LA-300");
  assert.match(box.id, /^[0-9a-f-]{36}$/);
  assert.deepEqual(box.size, { x: 0.3, y: 0.2, z: 0.25 });
  assert.deepEqual((await repo.listShapes()).map((s) => s.name), ["Crate", "Actuator"]);

  const moved = await repo.updateShape(box.id, { position: { x: 0.6, z: 0.1 }, rotationY: 90
});
  assert.deepEqual([moved.position, moved.rotationY, moved.name, moved.color], [{ x: 0.6, z:
0.1 }, 90, "Crate", "#d97706"]);
  const renamed = await repo.updateShape(box.id, { name: "Tote", color: "#15803d" });
  assert.deepEqual([renamed.name, renamed.color, renamed.position], ["Tote", "#15803d", { x:
0.6, z: 0.1 }]);
  assert.ok(Date.parse(renamed.updatedAt) >= Date.parse(renamed.createdAt));

  await repo.deleteShape(can.id);
  assert.deepEqual((await repo.listShapes()).map((s) => s.id), [box.id]);
  await assert.rejects(repo.deleteShape(can.id), NotFoundError);
  await assert.rejects(repo.updateShape("00000000-0000-4000-8000-000000000000", { rotationY: 1
}), NotFoundError);
});
```

One more contract test, so the three back ends agree on shapes too: creation with and without a SKU, oldest-first order, partial updates that leave other fields alone, and not-found on a second delete. The API tests add validation, the unknown-SKU check, and CORS: a localhost page gets the headers, `https://evil.example` gets a 403.

PART 5.13 · `web/src/api.ts`

The React page

Four views over the same data: stock, design check, reservations and the ledger.

A typed client

web/src/api.ts · lines 23–60

```
export class ApiError extends Error {
  readonly status: number;
  readonly body: Record<string, unknown>;
  constructor(status: number, message: string, body: Record<string, unknown>) {
    super(message);
    this.status = status;
    this.body = body;
  }
}

async function call<T>(method: string, path: string, body?: unknown): Promise<T> {
  const res = await fetch(`/api${path}`, {
    method,
    headers: body === undefined ? undefined : { "content-type": "application/json" },
    body: body === undefined ? undefined : JSON.stringify(body),
  });
  const data = (await res.json().catch(() => ({}))) as Record<string, unknown>;
  if (!res.ok) {
    const problems = Array.isArray(data.problems) ? `: ${(data.problems as string[]).join(";
")}` : "";
    throw new ApiError(res.status, `${String(data.error ?? res.statusText)}${problems}`, data);
  }
  return data as T;
}

export const api = {
  info: () => call<Info>("GET", "/info"),
  parts: () => call<Part[]>("GET", "/parts"),
  movements: (sku?: string, limit = 50) => call<Movement[]>("GET", `/movements?
limit=${limit}${sku ? `&sku=${encodeURIComponent(sku)}` : ""}`),
  createPart: (part: Omit<Part, "reserved" | "available" | "low" | "updatedAt">) => call<Part>
("POST", "/parts", part),
  receive: (sku: string, qty: number, note: string) => call<Part>("POST",
`/parts/${encodeURIComponent(sku)}/receive`, { qty, note }),
  consume: (sku: string, qty: number, note: string) => call<Part>("POST",
`/parts/${encodeURIComponent(sku)}/consume`, { qty, note }),
  check: (design: unknown) => call<CheckResult>("POST", "/check", { design }),
  reserve: (quoteRef: string, design: unknown) =>
```

```
    call<{ ok: true; reservation: Reservation; warnings: string[] }>("POST", "/reservations", {
quoteRef, design }),
  reservations: () => call<Reservation[]>("GET", "/reservations"),
  release: (id: string) => call<Reservation>("POST", `/reservations/${id}/release`),
  fulfil: (id: string) => call<Reservation>("POST", `/reservations/${id}/fulfil`),
};
```

One `call` function handles every request: JSON in, JSON out, and any non-2xx response becomes an `ApiError` that keeps the status and body. The design view needs the body of a 409 to show the shortages, which is why it isn't thrown away.

App state and refreshing

web/src/App.tsx · lines 18–63

```
export function App() {
  const [tab, setTab] = useState<Tab>("stock");
  const [info, setInfo] = useState<Info | null>(null);
  const [parts, setParts] = useState<Part[]>([]);
  const [reservations, setReservations] = useState<Reservation[]>([]);
  const [version, setVersion] = useState(0); // bumps after every change, so views refetch
  const [error, setError] = useState<string | null>(null);

  const refresh = useCallback(async () => {
    try {
      const [p, r, i] = await Promise.all([api.parts(), api.reservations(), api.info()]);
      setParts(p);
      setReservations(r);
      setInfo(i);
      setError(null);
    } catch (e) {
      setError(`Can't reach the API: ${(e as Error).message}`);
    }
  }, []);

  const changed = useCallback(() => {
    setVersion((v) => v + 1);
    void refresh();
  }, [refresh]);

  // Load now, then every 10 s and whenever the tab gets focus, so changes made elsewhere show
up.
```

```
  useEffect(() => {
    void refresh();
    const timer = setInterval(() => void refresh(), 10_000);
    const onFocus = () => void refresh();
    window.addEventListener("focus", onFocus);
    return () => {
      clearInterval(timer);
      window.removeEventListener("focus", onFocus);
    };
  }, [refresh]);

  const kpis = useMemo(() => {
    const active = reservations.filter((r) => r.status === "active");
    return [
      { label: "SKUs", value: parts.length },
      { label: "Low stock", value: parts.filter((p) => p.low).length, tone: parts.some((p) =>
p.low) ? "warn" : undefined },
      { label: "Units reserved", value: parts.reduce((n, p) => n + p.reserved, 0) },
      { label: "Active reservations", value: active.length },
    ];
  }, [parts, reservations]);
```

The app holds parts and reservations, and a `version` number that increases after every change. Views that load their own data (a part's history, the ledger) list `version` in their effect dependencies, so any change anywhere refetches them.

It also refreshes every 10 s and when the window regains focus. With two API replicas and possibly two users, the page can't assume it made the last change; this is the simplest way to stay current. WebSockets or server-sent events would push changes instead.

The part panel

web/src/StockView.tsx · lines 99–131

```
function PartPanel({ part, version, onChanged, onClose }: { part: Part; version: number;
onChanged: () => void; onClose: () => void }) {
  const [history, setHistory] = useState<Movement[]>([]);
  const [qty, setQty] = useState("1");
  const [note, setNote] = useState("");
  const [busy, setBusy] = useState(false);
  const [message, setMessage] = useState<{ ok: boolean; text: string } | null>(null);

  useEffect(() => {
```

```
    let live = true;
    api.movements(part.sku, 12).then((m) => live && setHistory(m)).catch(() => {});
    return () => {
      live = false;
    };
  }, [part.sku, part.updatedAt, version]);

  async function submit(kind: "receive" | "consume", e?: FormEvent) {
    e?.preventDefault();
    setBusy(true);
    setMessage(null);
    try {
      const n = Number(qty);
      const p = kind === "receive" ? await api.receive(part.sku, n, note) : await
api.consume(part.sku, n, note);
      setMessage({ ok: true, text: `${kind === "receive" ? "Received" : "Consumed"} ${n}. On
hand is now ${p.onHand}.` });
      setNote("");
      onChanged();
    } catch (err) {
      setMessage({ ok: false, text: (err as Error).message });
    } finally {
      setBusy(false);
    }
  }

  const short = Math.max(0, part.reorderLevel + 1 - part.available);
```

The effect's `live` flag ignores a response that arrives after the panel closed or switched part: without it, a slow request for one part could overwrite another's history. One `submit` serves both buttons, and errors from the API (like “Only 4 of LA-300 available”) are shown as they are.

Checking and reserving a design

web/src/DesignView.tsx · lines 23–53

```
async function check() {
  setError(null);
  try {
    setResult(await api.check(parsed()));
  } catch (e) {
    setResult(null);
```

```
    setError((e as Error).message);
  }
}

// Re-check whenever stock changes (including from other tabs or users).
useEffect(() => {
  if (result) void check();
}, [version]);

async function reserve() {
  setBusy(true);
  setOutcome(null);
  try {
    const r = await api.reserve(quoteRef, parsed());
    setOutcome({ ok: true, text: `Reserved ${r.reservation.lines.length} lines for
${r.reservation.quoteRef}.` });
    setQuoteRef(`Q-${Date.now()}`);
    onChanged();
  } catch (e) {
    const shortages = e instanceof ApiError ? (e.body.shortages as Shortage[] | undefined) :
undefined;
    setOutcome({ ok: false, text: shortages ? "Nothing was reserved: the database found these
short." : (e as Error).message, shortages });
    onChanged();
  } finally {
    setBusy(false);
  }
}
```

Reserving catches the `ApiError` and pulls `shortages` out of its body, so a refusal reads as “the database found these short”, with each line. The view calls `onChanged` even after a refusal: the refusal proves the page's numbers were stale.

The stock bar

web/src/ui.tsx · lines 3–19

```
/**
 * On hand as a bar: the reserved share, the available share, and a tick at the reorder
 * level. Scaled to whichever is bigger, stock or twice the reorder level, so a part
 * near its reorder level always looks it.
 */
export function StockBar({ part, large = false }: { part: Part; large?: boolean }) {
  const scale = Math.max(part.onHand, part.reorderLevel * 2, 1);
  const pct = (n: number) => `${Math.min(100, (n / scale) * 100)}%`;
  return (
    <div className={`bar ${large ? "large" : ""} ${part.low ? "is-low" : ""}`} role="img"
      aria-label={`${part.available} available, ${part.reserved} reserved, reorder at
${part.reorderLevel}`}>
      <span className="reserved" style={{ width: pct(part.reserved) }} />
      <span className="available" style={{ width: pct(part.available) }} />
      <span className="reorder" style={{ left: pct(part.reorderLevel) }} />
    </div>
  );
}
```

Reserved and available stack in one bar, with a tick at the reorder level. Scaling to at least twice the reorder level means a part close to reordering always looks close, even if it has a big absolute number. The `aria-label` gives screen readers the same three numbers the bar shows.

PART 5.14 · `docker-compose.yml`

Containers: Compose profiles and Kubernetes overlays

One set of images, two databases. Compose chooses with profiles; Kubernetes with kustomize overlays.

One API definition, reused

docker-compose.yml · lines 1–19

```
# Runs the inventory app with one database or the other: http://localhost:8081
#
#   docker compose --profile postgres up --build     # PostgreSQL 17
#   docker compose --profile mongo up --build        # MongoDB 8
#
# The API and web images are the same either way; only the API's DB setting changes.
# Run the repository contract tests against BOTH databases (throwaway containers, no volumes):
#
#   docker compose --profile test run --rm --build api-test
#   docker compose --profile test down
name: inventory

x-api: &api
  build: ./api
  image: inventory-api:1.0
  networks:
    default:
      aliases: [api] # nginx forwards /api to "api", whichever database is behind it
```

`x-api` is a YAML **anchor** (`&api`), merged into both API services with `<<: *api`. The network alias `api` is the trick that makes switching invisible to the front end: nginx forwards to `api`, and whichever API service is running answers to that name.

Profiles

docker-compose.yml · lines 50–82

```
api-postgres:
  <<: *api
  profiles: [postgres]
  environment:
    DB: postgres
    PGHOST: postgres
    PGUSER: inventory
    PGPASSWORD: inventory-dev-password
    PGDATABASE: inventory
  depends_on:
    postgres:
      condition: service_healthy
```

```
api-mongo:
  <<: *api
  profiles: [mongo]
  environment:
    DB: mongo
    MONGO_HOST: mongo
    MONGO_USER: inventory
    MONGO_PASSWORD: inventory-dev-password
    MONGO_DB: inventory
  depends_on:
    mongo:
      condition: service_healthy

web:
  profiles: [postgres, mongo]
  build: ./web
  image: inventory-web:1.0
  ports:
    - "8081:80"
```

A service with `profiles` only starts when that profile is named: `--profile postgres` starts Postgres, its API and the web server; `--profile mongo` the Mongo pair and the same web server. `depends_on` with `service_healthy` waits for the database's healthcheck, not just for its container to exist.

Throwaway test databases

`docker-compose.yml · lines 83–122`

```
# ---- tests: fresh databases in memory (tmpfs), so your app data is never touched ----
postgres-test:
  profiles: [test]
  image: postgres:17-alpine
  environment:
    POSTGRES_USER: inventory
    POSTGRES_PASSWORD: test
    POSTGRES_DB: inventory_test
  tmpfs: [/var/lib/postgresql/data]
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U inventory -d inventory_test"]
    interval: 2s
```

```
    retries: 30

mongo-test:
  profiles: [test]
  image: mongo:8
  tmpfs: [/data/db]
  healthcheck:
    test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]
    interval: 2s
    retries: 30

api-test:
  profiles: [test]
  image: inventory-api:1.0
  build: ./api
  command: ["node", "--test", "tests/**/*.test.ts"]
  environment:
    LOG_REQUESTS: "0"
    PGHOST: postgres-test
    PGUSER: inventory
    PGPASSWORD: test
    PGDATABASE: inventory_test
    MONGO_URL: mongodb://mongo-test:27017
  depends_on:
    postgres-test:
      condition: service_healthy
    mongo-test:
      condition: service_healthy
```

The test profile starts separate databases on **tmpfs**, memory-backed storage that vanishes with the container. The contract tests wipe their database between tests; giving them their own instances means they can never wipe your demo data.

The API image

api/Dockerfile · lines 1–13

```
# Node 22.18+ runs TypeScript directly (type stripping), so there's no build stage:
# the image holds the production dependencies and the .ts sources.
FROM node:22-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev && npm cache clean --force
COPY src ./src
COPY tests ./tests
# Don't run as root.
USER node
EXPOSE 8080
CMD ["node", "src/server.ts"]
```

Node 22.18+ strips TypeScript types itself, so there's no compile step and no build stage: install production dependencies, copy the sources, and run `node src/server.ts` as the unprivileged `node` user.

Kubernetes: base plus overlay

k8s/postgres/kustomization.yaml · lines 1–24

```
# kubectl apply -k k8s/postgres
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: inventory
resources:
  - ../base
  - postgres.yaml
configMapGenerator:
  - name: api-config
    literals:
      - DB=postgres
      - PGHOST=postgres
      - PGDATABASE=inventory
secretGenerator:
  # Demo credentials. In a real cluster these come from a secret manager, not the repo.
  - name: api-db-credentials
    literals:
      - PGUSER=inventory
      - PGPASSWORD=inventory-dev-password
  - name: postgres-credentials
    literals:
      - POSTGRES_USER=inventory
      - POSTGRES_PASSWORD=inventory-dev-password
      - POSTGRES_DB=inventory
```

The base (namespace, API, web) is the same for both databases. Each overlay adds its database's StatefulSet and generates the API's ConfigMap and Secret. The API Deployment reads them with `envFrom`, so the overlay alone decides which database the same image talks to.

Generated ConfigMaps get a hash suffix in their name, and kustomize rewrites the Deployment to match. Changing a setting therefore changes the Deployment, which triggers a rolling restart automatically.

PART 5.15 · `api/src/domain/conveyors.ts`

Conveyor belts in the database

The arm app's conveyor belts (Chapter 1) are stored here too, beside its shapes, so a line layout is saved with the parts it uses. The pattern is the shapes' pattern; this part shows what's the same and what's new.

The model and its limits

api/src/domain/conveyors.ts · lines 1–45

```
/**
 * Conveyor belts placed in the robot arm's scene (Snack 1), stored beside its shapes.
 *
 * A belt is a box on the floor: its top surface is at `height`, it runs along its own
 * length, and it carries what rests on it at `speed` meters per second. Positive speed
 * moves things toward the belt's +x end (before rotation); negative runs it backwards.
 */
import { ValidationError } from "./errors.ts";

export interface Conveyor {
  id: string;
  name: string;
  /** Meters along the direction of travel. */
  length: number;
  /** Meters across. */
  width: number;
  /** Height of the belt surface above the floor, meters. */
  height: number;
  /** Meters per second; negative runs it backwards. */
  speed: number;
  /** Floor position of the belt's center. */
  position: { x: number; z: number };
  /** Degrees about the vertical axis; 0 means travel along +x. */
  rotationY: number;
  createdAt: string;
  updatedAt: string;
}

export type NewConveyor = Omit<Conveyor, "id" | "createdAt" | "updatedAt">;
/** A placed belt can be moved, turned, renamed and have its speed changed. Its size is fixed.
```

```
*/
export type ConveyorPatch = Partial<Pick<Conveyor, "name" | "position" | "rotationY" |
"speed">>;

export const CONVEYOR_LIMITS = {
  length: [0.3, 4],
  width: [0.1, 1.2],
  height: [0.05, 1.2],
  speed: [-1, 1],
  floor: 3,
} as const;

type Body = Record<string, unknown>;
const asBody = (v: unknown): Body => (v && typeof v === "object" && !Array.isArray(v) ? (v as
Body) : {});
const num = (v: unknown, [min, max]: readonly [number, number]) =>
  typeof v === "number" && Number.isFinite(v) && v >= min && v <= max ? v : null;
```

Meters, meters per second and degrees, like the shapes. A positive `speed` runs toward the belt's own +x end, so turning a belt never needs its speed changed. The limits are the physical sanity checks: belts from 30 cm to 4 m, tops between 5 cm and 1.2 m, at most 1 m/s either way, and positions inside the ±3 m floor the arm app draws.

Validation with defaults

`api/src/domain/conveyors.ts · lines 46–105`

```
function field<T>(problems: string[], value: T | null, message: string): T | undefined {
  if (value === null) {
    problems.push(message);
    return undefined;
  }
  return value;
}

function name(v: unknown) {
  return typeof v === "string" && v.trim() !== "" && v.trim().length <= 40 ? v.trim() : null;
}

function position(v: unknown) {
  const p = asBody(v);
  const range = [-CONVEYOR_LIMITS.floor, CONVEYOR_LIMITS.floor] as const;
```

```
  const x = num(p.x, range);
  const z = num(p.z, range);
  return x === null || z === null ? null : { x, z };
}

const MESSAGES = {
  name: "name must be text up to 40 characters",
  length: `length must be ${CONVEYOR_LIMITS.length[0]} to ${CONVEYOR_LIMITS.length[1]} m`,
  width: `width must be ${CONVEYOR_LIMITS.width[0]} to ${CONVEYOR_LIMITS.width[1]} m`,
  height: `height must be ${CONVEYOR_LIMITS.height[0]} to ${CONVEYOR_LIMITS.height[1]} m`,
  speed: `speed must be ${CONVEYOR_LIMITS.speed[0]} to ${CONVEYOR_LIMITS.speed[1]} m/s`,
  position: `position must be { x, z } in meters, each from -${CONVEYOR_LIMITS.floor} to
${CONVEYOR_LIMITS.floor}`,
  rotationY: "rotationY must be degrees from -360 to 360",
};

export function parseNewConveyor(input: unknown): NewConveyor {
  const b = asBody(input);
  const problems: string[] = [];
  const out = {
    name: field(problems, name(b.name), MESSAGES.name),
    length: field(problems, num(b.length, CONVEYOR_LIMITS.length), MESSAGES.length),
    width: field(problems, num(b.width, CONVEYOR_LIMITS.width), MESSAGES.width),
    height: field(problems, num(b.height, CONVEYOR_LIMITS.height), MESSAGES.height),
    speed: field(problems, num(b.speed ?? 0.2, CONVEYOR_LIMITS.speed), MESSAGES.speed),
    position: field(problems, position(b.position), MESSAGES.position),
    rotationY: field(problems, num(b.rotationY ?? 0, [-360, 360]), MESSAGES.rotationY),
  };
  if (problems.length) throw new ValidationError(problems);
  return out as NewConveyor;
}

export function parseConveyorPatch(input: unknown): ConveyorPatch {
  const b = asBody(input);
  const problems: string[] = [];
  const patch: ConveyorPatch = {};
  if (b.name !== undefined) patch.name = field(problems, name(b.name), MESSAGES.name);
  if (b.position !== undefined) patch.position = field(problems, position(b.position),
MESSAGES.position);
  if (b.rotationY !== undefined) patch.rotationY = field(problems, num(b.rotationY, [-360,
360]), MESSAGES.rotationY);
```

```
  if (b.speed !== undefined) patch.speed = field(problems, num(b.speed, CONVEYOR_LIMITS.speed),
MESSAGES.speed);
  const unknown = Object.keys(b).filter((k) => !["name", "position", "rotationY",
"speed"].includes(k));
  if (unknown.length) problems.push(`can't change ${unknown.join(", ")}; delete and re-create
the belt instead`);
  if (!problems.length && Object.keys(patch).length === 0) problems.push("nothing to change:
send name, position, rotationY or speed");
  if (problems.length) throw new ValidationError(problems);
  return patch;
}
```

`parseNewConveyor` collects every problem rather than stopping at the first, so a form can show them all together; the API test sends four mistakes and gets four messages. `speed` and `rotationY` are optional with defaults, 0.2 m/s and 0°: the smallest useful request is a name, a size and a position.

`parseConveyorPatch` allows name, position, angle and speed, and refuses size by name, for the same reason shapes refuse it: the arm app treats a belt of another size as another belt.

Postgres: one more table

`api/src/db/schema.sql · lines 66–80`

```
-- Conveyor belts in the robot arm's scene. Meters, meters per second, degrees.
CREATE TABLE IF NOT EXISTS conveyors (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL CHECK (length(name) BETWEEN 1 AND 40),
  length      double precision NOT NULL CHECK (length BETWEEN 0.3 AND 4),
  width       double precision NOT NULL CHECK (width BETWEEN 0.1 AND 1.2),
  height      double precision NOT NULL CHECK (height BETWEEN 0.05 AND 1.2),
  speed       double precision NOT NULL DEFAULT 0.2 CHECK (speed BETWEEN -1 AND 1),
  x           double precision NOT NULL CHECK (x BETWEEN -3 AND 3),
  z           double precision NOT NULL CHECK (z BETWEEN -3 AND 3),
  rotation_y  double precision NOT NULL DEFAULT 0,
  created_at  timestamptz NOT NULL DEFAULT clock_timestamp(),
  updated_at  timestamptz NOT NULL DEFAULT clock_timestamp()
);
```

The limits again as `CHECK` constraints. `DEFAULT 0.2` on `speed` mirrors the API default, so a row inserted by hand behaves the same. `created_at` uses `clock_timestamp()` rather than `now()`: `now()` is the start of the transaction, so two belts inserted in one transaction would tie and their order would depend on their random UUIDs.

api/src/db/postgres-sql.ts · lines 78–92

```
listConveyors: `
  SELECT id, name, length, width, height, speed, x, z, rotation_y, created_at, updated_at
    FROM conveyors ORDER BY created_at, id`,
createConveyor: `
  INSERT INTO conveyors (name, length, width, height, speed, x, z, rotation_y)
  VALUES ($1, $2, $3, $4, $5, $6, $7, $8)
  RETURNING id, name, length, width, height, speed, x, z, rotation_y, created_at, updated_at`,
updateConveyor: `
  UPDATE conveyors
     SET name = coalesce($2, name), x = coalesce($3, x), z = coalesce($4, z),
         rotation_y = coalesce($5, rotation_y), speed = coalesce($6, speed),
         updated_at = clock_timestamp()
   WHERE id = $1
  RETURNING id, name, length, width, height, speed, x, z, rotation_y, created_at, updated_at`,
deleteConveyor: `DELETE FROM conveyors WHERE id = $1 RETURNING id`,
```

The same `coalesce($n, column)` partial update as shapes. Because the API layer has already refused unknown fields, the SQL never needs to be assembled from user input: one fixed statement with six parameters covers every patch.

MongoDB: the document is the belt

api/src/db/mongo.ts · lines 44–45

```
/** Conveyors too: the nested position stays nested. */
interface ConveyorDoc extends Omit<Conveyor, "id" | "createdAt" | "updatedAt"> { _id: string;
createdAt: Date; updatedAt: Date }
```

api/src/db/mongo.ts · lines 244–268

```
async listConveyors(): Promise<Conveyor[]> {
  return (await this.conveyors.find({}).sort({ createdAt: 1, _id: 1
}).toArray()).map(toConveyor);
}

async createConveyor(c: NewConveyor): Promise<Conveyor> {
  const now = new Date();
  const doc: ConveyorDoc = { ...c, _id: randomUUID(), createdAt: now, updatedAt: now };
  await this.conveyors.insertOne(doc);
  return toConveyor(doc);
}
```

```
async updateConveyor(id: string, patch: ConveyorPatch): Promise<Conveyor> {
  const doc = await this.conveyors.findOneAndUpdate({ _id: id }, { $set: { ...patch, updatedAt:
new Date() } }, { returnDocument: "after" });
  if (!doc) throw new NotFoundError(`No conveyor ${id}`);
  return toConveyor(doc);
}

async deleteConveyor(id: string): Promise<void> {
  const r = await this.conveyors.deleteOne({ _id: id });
  if (r.deletedCount !== 1) throw new NotFoundError(`No conveyor ${id}`);
  // No foreign keys here: delete the belt's splitters ourselves. A splitter created for this
  // belt in the instant between the two deletes would be left pointing at nothing; the arm
  // app ignores splitters whose belt is gone, and Postgres's foreign key rules it out.
  await this.splitters.deleteMany({ conveyorId: id });
}
```

As with shapes, the document is the API object with `_id` for `id`, nested `position` and all. `findOneAndUpdate` with `returnDocument: "after"` changes and returns the belt in one round trip. In Postgres, `RETURNING` does the same job.

Routes and tests

api/src/app.ts · lines 89–96

```
// ---- Conveyor belts in the robot arm's scene ----
route("GET", "/api/conveyors", async () => [200, await repo.listConveyors()]);
route("POST", "/api/conveyors", async ({ body }) => [201, await
repo.createConveyor(parseNewConveyor(body))]);
route("PATCH", "/api/conveyors/:id", async ({ params, body }) => [200, await
repo.updateConveyor(params.id, parseConveyorPatch(body))]);
route("DELETE", "/api/conveyors/:id", async ({ params }) => {
  await repo.deleteConveyor(params.id);
  return [200, { deleted: params.id }];
});
```

api/tests/contract.ts · lines 214–225

```
test("conveyors: create, list, move, change speed, delete", async () => {
  const a = await repo.createConveyor({ name: "Infeed", length: 1.2, width: 0.3, height: 0.25,
speed: 0.2, position: { x: 0.6, z: -0.5 }, rotationY: 0 });
  const b = await repo.createConveyor({ name: "Outfeed", length: 0.8, width: 0.25, height:
0.15, speed: -0.1, position: { x: -0.4, z: 0.6 }, rotationY: 90 });
  assert.match(a.id, /^[0-9a-f-]{36}$/);
  assert.deepEqual((await repo.listConveyors()).map((c) => [c.name, c.speed]), [["Infeed",
0.2], ["Outfeed", -0.1]]);
  const moved = await repo.updateConveyor(a.id, { position: { x: 0.5, z: -0.6 }, speed: 0.35
});
  assert.deepEqual([moved.position, moved.speed, moved.length, moved.rotationY], [{ x: 0.5, z:
-0.6 }, 0.35, 1.2, 0]);
  await repo.deleteConveyor(b.id);
  assert.deepEqual((await repo.listConveyors()).map((c) => c.id), [a.id]);
  await assert.rejects(repo.deleteConveyor(b.id), NotFoundError);
  await assert.rejects(repo.updateConveyor("00000000-0000-4000-8000-000000000000", { speed: 0
}), NotFoundError);
});
```

The contract test runs on all three back ends: two belts in creation order, a partial update that changes position and speed and leaves length and angle alone, and not-found for a deleted belt and for a well-formed id that never existed. The Postgres SQL was also run against a real PostgreSQL 16, including the `CHECK` on length.

PART 5.16 · `api/src/domain/splitters.ts`

Splitters: sorting rules as data, and editable parts

The arm app's splitters (Chapter 1) send items left, right or straight on by rules. The rules are stored here, beside the belts, and the parts they test can now be edited, so the sorting can be changed by changing the database.

The model

api/src/domain/splitters.ts · lines 1–76

```
/**
 * Splitters: diverter gates on a conveyor belt that send each item left, right or straight on,
 * by rules you write. The rules are data, stored here, so they can be changed from the arm
app,
```

```
 * the API or the database directly, even while the line is running.
 *
 * A splitter has an ordered list of rules. Each rule is one or more conditions (all must hold)
 * and an exit. The first rule that matches an item decides its exit; if none matches, the
 * splitter's `otherwise` exit does.
 *
 * Conditions can test the shape itself (kind, color, size, name, SKU) or the inventory part it
 * stands for (category, location, stock). Part fields are read when the item reaches the gate,
 * so changing a part in the inventory changes where its items go.
 */
import { ValidationError } from "./errors.ts";

export const EXITS = ["left", "straight", "right"] as const;
export type Exit = (typeof EXITS)[number];

/** Every field a condition can test, and its type. Sizes are meters, like the rest of the API.
*/
export const FIELDS = {
  kind: "text",
  color: "text",
  name: "text",
  sku: "text",
  width: "number",
  height: "number",
  depth: "number",
  isPart: "boolean",
  "part.category": "text",
  "part.location": "text",
  "part.name": "text",
  "part.onHand": "number",
  "part.available": "number",
  "part.low": "boolean",
} as const;
export type Field = keyof typeof FIELDS;
type FieldType = (typeof FIELDS)[Field];

export const OPS: Record<FieldType, readonly Op[]> = {
  text: ["eq", "ne", "in", "contains"],
  number: ["eq", "ne", "lt", "lte", "gt", "gte"],
  boolean: ["eq"],
};
```

```
export type Op = "eq" | "ne" | "in" | "contains" | "lt" | "lte" | "gt" | "gte";

export interface Condition {
  field: Field;
  op: Op;
  /** Text, a number, true/false, or a list of texts for "in". */
  value: string | number | boolean | string[];
}

export interface Rule {
  conditions: Condition[];
  exit: Exit;
}

export interface Splitter {
  id: string;
  name: string;
  conveyorId: string;
  /** Meters from the start of the belt (its -x end before turning) to the gate. */
  along: number;
  rules: Rule[];
  /** Where an item goes when no rule matches. */
  otherwise: Exit;
  createdAt: string;
  updatedAt: string;
}

export type NewSplitter = Omit<Splitter, "id" | "createdAt" | "updatedAt">;
/** Everything but the belt can change: move a gate to another belt by re-creating it. */
export type SplitterPatch = Partial<Pick<Splitter, "name" | "along" | "rules" | "otherwise">>;

export const MAX_RULES = 20;
export const MAX_CONDITIONS = 5;
```

A splitter belongs to a belt (`conveyorId`) and sits `along` meters from the belt's start. Its rules are an ordered list; each rule is up to five conditions and an exit; `otherwise` catches the rest. The field and operator tables are the same as the arm app's, so a rule the API accepts is a rule the arm app can evaluate.

Validating a rule list

api/src/domain/splitters.ts · lines 78–180

```
type Body = Record<string, unknown>;
const asBody = (v: unknown): Body => (v && typeof v === "object" && !Array.isArray(v) ? (v as
Body) : {});

/** Checks one condition, returning a normalized copy or a problem. Texts are compared ignoring
case. */
function parseCondition(c: unknown, where: string, problems: string[]): Condition | null {
  const b = asBody(c);
  const field = b.field as Field;
  if (!(typeof field === "string" && field in FIELDS)) {
    problems.push(`${where}: field must be one of ${Object.keys(FIELDS).join(", ")}`);
    return null;
  }
  const type = FIELDS[field];
  const op = b.op as Op;
  if (!OPS[type].includes(op)) {
    problems.push(`${where}: ${field} can be compared with ${OPS[type].join(", ")}`);
    return null;
  }
  const v = b.value;
  if (type === "boolean") {
    if (typeof v !== "boolean") return problems.push(`${where}: value must be true or false`),
null;
    return { field, op, value: v };
  }
  if (type === "number") {
    if (typeof v !== "number" || !Number.isFinite(v)) return problems.push(`${where}: value
must be a number`), null;
    return { field, op, value: v };
  }
  if (op === "in") {
    const list = Array.isArray(v) ? v : null;
    if (!list || list.length === 0 || list.length > 20 || !list.every((s) => typeof s ===
"string" && s.trim() !== "" && s.length <= 80)) {
      return problems.push(`${where}: value must be a list of 1 to 20 texts`), null;
    }
    return { field, op, value: (list as string[]).map((s) => s.trim().toLowerCase()) };
```

```
  }
  if (typeof v !== "string" || v.trim() === "" || v.length > 80) return
problems.push(`${where}: value must be text up to 80 characters`), null;
  return { field, op, value: v.trim().toLowerCase() };
}

function parseRules(v: unknown, problems: string[]): Rule[] | undefined {
  if (!Array.isArray(v) || v.length > MAX_RULES) {
    problems.push(`rules must be a list of at most ${MAX_RULES} rules`);
    return undefined;
  }
  const rules: Rule[] = [];
  v.forEach((r, i) => {
    const b = asBody(r);
    const where = `rule ${i + 1}`;
    if (!EXITS.includes(b.exit as Exit)) problems.push(`${where}: exit must be left, straight
or right`);
    const cs = b.conditions;
    if (!Array.isArray(cs) || cs.length === 0 || cs.length > MAX_CONDITIONS) {
      problems.push(`${where}: conditions must be a list of 1 to ${MAX_CONDITIONS}`);
      return;
    }
    const conditions = cs.map((c, j) => parseCondition(c, `${where}, condition ${j + 1}`,
problems));
    if (conditions.every((c) => c !== null) && EXITS.includes(b.exit as Exit)) rules.push({
conditions: conditions as Condition[], exit: b.exit as Exit });
  });
  return rules;
}

const name = (v: unknown) => (typeof v === "string" && v.trim() !== "" && v.trim().length <= 40
? v.trim() : null);
const along = (v: unknown) => (typeof v === "number" && Number.isFinite(v) && v >= 0 && v <= 4
? v : null);

/** Shape checks only; that `along` fits on the belt is checked against the belt in app.ts. */
export function parseNewSplitter(input: unknown): NewSplitter {
  const b = asBody(input);
  const problems: string[] = [];
  const n = name(b.name);
  if (n === null) problems.push("name must be text up to 40 characters");
```

```
  if (typeof b.conveyorId !== "string" || !UUID.test(b.conveyorId)) problems.push("conveyorId
must be a conveyor's id");
  const a = along(b.along);
  if (a === null) problems.push("along must be meters from the start of the belt, 0 to 4");
  const otherwise = (b.otherwise ?? "straight") as Exit;
  if (!EXITS.includes(otherwise)) problems.push("otherwise must be left, straight or right");
  const rules = parseRules(b.rules ?? [], problems);
  if (problems.length) throw new ValidationError(problems);
  return { name: n!, conveyorId: b.conveyorId as string, along: a!, rules: rules!, otherwise };
}

export function parseSplitterPatch(input: unknown): SplitterPatch {
  const b = asBody(input);
  const problems: string[] = [];
  const patch: SplitterPatch = {};
  if (b.name !== undefined) {
    const n = name(b.name);
    if (n === null) problems.push("name must be text up to 40 characters");
    else patch.name = n;
  }
  if (b.along !== undefined) {
    const a = along(b.along);
    if (a === null) problems.push("along must be meters from the start of the belt, 0 to 4");
    else patch.along = a;
  }
  if (b.otherwise !== undefined) {
    if (!EXITS.includes(b.otherwise as Exit)) problems.push("otherwise must be left, straight
or right");
    else patch.otherwise = b.otherwise as Exit;
  }
  if (b.rules !== undefined) patch.rules = parseRules(b.rules, problems);
  const unknown = Object.keys(b).filter((k) => !["name", "along", "rules",
"otherwise"].includes(k));
  if (unknown.length) problems.push(`can't change ${unknown.join(", ")}; delete and re-create
the splitter instead`);
  if (!problems.length && Object.keys(patch).length === 0) problems.push("nothing to change:
send name, along, rules or otherwise");
  if (problems.length) throw new ValidationError(problems);
  return patch;
}
```

Validation walks every rule and condition and reports each problem with its position, “rule 2, condition 1: kind can be compared with eq, ne, in, contains”, so whoever wrote the rule (a person in the panel, or a script against the API) can find it. The operator must suit the field's type, and the value must suit the operator: a number, true/false, text, or a list of texts for `in`. Text values are trimmed and lowercased once here, so the stored rules are already in the form they're compared in.

A patch replaces the **whole** rule list. Reordering, inserting and deleting rules are then all one operation, and two people editing the same splitter can't interleave half of each other's changes: the last save wins, whole.

Postgres: rules as jsonb

`api/src/db/schema.sql · lines 82–96`

```
-- Splitters: diverter gates on a belt, with their sorting rules. The rules are a list of
-- objects that the API validates and only the arm app interprets, so they're stored as one
-- jsonb value rather than spread over rule and condition tables. Deleting a belt deletes its
-- splitters (ON DELETE CASCADE).
CREATE TABLE IF NOT EXISTS splitters (
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name         text NOT NULL CHECK (length(name) BETWEEN 1 AND 40),
  conveyor_id  uuid NOT NULL REFERENCES conveyors (id) ON DELETE CASCADE,
  along        double precision NOT NULL CHECK (along BETWEEN 0 AND 4),
  rules        jsonb NOT NULL DEFAULT '[]' CHECK (jsonb_typeof(rules) = 'array' AND
jsonb_array_length(rules) <= 20),
  otherwise    text NOT NULL DEFAULT 'straight' CHECK (otherwise IN ('left', 'straight',
'right')),
  created_at   timestamptz NOT NULL DEFAULT clock_timestamp(),
  updated_at   timestamptz NOT NULL DEFAULT clock_timestamp()
);
CREATE INDEX IF NOT EXISTS splitters_conveyor ON splitters (conveyor_id);
```

The rules go in one `jsonb` column rather than `rules` and `conditions` tables. The database never queries inside a rule; only the arm app interprets them, and they're always read and written as a whole list. Splitting them into tables would add two joins and an ordering column to rebuild a value that's naturally a document. The `CHECK` still guards the outline: it must be an array of at most 20.

`REFERENCES conveyors (id) ON DELETE CASCADE` ties a splitter to its belt: delete the belt and its splitters go with it, in the same statement, and a splitter for a belt that doesn't exist can't be inserted.

api/src/db/postgres-sql.ts · lines 93–107

```
listSplitters: `
  SELECT id, name, conveyor_id, along, rules, otherwise, created_at, updated_at
    FROM splitters ORDER BY created_at, id`,
createSplitter: `
  INSERT INTO splitters (name, conveyor_id, along, rules, otherwise)
  VALUES ($1, $2, $3, $4::jsonb, $5)
  RETURNING id, name, conveyor_id, along, rules, otherwise, created_at, updated_at`,
updateSplitter: `
  UPDATE splitters
     SET name = coalesce($2, name), along = coalesce($3, along),
         rules = coalesce($4::jsonb, rules), otherwise = coalesce($5, otherwise),
         updated_at = clock_timestamp()
   WHERE id = $1
  RETURNING id, name, conveyor_id, along, rules, otherwise, created_at, updated_at`,
deleteSplitter: `DELETE FROM splitters WHERE id = $1 RETURNING id`,
```

api/src/db/postgres.ts · lines 211–235

```
async listSplitters(): Promise<Splitter[]> {
  const { rows } = await this.pool.query<SplitterRow>(Q.listSplitters);
  return rows.map(toSplitter);
}

async createSplitter(s: NewSplitter): Promise<Splitter> {
  // The driver turns jsonb into objects on the way out; on the way in it needs a JSON string.
  const { rows } = await this.pool.query<SplitterRow>(Q.createSplitter, [s.name, s.conveyorId,
s.along, JSON.stringify(s.rules), s.otherwise]);
  return toSplitter(rows[0]);
}

async updateSplitter(id: string, p: SplitterPatch): Promise<Splitter> {
  if (!UUID.test(id)) throw new NotFoundError(`No splitter ${id}`);
  const { rows } = await this.pool.query<SplitterRow>(Q.updateSplitter, [
    id, p.name ?? null, p.along ?? null, p.rules ? JSON.stringify(p.rules) : null, p.otherwise
?? null,
  ]);
  if (!rows[0]) throw new NotFoundError(`No splitter ${id}`);
  return toSplitter(rows[0]);
}
```

```
async deleteSplitter(id: string): Promise<void> {
  if (!UUID.test(id)) throw new NotFoundError(`No splitter ${id}`);
  const { rowCount } = await this.pool.query(Q.deleteSplitter, [id]);
  if (!rowCount) throw new NotFoundError(`No splitter ${id}`);
}
```

The `pg` driver parses `jsonb` into objects when reading, but needs a JSON string when writing, hence `JSON.stringify(s.rules)` and the `$4::jsonb` cast. `coalesce($4::jsonb, rules)` keeps the old rules when the patch doesn't include any.

MongoDB: no foreign keys

api/src/db/mongo.ts · lines 261–290

```
async deleteConveyor(id: string): Promise<void> {
  const r = await this.conveyors.deleteOne({ _id: id });
  if (r.deletedCount !== 1) throw new NotFoundError(`No conveyor ${id}`);
  // No foreign keys here: delete the belt's splitters ourselves. A splitter created for this
  // belt in the instant between the two deletes would be left pointing at nothing; the arm
  // app ignores splitters whose belt is gone, and Postgres's foreign key rules it out.
  await this.splitters.deleteMany({ conveyorId: id });
}

async listSplitters(): Promise<Splitter[]> {
  return (await this.splitters.find({}).sort({ createdAt: 1, _id: 1
}).toArray()).map(toSplitter);
}

async createSplitter(s: NewSplitter): Promise<Splitter> {
  const now = new Date();
  const doc: SplitterDoc = { ...s, _id: randomUUID(), createdAt: now, updatedAt: now };
  await this.splitters.insertOne(doc);
  return toSplitter(doc);
}

async updateSplitter(id: string, patch: SplitterPatch): Promise<Splitter> {
  const doc = await this.splitters.findOneAndUpdate({ _id: id }, { $set: { ...patch, updatedAt:
new Date() } }, { returnDocument: "after" });
  if (!doc) throw new NotFoundError(`No splitter ${id}`);
  return toSplitter(doc);
}
```

```
async deleteSplitter(id: string): Promise<void> {
  const r = await this.splitters.deleteOne({ _id: id });
  if (r.deletedCount !== 1) throw new NotFoundError(`No splitter ${id}`);
}
```

The document holds the rules as they are, nested lists and all. MongoDB has no foreign keys, so `deleteConveyor` deletes the belt's splitters itself, in a second step. The comment says what that costs: a splitter created in the instant between the two deletes would point at a belt that's gone. The arm app skips splitters whose belt is missing, so it's harmless here; in a system where it mattered, a transaction on a replica set would close the gap.

Routes: does it fit on the belt?

api/src/app.ts · lines 98–123

```
// ---- Splitters: diverter gates on belts, with their sorting rules ----
/** The gate has to be on its belt: between the belt's start (0) and its length. */
async function checkFits(conveyorId: string, along: number) {
  const belt = (await repo.listConveyors()).find((c) => c.id === conveyorId);
  if (!belt) throw new ValidationError([`No conveyor ${conveyorId}`]);
  if (along > belt.length) throw new ValidationError([`along must be at most ${belt.length} m,
the length of ${belt.name}`]);
}
route("GET", "/api/splitters", async () => [200, await repo.listSplitters()]);
route("POST", "/api/splitters", async ({ body }) => {
  const s = parseNewSplitter(body);
  await checkFits(s.conveyorId, s.along);
  return [201, await repo.createSplitter(s)];
});
route("PATCH", "/api/splitters/:id", async ({ params, body }) => {
  const patch = parseSplitterPatch(body);
  if (patch.along !== undefined) {
    const current = (await repo.listSplitters()).find((s) => s.id === params.id);
    if (!current) throw new NotFoundError(`No splitter ${params.id}`);
    await checkFits(current.conveyorId, patch.along);
  }
  return [200, await repo.updateSplitter(params.id, patch)];
});
route("DELETE", "/api/splitters/:id", async ({ params }) => {
  await repo.deleteSplitter(params.id);
  return [200, { deleted: params.id }];
});
```

“The gate is on its belt” involves two records, so it's checked in the route: find the belt, compare `along` with its length. A belt's length can never change (its patch refuses size), so the check can't be invalidated later. The error names the belt and its length: “along must be at most 1.6 m, the length of Sort belt”.

Editing parts, so the database can steer the sorting

api/src/domain/newPart.ts · lines 45–77

```
/** What can change about an existing part. Stock changes go through receive and consume, so
the ledger explains them. */
export type PartPatch = Partial<Pick<SeedPart, "name" | "category" | "location" |
"reorderLevel">>;

export function parsePartPatch(input: unknown): PartPatch {
  const b = (input && typeof input === "object" && !Array.isArray(input) ? input : {}) as
Record<string, unknown>;
  const problems: string[] = [];
  const patch: PartPatch = {};
  if (b.name !== undefined) {
    if (typeof b.name !== "string" || b.name.trim() === "" || b.name.trim().length > 80)
problems.push("name must be text up to 80 characters");
    else patch.name = b.name.trim();
  }
  if (b.category !== undefined) {
    if (!CATEGORIES.includes(b.category as Category)) problems.push(`category must be one of
${CATEGORIES.join(", ")}`);
    else patch.category = b.category as Category;
  }
  if (b.location !== undefined) {
    if (typeof b.location !== "string" || b.location.trim().length > 20)
problems.push("location must be text up to 20 characters");
    else patch.location = b.location.trim();
  }
  if (b.reorderLevel !== undefined) {
    const v = b.reorderLevel;
    if (typeof v !== "number" || !Number.isInteger(v) || v < 0 || v > MAX_QTY)
problems.push(`reorderLevel must be a whole number from 0 to ${MAX_QTY}`);
    else patch.reorderLevel = v;
  }
  const unknown = Object.keys(b).filter((k) => !["name", "category", "location",
"reorderLevel"].includes(k));
```

```
  if (unknown.length) {
    const stock = unknown.filter((k) => ["onHand", "reserved", "available"].includes(k));
    problems.push(stock.length ? "stock changes go through receive and consume, so the ledger
records them" : `can't change ${unknown.join(", ")}`);
  }
  if (!problems.length && Object.keys(patch).length === 0) problems.push("nothing to change:
send name, category, location or reorderLevel");
  if (problems.length) throw new ValidationError(problems);
  return patch;
}
```

Rules can test a part's category, location and stock, so changing a part changes where its items go. Stock already changes through receive and consume. This adds `PATCH /api/parts/{sku}` for the rest: name, category, location and reorder level. Stock fields are refused with a reason: a direct edit to `onHand` would bypass the ledger, and every change to stock must be explained by a movement.

api/src/db/postgres-sql.ts · lines 25–30

```
updatePart: `
  UPDATE parts
     SET name = coalesce($2, name), category = coalesce($3, category),
         location = coalesce($4, location), reorder_level = coalesce($5, reorder_level),
updated_at = now()
   WHERE sku = $1
  RETURNING sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at`,
```

One `coalesce` statement again. The category column has no `CHECK`, so with SQL you can give a part a category of your own, and write a splitter rule for it; the API's list of categories is for the forms.

Tested on every database

api/tests/contract.ts · lines 227–251

```
test("splitters: rules stored as written, updated whole, deleted with their belt", async () =>
{
  const belt = await repo.createConveyor({ name: "Sorter", length: 2, width: 0.4, height: 0.3,
speed: 0.3, position: { x: 0.8, z: 0 }, rotationY: 0 });
  const rules = [
    { conditions: [{ field: "part.category" as const, op: "eq" as const, value: "plate" }],
exit: "left" as const },
    { conditions: [{ field: "kind" as const, op: "in" as const, value: ["sphere", "cylinder"]
}, { field: "height" as const, op: "gt" as const, value: 0.1 }], exit: "right" as const },
```

```
  ];
  const s = await repo.createSplitter({ name: "Gate 1", conveyorId: belt.id, along: 1.2, rules,
otherwise: "straight" });
  assert.deepEqual([s.conveyorId, s.along, s.rules, s.otherwise], [belt.id, 1.2, rules,
"straight"]);
  // Reordering is a whole-list update; other fields stay.
  const swapped = await repo.updateSplitter(s.id, { rules: [rules[1], rules[0]], otherwise:
"left" });
  assert.deepEqual([swapped.rules.map((r) => r.exit), swapped.otherwise, swapped.along,
swapped.name], [["right", "left"], "left", 1.2, "Gate 1"]);
  assert.deepEqual((await repo.listSplitters()).map((x) => x.id), [s.id]);
  await repo.deleteConveyor(belt.id);
  assert.deepEqual(await repo.listSplitters(), []);
  await assert.rejects(repo.updateSplitter(s.id, { name: "x" }), NotFoundError);
  await assert.rejects(repo.deleteSplitter(s.id), NotFoundError);
});

test("updatePart changes details but never stock", async () => {
  const before = (await repo.getParts(["PL-200"]))[0];
  const p = await repo.updatePart("PL-200", { category: "gripper", location: "Z-9" });
  assert.deepEqual([p.category, p.location, p.name, p.onHand, p.reorderLevel], ["gripper", "Z-
9", before.name, before.onHand, before.reorderLevel]);
  assert.equal((await repo.getParts(["PL-200"]))[0].category, "gripper");
  await assert.rejects(repo.updatePart("NOPE-1", { location: "A" }), NotFoundError);
});
```

The splitter contract test stores two rules (one with two conditions and an `in` list) and expects them back exactly, swaps their order with a whole-list update, then deletes the belt and expects the splitter gone: that's `ON DELETE CASCADE` in Postgres, and the explicit delete in MongoDB and memory. The SQL was also run on a real PostgreSQL 16, including the `CHECK`s on the rule list and the exit, and the foreign key.

api/tests/api.test.ts · lines 159–191

```
test("splitters: validate rules, fit on the belt, patch, delete", async () => {
  const belt = (await call("POST", "/api/conveyors", { name: "Sort belt", length: 1.6, width:
0.4, height: 0.3, position: { x: 0.8, z: 0 } })).body;
  const rule = { conditions: [{ field: "color", op: "eq", value: " #E11D48 " }], exit: "left"
};
  const made = await call("POST", "/api/splitters", { name: "Reds", conveyorId: belt.id, along:
1, rules: [rule] });
  assert.equal(made.status, 201);
```

```
  assert.deepEqual([made.body.rules[0].conditions[0].value, made.body.otherwise], ["#e11d48",
"straight"]); // trimmed, lowercased; default exit

  const bad = await call("POST", "/api/splitters", { name: "Bad", conveyorId: belt.id, along:
1, rules: [
    { conditions: [{ field: "weight", op: "eq", value: 1 }], exit: "left" },
    { conditions: [{ field: "kind", op: "gt", value: "box" }], exit: "up" },
    { conditions: [], exit: "right" },
  ] });
  assert.equal(bad.status, 400);
  assert.deepEqual(bad.body.problems.map((p: string) => p.split(":")[0]), ["rule 1, condition
1", "rule 2", "rule 2, condition 1", "rule 3"]);
  const tooFar = await call("POST", "/api/splitters", { name: "Off the end", conveyorId:
belt.id, along: 2, rules: [] });
  assert.match(tooFar.body.problems[0], /at most 1.6 m/);
  assert.equal((await call("PATCH", `/api/splitters/${made.body.id}`, { along: 1.7 })).status,
400);
  assert.equal((await call("PATCH", `/api/splitters/${made.body.id}`, { conveyorId: belt.id
})).status, 400);
  const moved = await call("PATCH", `/api/splitters/${made.body.id}`, { along: 0.5, otherwise:
"right", rules: [] });
  assert.deepEqual([moved.body.along, moved.body.otherwise, moved.body.rules], [0.5, "right",
[]]);
  assert.equal((await call("DELETE", `/api/splitters/${made.body.id}`)).status, 200);
  assert.equal((await call("DELETE", `/api/conveyors/${belt.id}`)).status, 200);
});

test("parts: change details, not stock", async () => {
  const p = await call("PATCH", "/api/parts/MC-1", { category: "motor", location: "C-01-1" });
  assert.deepEqual([p.status, p.body.category, p.body.location], [200, "motor", "C-01-1"]);
  const stock = await call("PATCH", "/api/parts/MC-1", { onHand: 99 });
  assert.equal(stock.status, 400);
  assert.match(stock.body.problems[0], /receive and consume/);
  assert.equal((await call("PATCH", "/api/parts/MC-1", { category: "widget" })).status, 400);
  assert.equal((await call("PATCH", "/api/parts/NOPE-1", { location: "A" })).status, 404);
});
```

CHAPTER 5

Interview questions

Q1 **Why both PostgreSQL and MongoDB? Which would you choose?**

To show the same business rules on two consistency models. For this domain I'd choose Postgres: stock, reservations and orders are relational, constraints catch bugs, and multi-row transactions make all-or-nothing trivial. MongoDB fits better when records have varied shapes (a catalog where every part type has different attributes) or need to scale horizontally, and its single-document atomicity is enough when each operation touches one document.

Q2 **Two people reserve the last actuator at the same moment. Walk me through what happens on each database.**

Postgres: both call `inv_reserve`; the first locks the part row with `FOR UPDATE`, the second waits. The first checks, reserves and commits; the second then gets the lock, sees zero available and returns the shortage. MongoDB: both send a conditional `$inc`; the server applies them one at a time, the first matches the filter, the second no longer does and modifies nothing, so the saga undoes any earlier holds and reports the shortage. Either way, exactly one succeeds.

Q3 **What's a lost update, and how do you prevent it?**

Two transactions read the same value, each computes a new one, and the second write overwrites the first. Prevent it with a lock (`SELECT … FOR UPDATE`), a conditional or atomic update (`SET x = x - 1 WHERE x >= 1`), optimistic version checks, or a stricter isolation level with retries. Back it with a constraint so the bad state can't be stored at all.

Q4 **Two people add a part with the same SKU at the same moment. What stops a duplicate?**

The database's unique index, not a check in the code. A “does it exist?” query followed by an insert leaves a gap a second request can slip through. In Postgres the insert says `ON CONFLICT (sku) DO NOTHING`, and an empty `RETURNING` means someone else won; in MongoDB the SKU is the `_id`, so the second `insertOne` fails with duplicate-key error 11000. Both become a 409. The form's own “already exists” message is only a convenience: it can be out of date by the time the button is clicked.

Q5 **How do you avoid deadlocks?**

Lock resources in a consistent global order (here, by SKU), keep transactions short, and don't wait on user input inside one. Postgres detects deadlocks and aborts one transaction, so code should also be ready to retry, but with ordered locking it shouldn't happen.

Q6 **MongoDB has transactions. Why didn't you use one?**

Multi-document transactions need a replica set; the demo runs a standalone container to keep setup simple. Single-document atomic updates already prevent over-reservation; the saga adds all-or-nothing with compensation. On a replica set I'd wrap the same steps in `withTransaction`, which removes the window where a partial hold is visible.

Q7 **What does the repository pattern buy you, and what does it cost?**

Storage can change without touching routes or UI, tests can use an in-memory version, and the domain operations get names. The cost is a lowest common denominator: features one database is great at (Postgres window functions, Mongo aggregation pipelines) have to be expressed through the interface, or the interface grows database-specific methods.

Q8 **How do you know the three implementations really behave the same?**

One contract suite, nineteen tests, run against each: creating parts (including duplicate SKUs), filtering, ledger entries, consuming around reservations, all-or-nothing with every shortage reported, concurrent reservations and concurrent closes. A database-specific quirk shows up as one run failing.

Q9 **How is the database set up? Is there a migration step?**

The API does it on start: wait for the database (Compose healthcheck, plus a retry loop for Kubernetes), run an idempotent schema script, and seed only if the parts table is empty. That's fine for a demo and safe with several replicas. A production system would use versioned migrations (node-pg-migrate, Flyway) so schema changes are reviewed, ordered and reversible, and would run them once per deploy instead of from every pod.

Q10 **How does the API behave in Kubernetes when the database goes down?**

Liveness only checks the process, so pods aren't restarted in a loop. Readiness checks the database, so pods are taken out of the service and nginx gets errors quickly instead of hanging. When the database returns, readiness passes and traffic resumes. On first start, the API retries the connection for up to a minute instead of crashing.

Q11 **How would you scale this to thousands of SKUs and many warehouses?**

Paginate parts with keyset pagination (by SKU) instead of returning everything; index search with a trigram index (`pg_trgm`) instead of a scan; add a `warehouse_id` to stock rows so a part has a row per location; and move refreshes from polling to server-sent events. Reservation contention is per SKU, so it parallelises well across parts.

Q12 **The arm app runs on another port. How can it call your API, and is that safe?**

Browsers block cross-origin reads unless the server allows them with CORS headers. The API answers the OPTIONS preflight and adds `Access-Control-Allow-Origin` for pages on localhost by default, or an explicit list in production. It's safe in the sense that matters: CORS only controls what browsers let other sites' scripts read. It isn't access control, so a real deployment still needs authentication on the API.

Q13 **Why store a splitter's rules as jsonb instead of rule and condition tables?**

The database never queries inside a rule; rules are always read and written as one ordered list, and only the client interprets them. jsonb keeps that value whole, with a CHECK on its outline, and avoids two joins and an ordering column. If we needed to ask “which splitters test part.category?” across thousands of them, a jsonb path query or a GIN index would do it, and that's when I'd reconsider tables.

CHAPTER 5

Exercises

5.1 Reservations that expire

Give reservations an `expires_at` and release expired ones automatically, on both databases.

Hint. A periodic job calling `release` for each expired, still-active reservation reuses all the existing locking and ledger logic.

5.2 Mongo with a transaction

Run MongoDB as a one-node replica set in Compose and implement `reserve` inside `session.withTransaction`. Keep the contract tests green.

Hint. Start mongod with `--replSet rs0` and run `rs.initiate()` once from a healthcheck or init container. Pass `{ session }` to every operation inside the callback.

5.3 Idempotent reservations

Make reserving the same `quoteRef` twice return the existing reservation instead of creating a second one.

Hint. A unique index on `quote_ref` (Postgres) or `quoteRef` (Mongo), and handling the duplicate-key error by reading the existing reservation.

5.4 Reorder suggestions

Add `GET /api/reorder` returning, for every low part, how many to order to reach twice its reorder level, and show it on the stock page.

Hint. Write it once in the service from `listParts({ lowOnly: true })`, not in each repository: it's arithmetic on data every repository already returns.

5.5 Optimistic edits

Add editing of a part's name, bin and reorder level with optimistic concurrency: a `version` field, and 409 if it changed since the form loaded.

Hint. `UPDATE … SET …, version = version + 1 WHERE sku = $1 AND version = $2`; in Mongo, the same condition in the filter and `$inc: { version: 1 }`.

APPENDIX A

Running the projects

Every project needs Node.js 20 or newer. From `Portfolio/vention/`:

Project	Commands
`snack1-robot-arm`	`npm install` · `npm run dev` (http://localhost:5173) · `npm test` · `npm run build:gripper` regenerates the .glb
`snack2-control-panel`	`npm install` · `npm run dev` · `npm test` (IK and program tests) · `npm run build`
`snack3-conveyor-physics`	`npm install` · `npm run dev` · `npm test` · `npm run bench` (headless physics benchmark)
`snack4-refactor-kata`	`npm install` · `npm test` (spec + golden master) · `npm run typecheck` · `npm run demo`
`snack5-inventory`	`docker compose --profile postgres up --build` (or `--profile mongo`), then http://localhost:8081 · `docker compose --profile test run --rm --build api-test` runs the contract tests on both databases · without Docker: `cd api; npm install; npm test`

Snack 4's full commit history ships as `history/refactor-steps.bundle`: `git clone history/refactor-steps.bundle steps`, then `git log --stat` in the new folder.

APPENDIX B

Glossary

AABB
Axis-aligned bounding box: a box whose faces line up with the world axes, described by its center and half sizes or its min and max corners.

Accessor (glTF)
Says how to read a range of a bufferView as numbers: component type, element type (VEC3, SCALAR…), count, bounds.

ACID
Atomicity, consistency, isolation, durability: the guarantees a database transaction gives.

Bin packing
Packing items into the fewest fixed-size bins; cut lists into stock bars are an instance.

Broad phase
The cheap first pass of collision detection that finds pairs which might touch.

Buffer view (glTF)
A byte range within a buffer, usable directly as a GPU buffer.

Characterization test
A test that records what code currently does, to detect changes, rather than what it should do.

CHECK constraint
A rule on a table that the database enforces on every write, rejecting rows that break it.

Conditional update
An update whose filter contains the rule (“only if enough is available”), so check and write are one atomic step.

Contract test
One test suite run against every implementation of an interface, so they behave the same.

CORS
Cross-origin resource sharing: headers by which a server lets pages from other origins read its responses in a browser.

Counting sort
A non-comparison sort that buckets items by a small integer key in O(n).

CTE
Common table expression (WITH … AS): names a sub-query; in Postgres it can also chain data-changing statements.

Deadlock
Two transactions each waiting for a lock the other holds; avoided by locking in a consistent order.

Denavit–Hartenberg
A standard way to describe each joint of a robot arm with four parameters, giving one transform matrix per joint.

Diverter
A device on a conveyor that steers items off the line to one side, here a hinged flap.

Document database
A database storing self-contained JSON-like documents, which can embed arrays and sub-objects (MongoDB).

Draw call
One request from the CPU to the GPU to draw geometry with a given state. Each has fixed overhead.

Emissive
Light a material gives off regardless of the scene's lights; used to make the selected housing glow.

Event delegation
One listener on a container handling events from its children, identified by attributes, so re-rendered children need no new listeners.

First-fit decreasing
A bin-packing heuristic: sort items largest first, put each in the first bin with room.

Fixed time step
Advancing a simulation in equal small steps, independent of the display's frame rate, so results don't depend on it.

Foreign key
A column that must match a key in another table, so references are always valid.

Forward kinematics
Computing the tool's position from joint angles. Always one answer.

Frustum culling
Skipping objects whose bounding sphere is outside the camera's view.

Golden master
Recorded outputs of existing code for many inputs, compared against new code.

Idempotency key
A client-chosen key that makes retrying a request safe: the server returns the stored result instead of acting twice.

Impulse
An instant change in momentum, the solver's unit of correction.

Instancing
Drawing one geometry many times in one draw call, with per-instance data such as a matrix and color.

Interpolation (α)
Drawing a moment between the last two physics states, by the fraction α of a step left over.

Inverse kinematics
Computing joint angles that put the tool at a target. Zero, several or infinitely many answers.

Island
A group of bodies connected by contacts; they sleep and wake together.

Isolation level
How much concurrent transactions can see of each other (READ COMMITTED, REPEATABLE READ, SERIALIZABLE).

Jacobian
The matrix of partial derivatives of tool position with respect to joint angles; maps joint speeds to tool speed.

jsonb
PostgreSQL's binary JSON type: stores a document in a column, with operators and indexes to query inside it.

Kerf
The width of material a saw blade removes with each cut.

Kinematic body
A body moved by code, with a velocity, that pushes others but is never pushed itself.

Kinematic simulation
Motion set directly (positions and velocities) rather than computed from forces.

Kustomize overlay
A directory that builds on a base set of Kubernetes manifests, adding or changing resources.

Little-endian
Byte order with the least significant byte first; required by the glTF container.

Local frame
A coordinate system attached to an object, in which its own geometry is simple; points are transformed into it.

Lost update
A race where two transactions read the same value and one's write silently overwrites the other's.

moveJ / moveL
Robot moves that interpolate joint angles (curved tool path) or the tool position (straight line).

Narrow phase
The exact collision test on candidate pairs, producing contact normal and depth.

Normal
A unit vector perpendicular to a surface; drives lighting and, in physics, the direction of contact forces.

Normalization
Storing each fact once in its own table, related by keys, instead of repeating it.

Object pool
Preallocated objects or slots reused instead of created and destroyed.

ON DELETE CASCADE
A foreign-key option that deletes the referencing rows when the referenced row is deleted.

Origin
The scheme, host and port of a page (http://localhost:5173); browsers isolate pages by origin.

PBR
Physically based rendering: materials described by base color, metalness and roughness, lit consistently in any environment.

Preflight
The OPTIONS request a browser sends before a cross-origin request that isn't simple, asking whether it's allowed.

Pure function
A function whose output depends only on its inputs and which has no side effects.

Quaternion
Four numbers representing a 3D rotation, free of gimbal lock and easy to interpolate.

Raycasting
Intersecting a ray (here from the camera through the mouse) with scene geometry, for picking.

Re-parenting
Moving an object to a new parent in the scene graph; `attach` keeps its world pose, `add` keeps its local one.

Readiness probe
A Kubernetes check that decides whether a pod receives traffic; separate from liveness, which decides restarts.

Reconciliation
React's comparison of a new render's output with the previous one, to find the DOM changes to make.

Refactoring
Changing the structure of code without changing its behaviour, in small steps.

Replica set
A group of MongoDB servers holding the same data; required for multi-document transactions.

Repository pattern
Hiding storage behind an interface of domain operations, so the rest of the code doesn't know the database.

Ring buffer
A fixed array used as a queue by wrapping an index, so nothing is shifted or allocated.

Row lock
A lock on specific rows (SELECT … FOR UPDATE) that makes other writers wait until the transaction ends.

Rule engine
A small interpreter that evaluates conditions stored as data, instead of decisions written as code.

Saga
A multi-step operation made of individually atomic steps, with compensating steps to undo earlier ones on failure.

Scene graph
A tree of objects whose transforms compose from parent to child.

Seam
A place where code's behaviour can be observed or substituted without editing it, which makes testing possible.

Semi-implicit Euler
Integration that updates velocity first, then position from the new velocity; stable and cheap.

Sequential impulses
Solving contact constraints one at a time and repeating, a form of projected Gauss–Seidel.

Shock propagation
Treating the lower body of a vertical contact as immovable on a final, bottom-up pass, so stacks settle.

Singularity
A pose where the arm loses the ability to move the tool in some direction; the Jacobian loses rank.

Slop
Overlap allowed between resting bodies before position correction acts, to prevent jitter.

Smoothstep
The easing curve $3t^2 - 2t^3$: from 0 to 1 with zero slope at both ends.

Spatial hash
A uniform grid of cells stored in a hash table, for finding nearby objects in O(n).

Spiral of death
A slow frame causing more physics steps, making the next frame slower; avoided by capping steps.

StatefulSet
A Kubernetes workload with stable pod names and a persistent volume per pod, used for databases.

Structure of arrays
Storing each property of many entities in its own array, instead of one object per entity.

TCP
Tool center point: the point on the tool that motion is planned for; here, between the fingertips.

Tearing (React)
Different components showing different versions of external data in the same commit.

Tessellation
Converting exact surfaces (from CAD) into triangles for rendering.

Winding order
The order a triangle's corners are listed; counter-clockwise as seen marks the front face.